

Matemática & JSPlotly

Sumário

1. Introdução.....	2
1.1 A Ferramenta JSPlotly.....	2
2. Padrões de Mudança: como funções descrevem o comportamento do mundo...	2
2.1 Quatro comportamentos importantes.....	5
2.1.1 Função linear.....	5
2.1.2 Função quadrática.....	5
2.1.3 Função exponencial.....	5
2.1.4 Função logarítmica.....	6
2.1.5 Inequações.....	8
3. Geometria integrada: do plano ao espaço e às coordenadas.....	13
3.1 Geometria plana: como descrever a forma.....	17
3.2 Geometria espacial: como a forma ocupa espaço.....	18
3.3 Geometria analítica: como descrever por coordenadas.....	19
4. Probabilidade e Estatística: quando o acaso revela padrões.....	29
4.0.1 Probabilidade: chance, modelo e frequência.....	34
4.1 Estatística: resumir e interpretar dados.....	34
4.2 Da chance ao padrão.....	36
5. Quando o movimento se repete: trigonometria, periodicidade e números complexos.....	41
5.1 Números complexos.....	48
6. Crescimento, Tempo e Decisão: Matemática Financeira.....	55
Agite antes de usar.....	62
6.1 Estrutura de JS.....	71
7. Matrizes e modelagem - Transformando relações em sistemas manipuláveis...	72
Aritmética.....	89
Arrays.....	90
Uma palavrinha sobre métodos.....	91
JavaScript, uma linguagem orientada a objetos.....	92
8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança.....	95

1. Introdução

1.1 A Ferramenta JSPlotly

2. Padrões de Mudança: como funções descrevem o comportamento do mundo



O MUNDO TE CHAMA

Imagine uma cidade pequena com 10 mil habitantes. Ou uma megalópole com mais de 10 milhões ! A cada ano, a população cresce. Mas será que esse crescimento acontece sempre do mesmo jeito ? Em alguns casos, o aumento pode parecer constante, mas em outros ele parece acelerar com o tempo. Há situações também em que o crescimento desacelera, se estabiliza ou até mesmo diminui. Isso nos leva a perguntar como diferentes tipos de função podem ajudar a descrever tendências e comportamentos do mundo real, o que será abordado neste capítulo.

Depois de explorar, você consegue:

- reconhecer diferentes padrões de variação;
- relacionar situações reais com modelos matemáticos;
- comparar funções linear, quadrática, exponencial, logarítmica, bem como inequações;
- interpretar graficamente o comportamento dessas funções;
- usar visualizações interativas para explorar algumas funções.

Figura 2.1: O perfil de uma curva de crescimento populacional pode variar dependendo do contexto em que a população se encontre (taxas de natalidade, mortalidade, etc).



MEXA ANTES DE ENTENDER 



FAZENDO APARECER

Uma função é uma regra que associa valores de entrada a valores de saída. Chato desse jeito, né ? Então que ta...um função é uma equação que pega um x e o transforma em y . Menos técnico, mas mais digerível.

Bom, quando representamos essa relação em um gráfico, começamos a enxergar alguns padrões. Cada tipo de função expressa um comportamento diferente. Na função linear, a variação ocorre de forma constante. Nas demais funções do *app* o gráfico se mostra em curvas. Na função quadrática, a variação muda resultando numa curva côncava (parece “feliz”) o convexa (parece “triste”). Na função exponencial, o crescimento pode se intensificar rapidamente, de uma hora pra outra. Já na função logarítmica, o crescimento começa intenso e depois vai desacelerando. Essas diferenças representam formas distintas de mudança no mundo real, por exemplo, na tendência de crescimento populacional indagada na acima.

2.1 Quatro comportamentos importantes

2.1.1 Função linear

A função linear, também denominada *função afim*, relaciona-se a situações em que a taxa de variação é sempre constante. Sua equação é definida por:

$$y = ax + b \quad (2.1)$$

Nessa equação, o coeficiente a indica a inclinação da reta, e o valor b o ponto onde a reta intercepta o eixo vertical (também chamado de *eixo das ordenadas*, ou da *variável dependente*, ou comumente *eixo y*).

2.1.2 Função quadrática

As demais funções apresentadas neste capítulo variam conforme o valor aumenta no eixo horizontal (também denominado por *eixo das abscissas*, ou da *variável independente*, ou comumente como *eixo x*). Ilustrando, a função quadrática relaciona-se a situações em que a variação muda com o quadrado do tempo. Sua equação pode ser definida como:

$$y = ax^2 + bx + c \quad (2.2)$$

O gráfico resultante é uma parábola, e cuja *concavidade* depende do sinal de a . Teste no *app* da [Figura 2.2](#): valores positivos, concavidade pra cima (ou *perfil côncavo*); valores negativos, concavidade para baixo (ou *perfil convexo*).

2.1.3 Função exponencial

Esta função relaciona-se a processos em que o crescimento depende muito do valor atual, sendo escrita como:

$$y = a \cdot b^x \quad (2.3)$$

Isso acontece porque pequenas mudanças em x podem gerar grandes variações em y .

2.1.4 Função logarítmica

Essa função relaciona-se a processos em que o crescimento desacelera. Isso acontece porque o valor final varia depende da base b mostrada abaixo na [Equação 2.4](#). Por exemplo, se b for 10, então o termo $\log_b(x)$ fará com que grandes aumentos em x (10, 100, 1000) resultem em pouca diferença no resultado (1, 2, 3).

$$y = a \cdot \log_b(x) \quad (2.4)$$

Experimente junto ao *app* que o crescimento é rápido no início e depois se torna mais lento.

VOLTA PRO MUNDO



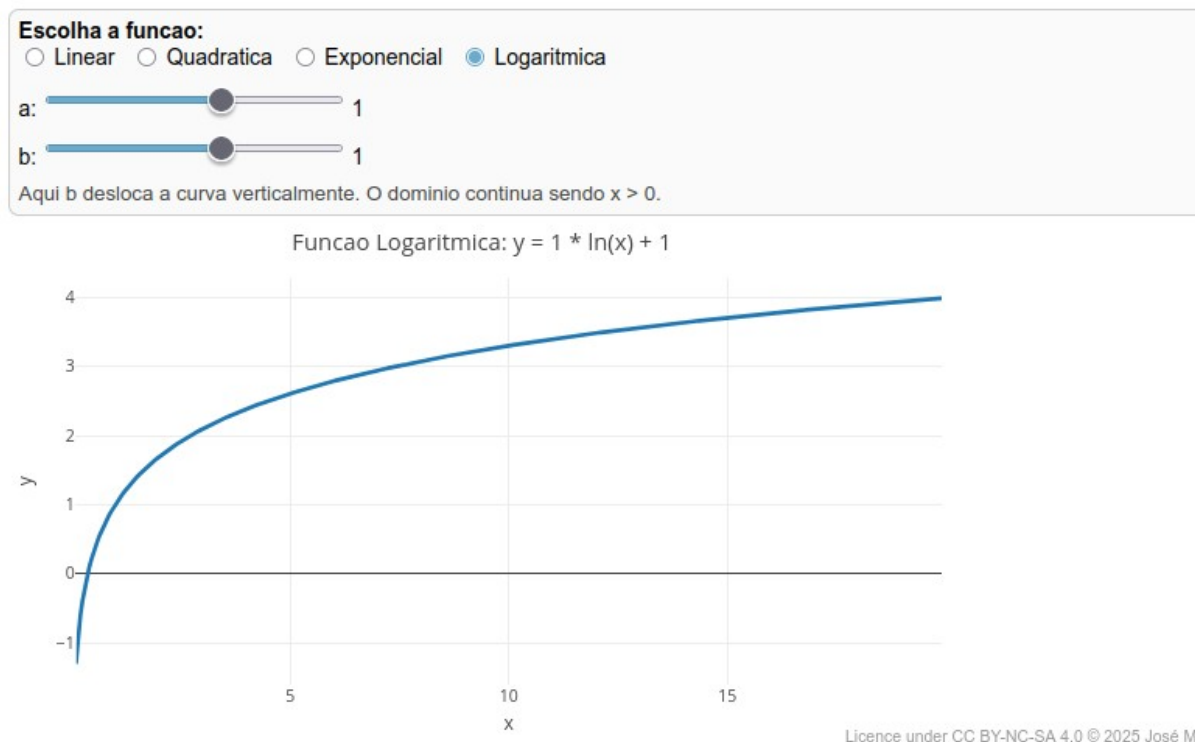
Vamos retomar o problema inicial. Se a população aumentasse sempre pela mesma quantidade, um modelo linear seria suficiente para explicá-la. Modelos são funções que descrevem comportamentos. Se o crescimento dependesse do número atual de habitantes, o modelo exponencial faria então mais sentido. Perceba que a escolha da função vai além de conhecer a função, mas sim o fenômeno que está sendo observado. Então a gente volta à questão que abre esse capítulo:

Qual comportamento descreve melhor uma situação real ?

O *app* da [Figura 2.2](#) é suficiente para explorar diversas possibilidades relacionadas às funções linear, quadrática, exponencial e logarítmica. Para isso, como você já deve ter percebido, basta alterar o tipo da função e seus valores, “rodando” em seguida o *app* pelo botão add. Mas também dá pra estudar outras relações de um modo mais integrado e...bom... arriscaria dizer...“divertido” !

Para uma exploração mais consistente, experimente o aplicativo *app* JSPlotly que segue. Da mesma forma que para o anterior, basta clicar no LINK que aparece na legenda da figura.

Figura 2.3: Aplicativo JSPlotly para explorar as funções deste capítulo de modo mais integrado. Para acessá-lo, como antes, basta clicar com o botão direito do mouse, ou na tela capacitiva, diretamente neste [LINK](#) para uma nova aba.



Agora a cena é outra ! Varie os valores de inclinação e intercepto da *função afim* pelas *barras deslizantes* (ou slider). Depois, selecione outras funções pelos *botões de rádio* (radio buttons). E tente modificar o perfil dos gráficos. Veja o que acontece com cada função quando você escolhe um parâmetro com valor zero (0) ou negativo. Essas mudanças mostram como os parâmetros alteram o comportamento das funções. Assim, você será capaz de responder algumas perguntas cruciais relacionadas ao



E SE...

Você verá essa seção ao longo do livro como a mais “*viajante*”. Isso porque a ideia aqui é explorar as possibilidades de variação para determinado tema, tanto as enraizadas nos conceitos que o envolverem, como as mais “*delirantes*”, também.

- E se a taxa de crescimento for zero ?
- E se for negativa ?
- E se a parábola abrir para baixo ?
- E se a exponencial crescer lentamente no início ?
- E se o crescimento desacelerar ao longo do tempo ?
- E se uma pedra for atirada...qual a função provável que descreverá sua trajetória ?
- E se a função precisar descrever um tobogã de parque aquático ?
- E se a função se parecer com uma serpente em S...tem equação para isso ?

2.1.5 Inequações

Existe um outro tipo de função que se comporta dependendo da faixa da variável independente. Essas funções são denominadas *inequações*. Quer um exemplo bem prático de inequações ? Em diversas situações no comércio, o preço de um produto muda em função da quantidade que é adquirida pelo comprador. Por exemplo, o preço do varejo (poucas unidades) e o do atacado (muitas unidades). Outra situação refere-se ao valor que se paga pelo uso de água encanada ou energia elétrica em seu município. Ele também depende de faixas de consumo, da econômica até a “*super esbanjadora*”.

Assim, as inequações aparecem quando queremos saber se o resultado depende de uma faixa determinada para valores de x , e não quando há uma expressão única que trabalha com esse x . Ou seja, saber quando uma expressão é maior, menor ou limitada por outra. Em vez de perguntar se:

$$f(x)=0?$$

A inequação pergunta se...

$$f(x)>0?$$

Ou também se...

$$f(x)<0?$$

E até mesmo se...

$$5<f(x)<10?$$

E isso muda a leitura de um gráfico, porque agora passa a importar os

intervalos em que a curva fica acima ou abaixo dos pontos que cruzam o eixo horizontal. Exemplificando para uma função linear, se:

$$f(x) = x - 2$$

O comportamento da função muda quando se estabelece uma inequação, tal como:

$$x - 2 > 0$$

Na prática, a relação acima equivale dizer que a expressão “*é verdadeira quando*”:

$$x > 2$$

Na elaboração de um gráfico, isso corresponde à parte da reta 2 unidades acima do eixo horizontal (se considerar esse iniciando na origem).

Pelo mesmo raciocínio, agora para uma *função quadrática*:

$$f(x) = x^2 - 4$$

A inequação torna-se:

$$x^2 - 4 < 0$$

E é verdadeira quando:

$$-2 < x < 2$$

No gráfico, isso corresponde ao trecho de uma parábola abaixo do eixo horizontal.

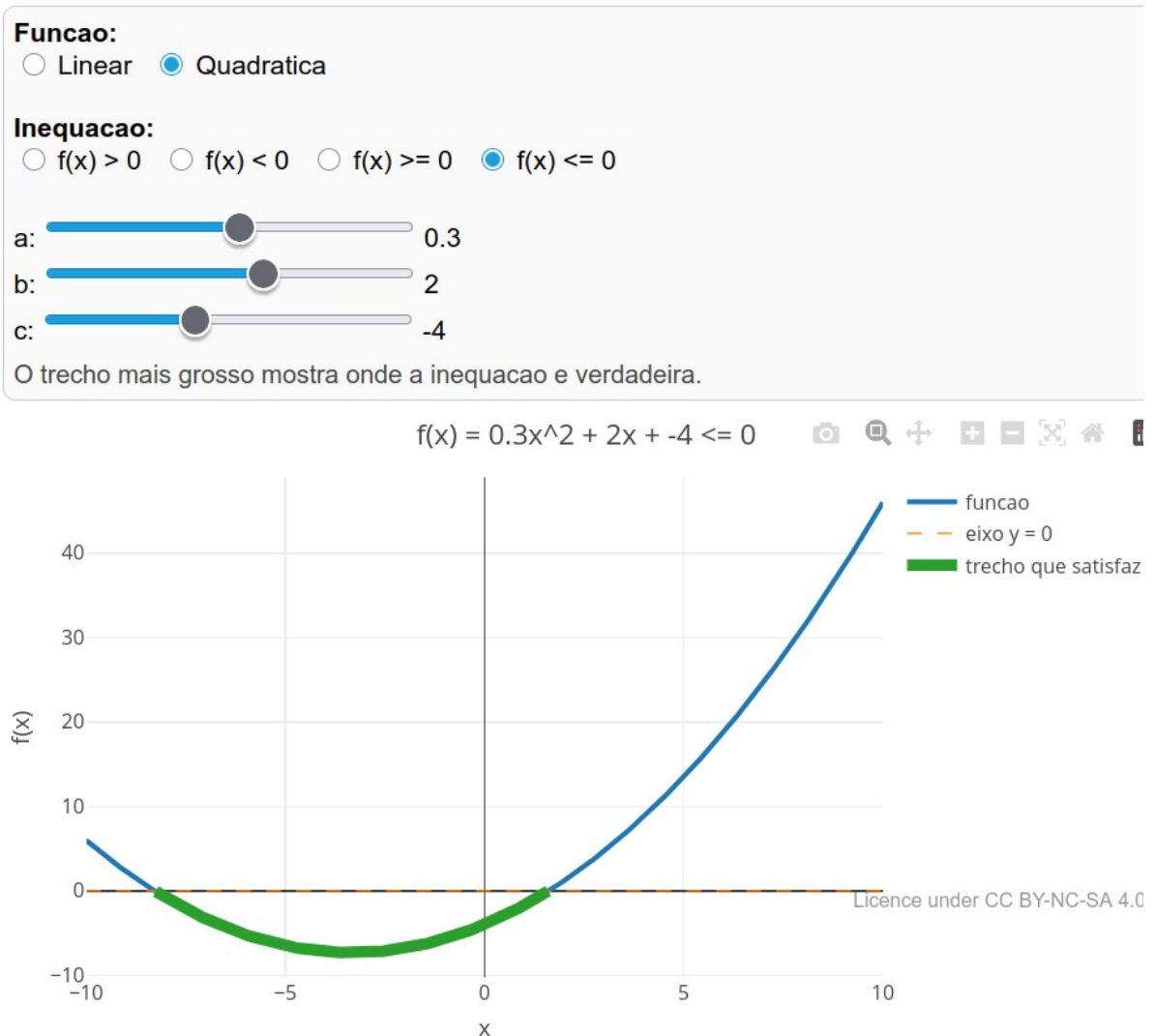
Resumindo, perceba que resolver uma inequação é descobrir em quais regiões do eixo x uma condição é satisfeita. Essas regiões podem ser à esquerda ou à direita de um ponto, entre dois pontos ou fora de um intervalo. Em alguns casos, também, a região pode abraçar todo o *domínio da função*.

Por que não testar isso na prática ?!

Aproveite o momento e use o *app* da [Figura 2.4](#) que segue para visualizar o efeito gráfico de inequações sobre uma função afim ou uma parábola.

Figura 2.4: Numa inequação o resultado depende da faixa escolhida. Para acessar o app, basta clicar neste [LINK](#) para uma nova aba.

2. Padrões de Mudança: como funções descrevem o comportamento do mundo



Agite antes de usar

Use o *app* para perceber que resolver uma inequação é localizar regiões do gráfico.

1. Escolha se deseja explorar uma função linear ou quadrática;
2. Escolha a condição da inequação: maior que zero, menor que zero, maior ou igual a zero, ou menor ou igual a zero;
3. Mova os controles a, b e c;
4. Observe o trecho mais grosso do gráfico. Esse trecho indica os valores de x para os quais a inequação é verdadeira.

Você também pode realizar alguns experimentos interessantes com o aplicativo, para sedimentar as ideias acima, veja:

1. Na função linear, coloque $a = 1$ e $b = -2$. Observe onde $f(x) > 0$;
2. Na função linear, coloque $a = -1$ e $b = 2$. Compare com o caso anterior;
3. Na função quadrática, use $a = 1$, $b = 0$, $c = -4$. Observe onde a parábola fica abaixo do eixo;
4. Troque para $f(x) < 0$ e veja como a solução muda;
5. Coloque $a = 0$ na função quadrática e observe como ela se aproxima de uma função linear.

MESMO PADRÃO, OUTROS MUNDOS



As funções apresentadas neste capítulo permeiam diversos contextos e áreas do conhecimento. Os mesmos padrões encontrados aqui também aparecem, por exemplo no custo de corrida com tarifa fixa (função afim), na trajetória de lançamento de objetos (função quadrática), no crescimento populacional e no de juros compostos (função exponencial), ou em processos de adaptação e saturação, como o volume sonoro medido em decibéis (função logartítmica).

POR DENTRO DA FERRAMENTA



Esse espaço é dedicado para se compreender um pouco a lógica de funcionamento de um *app* do capítulo, ilustrando-o para que você **pense computacionalmente** em como ele foi criado. O escolhido para esse capítulo é o da [Figura 2.2](#). O aplicativo segue um padrão simples. Abra-o novamente e compare as linhas de comando que lá estão com o padrão que segue.

Primeiro escolhe-se o tipo de função. Em seguida, define-se um intervalo de valores para a variável de entrada. Para cada valor de x , calcula-se o valor correspondente de y . Esses valores são armazenados e usados para gerar o gráfico.

Aqui faz-se uma observação importante sobre a linguagem envolvida, *JavaScript*. Perceba que os parâmetros da equação são apresentados de forma diferente que o conjunto de dados utilizado para construir os valores de x e de y das funções. Relembrando o código:

```
let c = 0;  
// Vetores  
const x_values = [];
```

Isso é chamado *declaração de variáveis* em linguagens de programação. Assim, `let` sinaliza para uma variável que tende a ser alterada, enquanto que `const` uma variável que tende a permanecer constante (os *vetores de dados* para x e y).

Resumindo a estrutura do código (um *pseudocódigo*):

1. Define-se o tipo de função;
2. Define-se os parametros;
3. Gera-se os valores de x ;
4. Para cada x :
 - a. calcula-se y ;
 - b. armazena-se os valores;
 - c. constroi-se o grafico

3. Geometria integrada: do plano ao espaço e às coordenadas

O MUNDO TE CHAMA

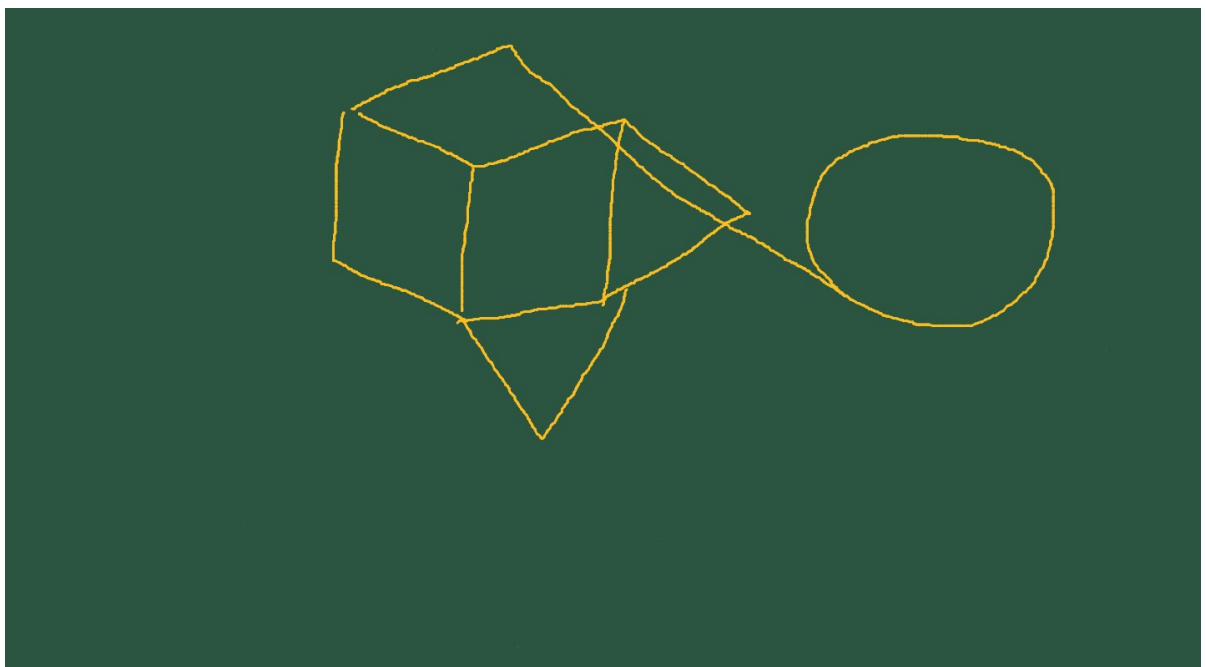


Uma caixa, um prédio, um copo, uma planta baixa, um mapa. Esses objetos são bastante distintos entre si, embora compartilhem em comum a forma. Formas são igualmente diferentes, umas se apresentando no *plano* e outras no *espaço*. Algumas parecem apenas um *contorno*, enquanto outras um *conjunto de pontos*, ou um *objeto* inteiro. Outra coisa: formas não são estáticas, podendo alterar-se por giro (*rotação*, *translação*), deslocamento, ou crescimento, entre outros.

Ilustrando, uma caixa de presente possui faces planas, ocupa espaço e pode ser descrita por algumas medidas. Já uma quadra esportiva pode ser vista como um retângulo no plano, mas também como um conjunto de pontos em um sistema de coordenadas. Um prédio pode ser desenhado por figuras simples, modelado como sólido e localizado por posições no espaço. Assim, uma forma pode mudar deixando de ser apenas um desenho, passando a ocupar o espaço, ou ainda ser descrita somente por coordenadas.

Esse é o objeto deste capítulo, que busca explicar o tronco da Geometria em três ramos principais: geometria plana, espacial, e analítica. A *geometria plana* permite reconhecer formas, medidas e relações em duas dimensões. A *geometria espacial* torna possível conceber como figuras planas podem gerar sólidos e volumes. E a *geometria analítica* propõe descrever essas formas por pontos, distâncias e posições no plano cartesiano.

Figura 3.1: O mundo não separa as geometrias, mas as integra entre forma, espaço e coordenadas.



Depois de explorar, você consegue:

- reconhecer como uma figura plana pode originar um sólido;
- relacionar área, volume e altura em construções geométricas;
- interpretar figuras por suas coordenadas no plano cartesiano;
- perceber que forma, medida e posição contam a mesma história geométrica;

MEXA ANTES DE ENTENDER

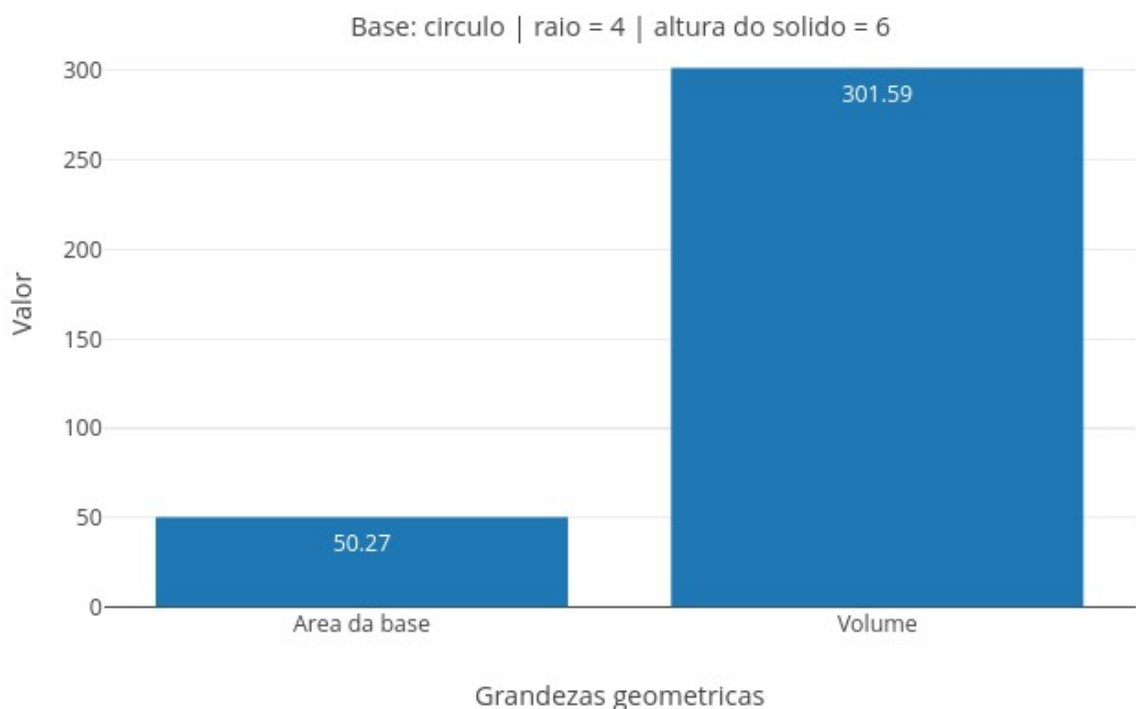


Embora seja comum estudar figuras planas, sólidos e coordenadas em momentos separados nos livros, no mundo real essas ideias aparecem juntas. Uma mesma forma pode ser vista como contorno no plano, transformada em objeto tridimensional, descrita por vértices, eixos, distâncias e equações. Essa integração aparece em plantas em 2D e construções em 3D da arquitetura e engenharia, em montagem de embalagens, nas coordenadas de um mapa geográfico, e na modelagem digital em jogos de videogame, por exemplo. Ou seja, o mundo não separa geometria plana, espacial e analítica. Somos nós que fazemos essa separação para compreender melhor como essas formas se apresentam no mundo.

A seguir é apresentado um JSPlotly que convida você a explorar essas conexões. Antes de qualquer definição formal, experimente-o. Sem pressa. Sem decorar fórmulas.

Figura 3.2: Aplicativo de JSPlotly para explorar base plana, sólidos e volumes. A área pertence à base plana, enquanto que o volume surge quando essa base ganha altura. Em muitos sólidos, o volume é dado pela relação área da base $\times$ altura. Clique com o botão direito do mouse neste [LINK](#) para abri-lo em uma nova janela, ou use a tela capacitiva de seu dispositivo móvel.

Da base plana ao solido: circulo -> cilindro



Agite antes de usar

O *app* mostra como uma *figura plana* pode gerar um *sólido geométrico* quando recebe uma *altura de extrusão* (ou de *camada*). Sem calcular nada ainda, observe o que acontece quando você muda valores simples ligados à forma escolhida e à altura do sólido gerado. Experimente variar o tipo de figura plana, as medidas da base, e a altura. E veja que o gráfico responde com área da base e volume do sólido. Verifique então o que muda no desenho plano e no sólido gerado.

Ao invés de decorar fórmulas, siga as instruções abaixo para alterar esses valores no código.

1. Escolha a forma da base. Altere a linha:

```
const forma = "circulo";
```

Você pode usar "quadrado", "retangulo", "circulo", e "triangulo".

2. Altere as medidas da base. Modifique:

```
const a = 4;
const b = 2;
```

O significado de a e b depende da forma escolhida:

- no quadrado, a é o lado;
- no retângulo, a e b são as dimensões da base;
- no círculo, a é o raio;
- no triângulo, a é a base e b é a altura do triângulo.

3. Altere a altura do sólido. Modifique:

```
const h = 6;
```

Essa altura transforma a figura plana em um sólido. Observe o gráfico de barras. O *app* mostra área da base e volume do sólido.

4. Compare diferentes formas. Teste, por exemplo:

- quadrado com lado 4;
- retângulo com base 4 e largura 2;
- círculo com raio 4;
- triângulo com base 4 e altura 2.

5. Teste a mesma base com alturas diferentes. Mantenha a forma e as medidas da base, mas altere h.

Perceba então o que pertence à geometria plana e o que passa a pertencer à geometria espacial. E tente responder às perguntas que seguem, mesmo sem ter estudado o conteúdo:

- A área da base muda quando apenas a altura do sólido muda ?
- Dois sólidos podem ter a mesma altura, mas volumes diferentes ?
- O volume pode mudar sem mudar a área da base ?

- Quais medidas crescem mais rapidamente ?
- O volume cresce mais por causa da base ou da altura ?
- Toda figura plana pode gerar um *sólido por extrusão*, ou seja, um objeto 3D criado por adição de uma 3a. dimensão a uma forma plana, criando um volume ?
- Se duas bases tiverem a mesma área, seus sólidos terão o mesmo volume para a mesma altura ?
- Aumentar a altura muda a área da base ?
- O que muda primeiro quando alteramos uma figura: o contorno, a área ou o volume do sólido associado ?
- A altura pertence à geometria plana, espacial ou ajuda a conectar as duas ?
- Quando colocamos uma figura no plano cartesiano, estamos mudando a forma ou apenas a maneira de descrevê-la ?
- Uma mesma forma pode parecer diferente e ainda conservar propriedades geométricas importantes ?
- Como localizar os *vértices* de uma figura usando *coordenadas* ?



Na *geometria plana*, uma figura pode ser descrita por suas medidas: lados, ângulos, área. Ao adicionar uma altura, surge a *geometria espacial*, um sólido, no caso. Existem diversos tipos de sólidos, sendo bem comuns o cubo, que surge de um quadrado, o paralelepípedo, que surge de um retângulo, ou o prisma triangular, que surge de um triângulo.

Em adição, cada ponto da figura pode ser representado por coordenadas (x, y) , e que também são apresentadas no centro da figura. Alterar essas coordenadas resulta no deslocamento da figura, enquanto que girar a figura muda sua orientação, mas não sua forma. É disso que trata a *geometria analítica*, que permite descrever a forma sem depender apenas do desenho.

3.1 Geometria plana: como descrever a forma

A geometria plana estuda figuras em duas dimensões. Nela aparecem comprimentos, perímetros, áreas, ângulos e relações entre lados.

Triângulos, retângulos, quadrados, paralelogramos, trapézios e circunferências são exemplos clássicos desse universo. Quando mudamos uma medida em uma figura plana, não alteramos apenas seu tamanho. Alteramos também relações como área, inclinação, diagonal e simetria. As equações abaixo exemplificam como calcular a área (A) e o perímetro (P) para algumas figuras planas comuns.

Triângulo:

$$A = \frac{b \cdot h}{2}$$

$$P = a + b + c$$

Quadrado

$$A = l^2$$

$$P = 4l$$

Retângulo

$$A = b \cdot h$$

$$P = 2(b + h)$$

Paralelogramo

$$A = b \cdot h$$

Losango

$$A = \frac{D \cdot d}{2}$$

Trapézio

$$A = \frac{(B + b) \cdot h}{2}$$

Círculo

$$A = \pi r^2$$

$$C = 2\pi r, \text{ circunferência}$$

3.2 Geometria espacial: como a forma ocupa espaço

Quando uma figura plana ganha profundidade, surge um sólido. Um retângulo pode gerar um paralelepípedo, enquanto que um círculo pode gerar um cilindro, e um triângulo, um prisma triangular. Nesse momento, a geometria deixa de tratar apenas de superfície e passa a considerar também volume, faces, arestas e vértices. Nesse caso, a área total do objeto é multiplicada a partir de sua base, surgindo o volume. As relações abaixo ilustam o volume e área de superfície para alguns sólidos.

Cubo

$$V = a^3$$

$$A = 6a^2$$

Paralelepípedo (Prisma Retangular)

$$V = a \cdot b \cdot c$$

$$A = 2(ab + bc + ac)$$

Prisma (geral)

$$V = A_b \cdot h$$

Pirâmide

$$V = \frac{A_b \cdot h}{3}$$

Cilindro

$$V = \pi r^2 h$$

$$A = 2\pi r(h + r)$$

Cone

$$V = \frac{1}{3} \pi r^2 h$$

$$A = \pi r(r + g)$$

Esfera

$$V = \frac{4}{3} \pi r^3$$

$$A = 4\pi r^2$$

3.3 Geometria analítica: como descrever por coordenadas

Uma figura também pode ser representada por pontos em um sistema cartesiano. Nesse caso, cada vértice passa a ter coordenadas, e relações geométricas podem ser estudadas por distâncias, inclinações, equações e transformações. Assim, uma mesma forma pode ser vista de três maneiras complementares: como *desenho e medida*, como *objeto no espaço*, e como um *conjunto de pontos e relações algébricas*.

Observando as fórmulas para volume de sólidos acima, você seria capaz de responder, por exemplo:

- se dobrar h dobra-se o volume ?
- no círculo, o que acontece quando o raio dobra ?
- qual forma gera maior volume com mesma altura ?
- o triângulo tem sempre volume menor que o retângulo com mesma base e mesma altura da base ?

Bom, relaxe ! Nos próximos passos, essa história se tornará mais paupável.

VOLTA PRO MUNDO

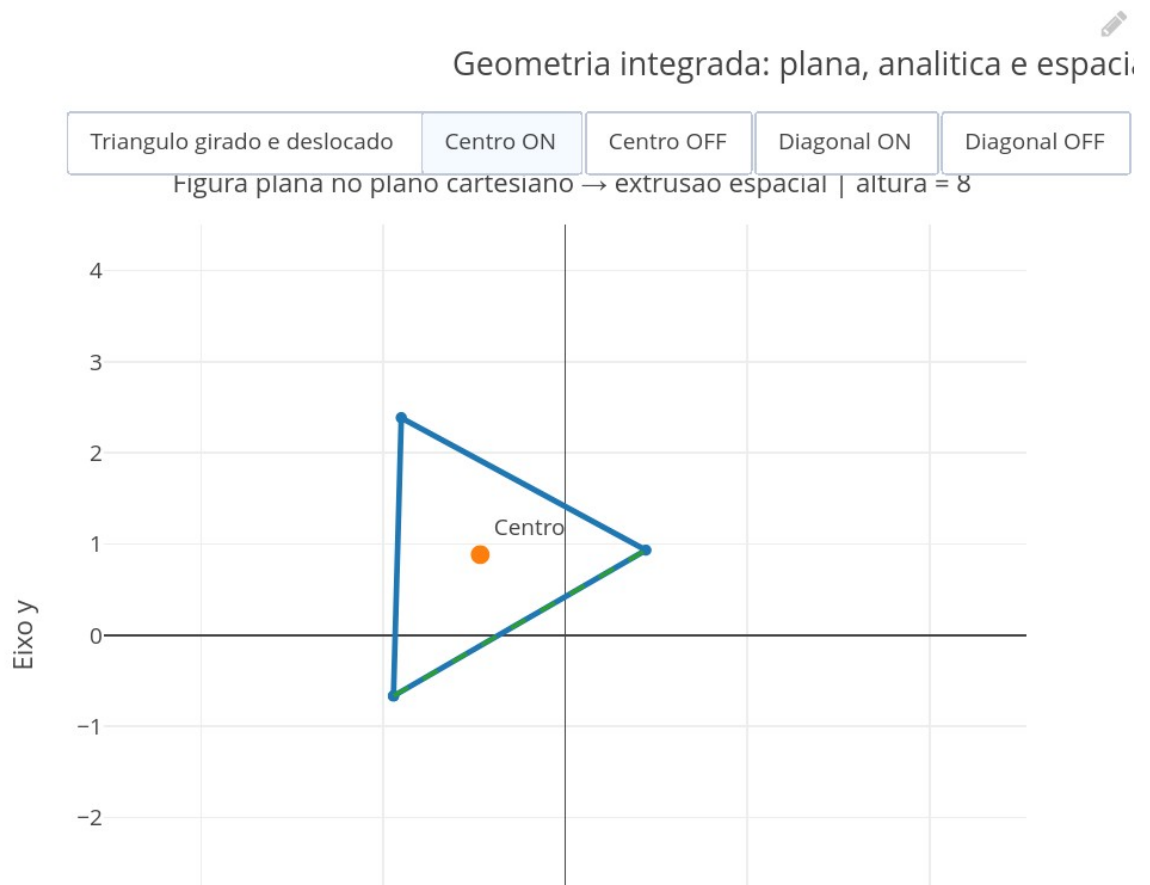


Volte às situações iniciais. Uma planta baixa usa geometria plana para representar paredes, portas e pisos. Uma maquete usa geometria espacial para transformar esse desenho em volume. Um programa de desenho técnico usa geometria analítica para localizar cada ponto e cada segmento com precisão. E adivinhe só o que fazem os jogos de videogame ?!! O que diferencia esses casos não é o objeto em si, mas o modo de descrevê-lo. Assim, se a forma é observada como contorno, estamos mais próximos da geometria plana. Se por outro lado a forma ganha profundidade, entramos na geometria espacial. E se a forma é descrita por coordenadas, então usamos a geometria analítica.

Hora de ir além !!!

Agora experimente o JSPlotly customizado abaixo para integrar esses conhecimentos para geometria plana, espacial, e analítica.

Figura 3.3: Aplicativo JSPlotly para estudo de geometria. Formas em contorno (plana), sólido (espacial), e pontos no plano (analítica). Clique neste [LINK](#) para uma nova aba para “rodá-lo”.



Agite antes de usar

Agora você pode “brincar” à vontade com figuras planas e sólidos !! O app conecta geometria plana com analítica, espacial, transformação, e volume. Você pode escolher a figura da base, alterar a altura de extrusão, e deslocar a figura no plano em x e y. E também pode girar a figura, mostrar ou esconder vértices, seu centro, e diagonal principal, bem como comparar a base no plano com a ideia de sólido gerado. Abra o app e

explore-o livremente, com calma. Troque o cenário (forma, posição, rotação), altere a altura, ligue e desligue o centro e a diagonal.

O *app* integra três dimensões da geometria: plana (formas no plano cartesiano), analítica (coordenadas, distâncias, centroide), e espacial (volume por extrusão). Tudinho em um único ambiente manipulável.

O script possui três níveis de interação:

- Menu *dropdown* (forma) - escolhe a base geométrica;
- Botões - liga/desliga centro, e liga/desliga diagonal
- Barra deslizante (*slider*) - controla a altura, gerando a animação

Você começa escolhendo uma forma plana (quadrado, retângulo ou triângulo). Essa forma pode ser *rotacionada* (giro), *transladada* (arraste), ter sua *área calculada automaticamente*, ou gerar um *sólido ao ser “esticada” na altura*. Traduzindo:

$$\text{Volume} = \text{Área da base} \times \text{Altura}$$

1. Ao rodar o script escolha uma forma no menu (Quadrado, Retângulo, Triângulo).
2. Observe os vértices no plano, o centro da figura, e a diagonal (ou lado no triângulo).
3. Use a barra deslizante (*slider*) para controlar a altura da extrusão.

Procedendo dessa forma, você verá que a área permanece constante, mas o volume cresce proporcionalmente. Isso ocorre porque o script usa alguns conceitos da geometria para transformações geométricas. Cada ponto da figura sofre uma rotação e uma translação. Essas transformações são aplicadas pelo seguinte trecho do código:

```
rotacionarPonto(x, y, ang)
transformarPontos(pontos, ang, dx, dy)
```

Área:

A área é calculada automaticamente pelos vértices:

```
areaPoligono(pontos)
```

Cálculo da área:

A área é calculada automaticamente pelos vértices:

```
areaPoligono(pontos)
```

Matematicamente:

$$A = \frac{1}{2} \left| \sum x_i y_{i+1} - x_{i+1} y_i \right|$$

Nessa equação, “ Σ ” é o símbolo para somatória.

Centroide:

O centro da figura é obtido pela média dos pontos:

```
centroideSimples(pontos)
```

Distâncias:

Para a diagonal:

```
distancia(p1, p2)
```

Volume por extrusão:

A mágica final:

```
volume = area * h
```

Ou seja:

$$V = A_b \cdot h$$

Também é possível explorar alguns contextos com o aplicativo da [Figura 3.3](#). Assim você verá com seus próprios olhos como pontos viram formas e que, quando transformadas, ocupam espaço e volume.

O que acontece quando mudamos a figura, sua posição, sua escala, sua rotação e sua altura?

1. *Área constante, volume variável:* fixe a forma e varie a altura. Perceba que a área não muda, mas o volume cresce linearmente.
2. *Comparando formas:* Troque o quadrado pra triângulo. Com essa mesma altura, qual gera maior volume ?
3. *Transformação sem mudança física:* Imagine rotacionar ou transladar a figura. E observe que embora a posição mude, sua área e volume não se alteram.
4. *Coordenadas e distâncias:* Observe as coordenadas dos vértices, o centro e a diagonal. Isso é geometria analítica.

Perceba também a diferença entre os objetos interativos da [Figura 3.2](#) e da [Figura 3.3](#). No primeiro você escolhe a forma, algumas medidas e o sólido correspondente. Nesse segundo você estabelece uma figura no plano cartesiano, e explora sua transformação geométrica, extrusão, área, perímetro, volume, vértices, diagonais e centro geométrico.

Se desejar avançar um pouco mais, utilize (ou não) os aplicativos de JSPlotly, e tente responder as questões que seguem.

1. Ao transformar uma figura plana em sólido, o que surge além da área ?
2. Como uma figura no plano pode gerar um sólido no espaço ?
3. Para que servem as coordenadas ao estudar uma figura geométrica ?
4. Uma translação altera a forma ou apenas a posição da figura ?
5. A rotação muda a forma ou apenas sua orientação ?

6. O que permanece invariável quando uma figura se move (translação ou rotação) ?
7. Como área, volume e distância aparecem juntos em uma construção geométrica ?
8. Como o volume depende da área da base e da altura ?
9. É possível manter a área e obter volumes diferentes ?
10. Dois sólidos com mesmo volume precisam ter a mesma forma ?
11. O que significa altura zero do ponto de vista geométrico ?
12. Como as coordenadas dos vértices mudam após uma transformação ?
13. Por que descrever uma figura por coordenadas não elimina sua interpretação visual ?
14. Como a diagonal se comporta em diferentes figuras ?
15. Qual elemento é mais determinante: base, altura ou posição no plano ?



E SE...

Se você acha que as perguntas acima já exauriram o tema...engana-se !!! Geometria é um campo aberto sem fim ! São múltiplas as divagações possíveis nessa área, como as exemplificadas abaixo.

- E se a altura aumentar e a base permanece fixa ?
- E se a altura do sólido for zero ?
- E se o formato permanecer o mesmo, mas as coordenadas mudarem, o que isso revela sobre a geometria ?
- E se a figura for transformada, o seu centro sofre alteração ?
- E se aumentarmos a escala e a altura, o que acontece quando girarmos a figura ?
- E se duas figuras planas tiverem a mesma área, mas formatos diferentes, o que isso sugere sobre os sólidos obtidos ao extrudá-las com a mesma altura ?
- E se um sólido tiver a altura dobrada e a base mantida, o que acontece com o volume ?
- E se duas bases ocupam posições diferentes no plano, mas têm mesma forma e mesma área, o que isso revela sobre a diferença entre localização e medida ?
- E se uma figura plana for ampliada, rotacionada e extrudada ao mesmo tempo, o que acontece com a figura final ?
- E se eu mover uma figura no plano, estou mudando a geometria da forma ou apenas sua localização ?

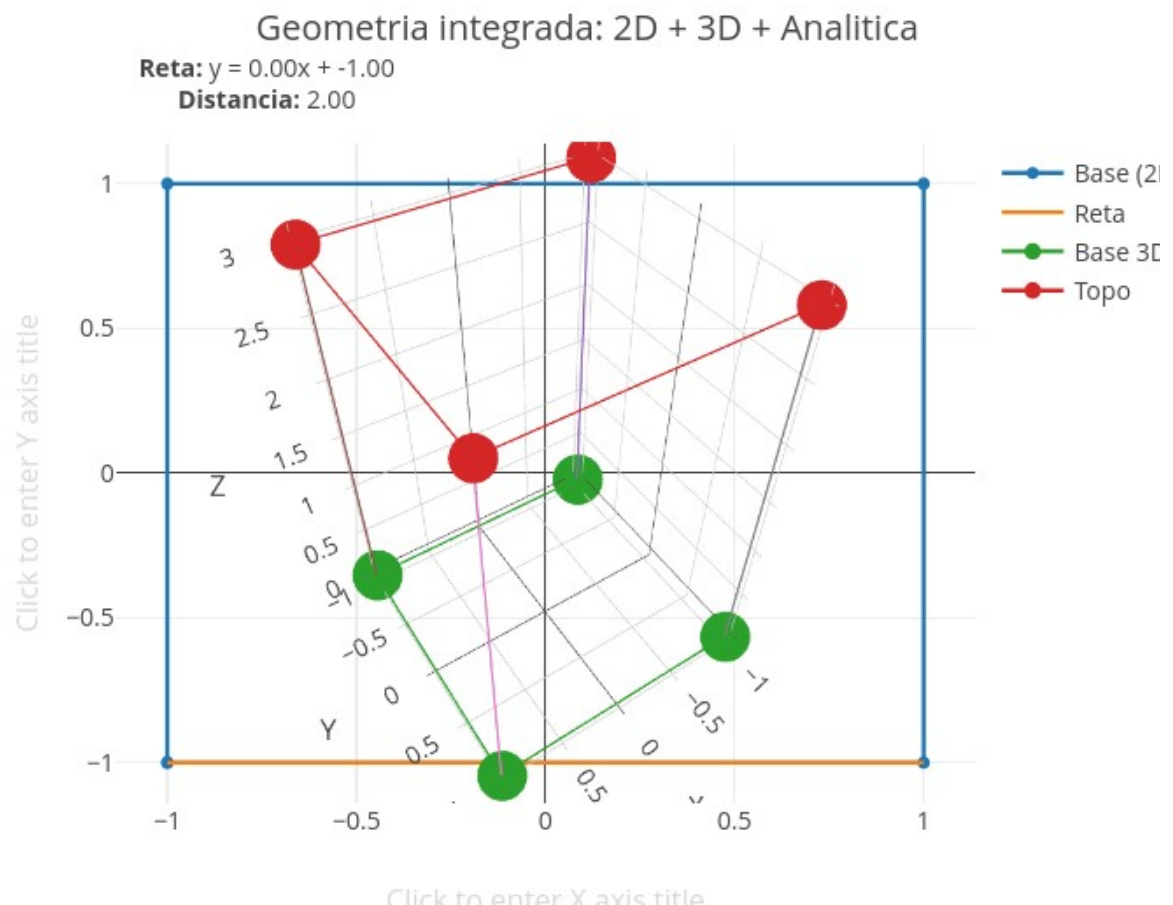


MESMO PADRÃO, OUTROS MUNDOS

A mesma lógica geométrica apresentada no capítulo também aparece em diversos contextos. Ilustrando, em plantas baixas e maquetes arquitetônicas, embalagens, caixas e recipientes, modelagem 3D e impressão tridimensional, mapas, quadras e terrenos no plano cartesiano, desenho técnico, engenharia e design, bem como em jogos, interfaces e ambientes virtuais.

Nesse sentido, faz-se interessante contemplar uma aprendizagem adicional, desta vez para objetos em 3D. Alguns cálculos para esses objetos são comumente utilizados na academia e indústria, como representar uma reta entre dois pontos, calcular a distância entre dois vértices, ou escrever uma figura apenas por equações. Para valorizar seu aprendizado, segue outro *script* de JSPlotly para um *app* autônomo visando uma abordagem da geometria espacial 3D.

Figura 3.4: JSPlotly ao estudo de geometria espacial 3D. A mesma estrutura matemática funcionando em múltiplas representações (gráfico, numérica, algébrica, e espacial). Abra o app clicando neste [LINK](#) com o botão direito para “brincar” em uma nova aba.



O script explora a geometria em 2D, 3D, e equações correlatas, tudo ao mesmo tempo. Agora você pode escolher a forma da base, ver a base no plano (2D), ver o sólido gerado (3D), visualizar vértices, a reta entre dois pontos, a distância entre pontos, bem como alterar a altura (extrusão).

E como prêmio por sua dedicação ímpar, você recebe geometria plana (forma de base e vértices), geometria analítica (equação da reta, distância

entre pontos, e interpretação geométrica da reta), e geometria espacial (extrusão real em 3D, sólido visível, e relação base × altura). Ufa !

Agite antes de usar

Como o *app* da [Figura 3.4](#), você começa com uma figura no plano. A partir dela, o script constrói uma reta entre dois pontos, calcula a distância entre esses pontos, e cria um sólido tridimensional gerado por extrusão. A relação matemática para distância e volume por detrás disso é bem clássica (Distância Euclidiana):

$$\text{Distância} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\text{Volume} = \text{Área da base} \cdot \text{Altura}$$

Ao rodar o script, observe três camadas: plano (2D; figura base e uma reta ligando dois vértices), álgebra (equação da reta e distância entre pontos), e espaço 3D (aparecimento do sólido). Parece chique mas, no fundo, é a mesma estrutura em três linguagens diferentes. Veja como acontece:

1. *Construção da figura*. Os pontos são definidos como coordenadas da geometria analítica:

```
const bases = {
  quadrado: [[-1, -1], [1, -1], [1, 1], [-1, 1]]
};
```

2. *Reta entre dois pontos*.

```
equacaoReta(p1,p2)
```

Se a reta não for vertical:

$$y = mx + b$$

Mas se a reta for vertical:

$$x = c$$

3. *Distância*.

```
distancia(p1,p2)
```

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

4. *Construção do sólido*. O script duplica a base no eixo z:

```
z = 0    → base
z = altura → topo
```

E conecta:

```
linhasLaterais
```

Isso gera a extrusão.

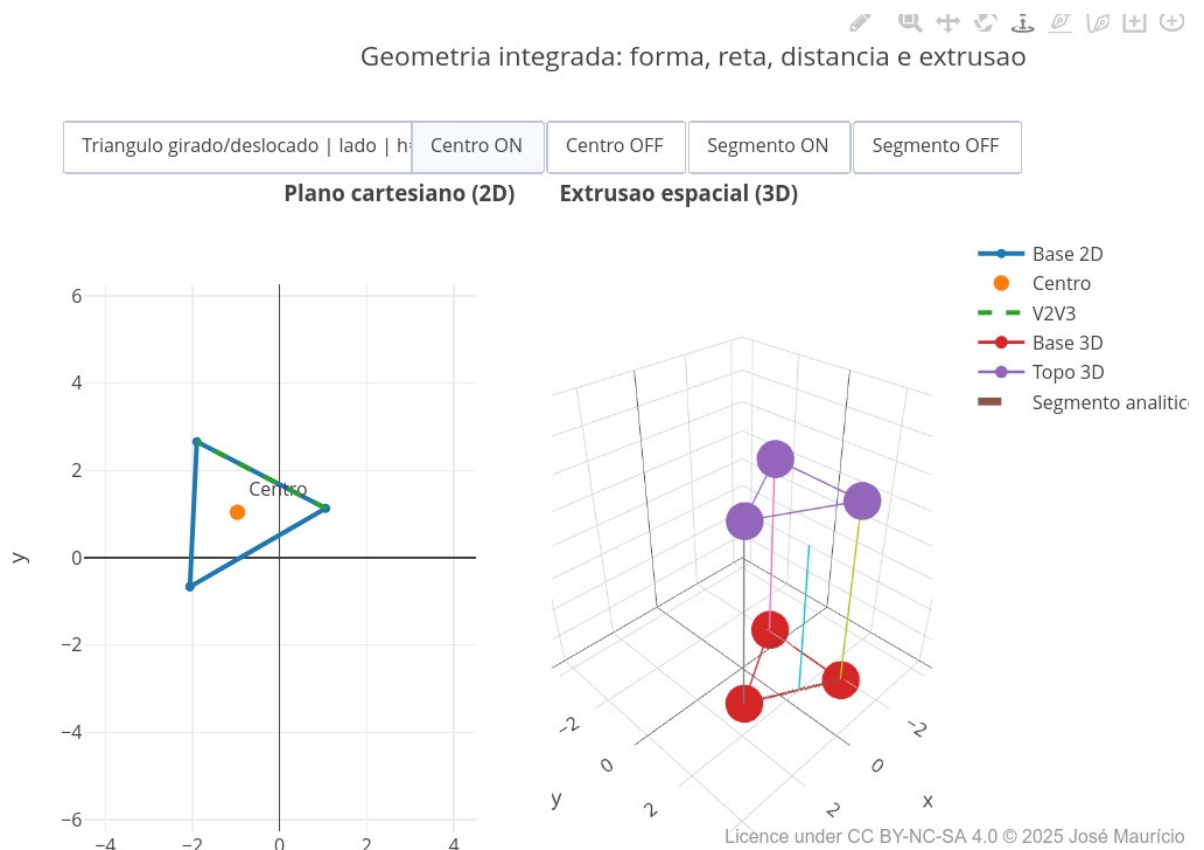
Então observe que a reta pertence ao plano. Além disso, o sólido nasce do plano. E a distância continua válida mesmo no espaço. Agora procure modificar o script para buscar cenários diferentes, como abaixo.

1. *Mudança de forma.* Troque quadrado para triângulo, e desse para retângulo, e observe como muda a equação da reta.
2. *Distância fixa vs forma variável.* Compare lados em diferentes figuras, e perceba que a distância depende dos pontos, não da forma inteira da figura.
3. *Plano vira espaço.* Observe a base 2D e o sólido 3D, e verifique o que muda e o que permanece.
4. *Reta no espaço* (conceito implícito). A reta está no plano, mas poderia existir no 3D. Isso está além deste texto, mas diz respeito ao estudo de vetores e de parametrização.

Exercitando-se no *app* você perceberá uma conexão direta entre campos distintos da Matemática, como álgebra(equação de reta), geometria (forma e distância), espaço (volume), e...por que não dizer... computação (construção dinâmica de figuras 2D e 3D).

E se estiver gostando da “*brincadeira*”, que tal um último script, na base do “*tudo junto e misturado*”, com geometria espacial 3D e analítica embutida ?

Figura 3.5: JSPlotly para geometria espacial 3D, geometria plana e geometria analítica. Três formas de descrever a mesma realidade. Clique neste [LINK](#) para acessá-lo.



Com esse último script é possível criar cenários aproximados aos de situações do cotidiano. Por exemplo:

1. *Crie uma embalagem.* Uma caixa começa como uma forma plana. Observe a base no plano cartesiano, aumente a altura, e relacione com embalagens do cotidiano. Como a área da base e a altura influenciam o volume da embalagem ?
2. *Arquitetura e construção.* Uma planta baixa é uma representação 2D de um objeto 3D. Assim, escolha um retângulo, observe seus vértices, e compare com o sólido gerado.
3. *Mapas e localização.* No plano cartesiano, cada ponto representa uma posição. Observe as coordenadas dos vértices de uma figura de sua escolha, desloque mentalmente a figura e a compare com a ideia de um mapa.
4. *Engenharia e medidas.* Segmentos representam distâncias reais. Observe um segmento destacado. Compare com outros segmentos em diferentes cenas.
5. *Modelagem digital (3D).* Objetos digitais são construídos a partir de pontos. Observe um sólido no painel 3D e relacione com softwares de modelagem.-

Enfim, perceba que, em todos os casos, estamos vendo que uma forma pode ser descrita, medida e transformada. A geometria plana fornece a base, a espacial mostra a ocupação do espaço, e a analítica organiza tudo por coordenadas.



Nos 4 scripts do capítulo, a ideia central foi a mesma: partir de uma base plana e observar como ela pode ser descrita, transformada e estendida. No primeiro script, você altera diretamente variáveis simples no editor. Isso ajuda a compreender a lógica do código sem excesso de recursos, observando de forma clara a relação entre o tipo de figura, as medidas da base, altura do sólido, a área e o volume.

No segundo script entram controles mais ricos, como menu, botões, controle deslizante (*slider*) e caixa de checagem (*checkbox*). Em ambos os casos, o objetivo não foi apenas desenhar figuras bonitas, mas transformar uma pergunta geométrica em modelo visual e manipulável.

Essa mesma semente pode ser reaproveitada em vários contextos, como arquitetura, design, modelagem, física, computação gráfica e problemas de localização.

No script da [Figura 3.4](#), por exemplo, *app* busca integrar três ideias: uma forma plana, definida por vértices, uma descrição analítica, baseada em coordenadas, e uma interpretação espacial, obtida pela extrusão (altura).

Internamente, a *lógica de programação* envolvida é:

- a figura é construída a partir de pontos (x, y) ;
- transformações como rotação e translação alteram esses pontos;
- a área da base é calculada;
- o volume surge ao multiplicar pela altura.

4. Probabilidade e Estatística: quando o acaso revela padrões



O MUNDO TE CHAMA

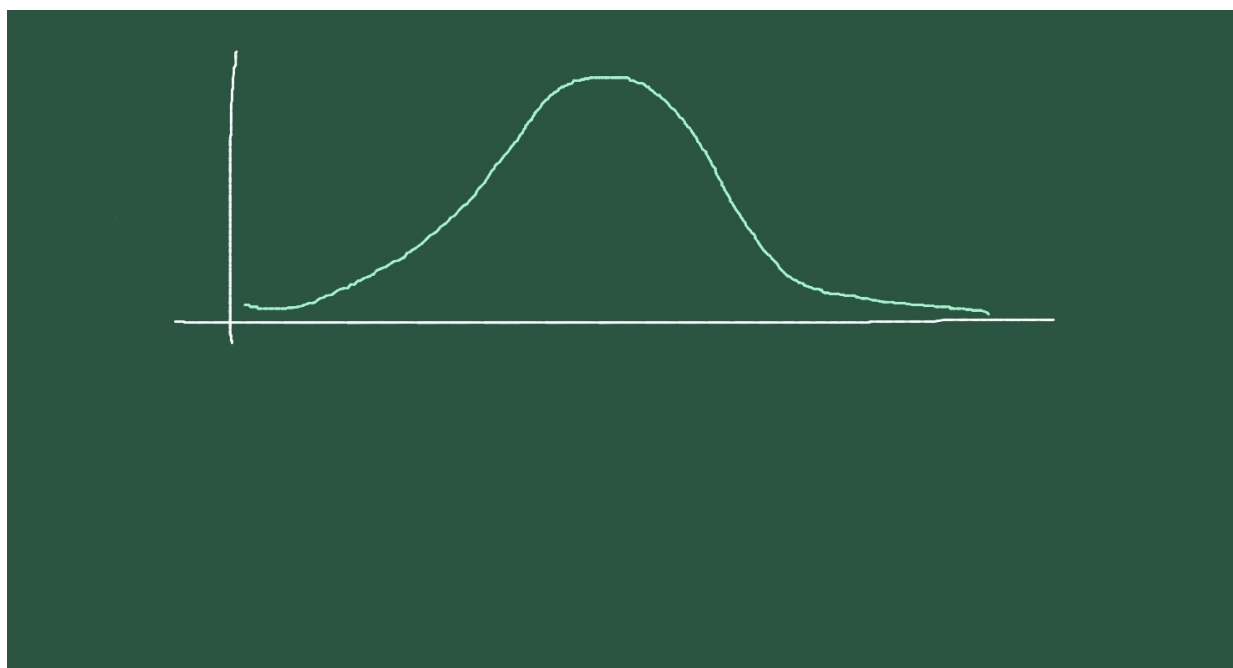
Probabilidade e Estatística formam um ramo essencial da Matemática escolar e científica. A primeira ajuda a descrever a chance de ocorrência de eventos em contextos de incerteza. A segunda organiza, resume e interpreta dados produzidos por observações, experimentos e pesquisas. Em conjunto, ambas permitem compreender fenômenos naturais e sociais, avaliar evidências, tomar decisões e reconhecer limites do acaso e dos padrões.

Este capítulo propõe explorar a ideia de que eventos aleatórios podem parecer imprevisíveis quando observados isoladamente mas, quando observados muitas vezes, revelam regularidades. É exatamente nesse ponto que a Estatística dialoga com a Probabilidade.

Depois de explorar, você consegue:

- compreender a diferença entre experimento aleatório, evento, frequência relativa e probabilidade;
- interpretar distribuições simples de dados obtidos em simulações;
- relacionar média e dispersão com variabilidade amostral;
- perceber como o aumento do número de observações pode estabilizar frequências;
- utilizar simulações interativas para investigar o comportamento de dados e eventos aleatórios.

Figura 4.1: O acaso depende do número de repetições.



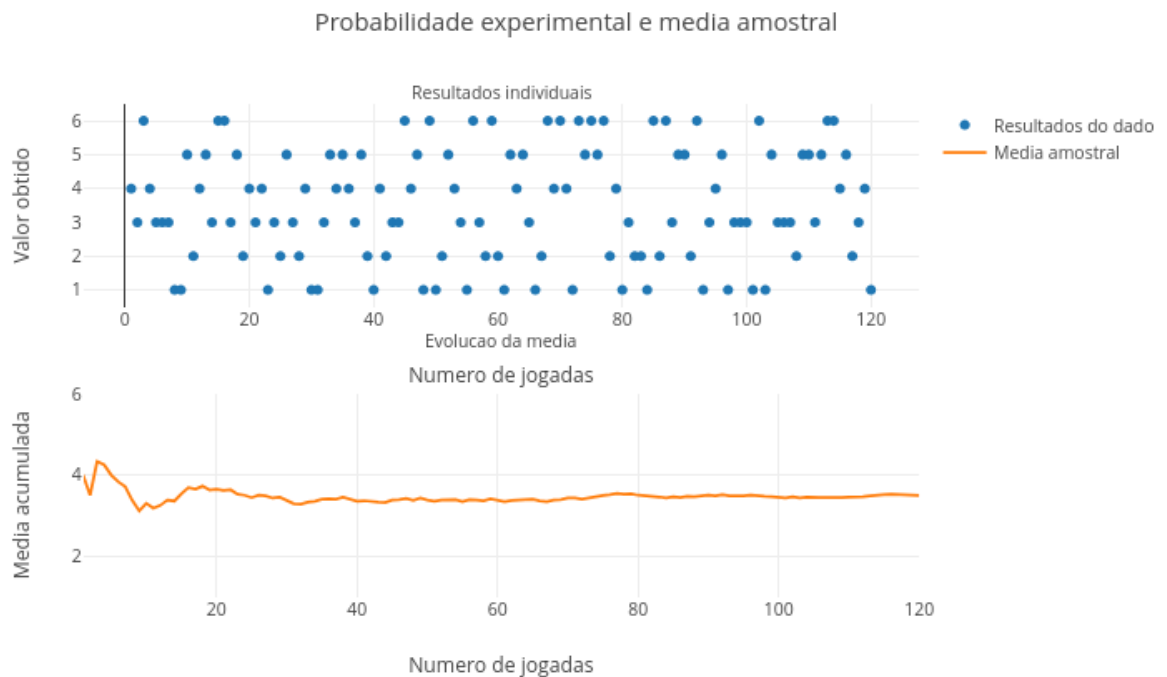
MEXA ANTES DE ENTENDER



A proposta inicial deste capítulo é observar o que acontece quando um experimento aleatório simples é repetido várias vezes. No primeiro objeto interativo, você poderá simular lançamentos de um dado e acompanhar tanto os resultados obtidos, como o comportamento da média amostral.

Antes de qualquer definição formal, experimente o aplicativo de JSPlotly a seguir.

Figura 4.2: Um resultado isolado pode ser imprevisível. Mas muitos resultados juntos revelam padrões. Clique com o botão direito neste [LINK](#) para o app.



Agite antes de usar

Esse objeto tenta aproximar a estrutura de um histograma, nome “*chique*” para um gráfico de barras que mostra frequências de um conjunto de dados contínuos agrupados por intervalos. Mas aqui não há barras, somente pontos. O *app* simula os resultados individuais de cada lance de um dado e a média acumulada ao longo das jogadas. Assim, o histograma muda quando a amostra é pequena ou grande. Além disso, o *app* também mostra como média e variabilidade se comportam em diferentes configurações, e como a frequência observada se aproxima, ou não, do comportamento esperado. Esse código integra alguns conceitos, como probabilidade experimental, amostra, média, oscilação inicial, e estabilização progressiva.

Cada jogada do dado é imprevisível. Pode sair 1, 2, 3, 4, 5 ou 6 (por óbvio, me perdoe !). Mas, quando repetimos muitas jogadas, a média começa a se aproximar de um valor esperado. Para um dado equilibrado, esse valor teórico é:

$$\mu = \frac{1+2+3+4+5+6}{6}$$

{eq-dado}

Com μ representando a média da população (total de jogadas). No exemplo acima, essa média prevista é igual a 3,5.

No primeiro gráfico do *app*, cada ponto representa uma jogada com valores variando de 1 a 6. O acaso aí aparece de forma irregular. Já no segundo gráfico, a linha mostra a média acumulada.

Inicie o aplicativo alterando o número de jogadas, quem sabe, entre 20 e 1000:

```
const n = 120;
```

Depois rode o *script* mais de uma vez. Observe que os resultados individuais mudam. No começo, a média oscila bastante mas, com mais jogadas, tende a ficar mais estável. E com muitas jogadas, a média tende a aproximar-se da média teórica prevista (3,5).

Além disso, se você rodar o código várias vezes, perceberá também que, embora cada execução seja diferente, o padrão geral tende a aparecer. A estatística começa exatamente aí, quando se observa muitos casos para entender o comportamento do conjunto.

No código, o dado é simulado aqui:

```
const valor = Math.floor(Math.random() * 6) + 1;
```

A soma acumulada é atualizada a cada jogada:

```
soma += valor;
```

E a média acumulada é calculada por:

```
medias.push(soma / i);
```

Matematicamente:

$$\hat{x}_n = \frac{x_1 + x_2 + \dots + x_n}{n} \quad (4.1)$$

Observe que o símbolo para média mudou ! Antes era μ , e agora é $\hat{x}$. Apesar de soar como uma pegadinha pra confundir você, há uma diferença estatística clara entre as duas representações: μ é para toda uma população, enquanto que $\hat{x}$ serve para representar a média de uma amostra dessa população. Num jogo de dados isso pode não ser tão expressivo, mas imagine situações em que seja necessário conhecer algo de toda uma população, como por exemplo, a idade de indivíduos numa metrópole com mais de 10 milhões de pessoas !

Se desejar sedimentar as ideias desta seção, tente responder as perguntas abaixo.

- A média varia muito nas primeiras jogadas ?
- Ela parece se estabilizar com o tempo ?
- Os resultados individuais seguem algum padrão visível ?

- O acaso impede totalmente a existência de regularidades ?
- Como descrever matematicamente eventos incertos ?
- O que muda quando repetimos muitas vezes um experimento aleatório ?
- Como os dados obtidos podem ser resumidos e interpretados ?
- Até que ponto uma amostra representa um comportamento esperado ?

A Probabilidade modela o que pode acontecer e com que chance. A Estatística examina o que de fato aconteceu, organizando e interpretando os resultados.

FAZENDO APARECER



4.0.1 Probabilidade: chance, modelo e frequência

A Probabilidade pode ser vista de duas formas complementares, o que esperamos que aconteça (o modelo teórico) e o que de fato acontece (a frequência observada). O ponto de encontro entre essas duas aparece quando repetimos muitas vezes um experimento.

Em um experimento aleatório, como lançar uma moeda ou um dado, não sabemos exatamente o resultado de cada tentativa individual. Ainda assim, podemos listar os resultados possíveis e, em casos simples, calcular a probabilidade teórica de cada um.

Por exemplo, para um dado equilibrado de seis faces, a probabilidade teórica P de sair qualquer valor específico é:

$$P(x) = \frac{1}{6}$$

{eq-probDado}

Entretanto, quando o experimento é realizado de verdade, os resultados observados nem sempre seguem imediatamente esse valor ideal. Essa frequência relativa observada f_r é definida por:

$$f_r = \frac{\text{número de ocorrências}}{\text{número total de tentativas}} \quad (4.2)$$

4.1 Estatística: resumir e interpretar dados

Os resultados de um experimento podem ser organizados em tabelas, distribuições e gráficos. A média aritmética, por exemplo, resume o valor central dos dados:

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_n}{n} \quad (4.3)$$

O desvio dessa média também nos ajuda a avaliar o grau de dispersão dos dados em torno do valor (ou ruído). Entre os desvios existentes, é bastante comum nas ciências o uso do desvio-padrão:

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}} \quad (4.4)$$

Com o símbolo Σ representando a somatória de elementos.

Uma maneira de resumir e interpretar dados de um experimento com amostra aleatória se dá pelo uso de um histograma. Para isso, experimente o JSPlotly que segue.

VOLTA PRO MUNDO



Agora sim, vamos experimentar um *histograma* de verdade !

Figura 4.3: O acaso de cada evento individual pode gerar regularidade quando observado em conjunto. Clique neste [LINK](#) para o app.



Nesse JSPlotly, o objeto interativo mostra como o histograma muda quando a amostra é pequena ou grande. Também apresenta como média e variabilidade se comportam em diferentes configurações. E ainda “mata” de vez como a frequência observada se aproxima, ou não, do comportamento esperado. Desse modo, o script possibilita simular a comparação entre diferentes tipos de experimentos, tamanhos amostrais e distribuições observadas.

Enquanto a média nos dá a noção de tendência para um conjunto de dados, a dispersão informa o quanto os dados variam em torno desse centro. Mesmo sem aprofundar ainda medidas como desvio padrão, você pode perceber visualmente que algumas amostras são mais concentradas e outras mais espalhadas.

Agite antes de usar

Antes de tentar entender qualquer fórmula...experimente o app.

Escolha um cenário no menu: dado justo, moeda justa, ou moeda viciada. Agora observe três coisas ao mesmo tempo: o histograma (o que saiu), a

média acumulada (o que está emergindo), e o painel numérico (o que está sendo medido). Vamos aos cenários:

1. *Comece pelo dado justo.* Rode várias vezes e observe se os valores parecem aleatórios, e se o gráfico é irregular. Agora olhe a média acumulada. Veja que ela começa instável, mas depois vai se organizando.
2. *Troque para moeda justa.* Observe que os valores agora são só 0 e 1. Então a média passa a representar uma proporção, e que tende a se aproximar de 0,5.
3. *Agora use a moeda viciada.* Sem mudar nada além do tipo, perceba que o comportamento muda, e a média converge para outro valor. Tente prever esse valor antes de olhar. Essa é a ideia de viés da amostragem.
4. *Mexa no tamanho da amostra.* Use o *slider* (barra deslizante), arrastando-o de 20 até 320, e verifique o que muda quando aumentamos o número de observações. O histograma fica mais “suave”? A média oscila menos?
5. *Rode várias vezes o mesmo cenário.* Mesmo tipo, mesmo valor de n . Observe que, embora cada execução seja diferente, o padrão geral se mantém o mesmo. Ou seja, o acaso muda, mas o comportamento global permanece.

4.2 Da chance ao padrão

Quando repetimos muitas vezes um experimento aleatório, as irregularidades das primeiras observações tendem a se suavizar. A frequência relativa passa a se aproximar da probabilidade teórica, e certos resumos estatísticos tornam-se mais estáveis. Isso equivale dizer que o acaso não desaparece, mas se organiza. Cada resultado individual continua imprevisível, mas o conjunto revela regularidades, permitindo que a repetição se transforme em informação.

Fica fácil então perceber que a leitura de dados em saúde, educação, economia, esportes e ciências depende do entendimento de probabilidade e estatística. Uma pesquisa com poucas pessoas pode não representar bem uma população. Isso porque um resultado extremo pode ocorrer por acaso e, dessa forma, produzir uma média isolada que pode esconder grande variabilidade. Apostas eleitorais que o digam !

E SE...



Essa seção é pra você sedimentar as ideias tratadas no capítulo, bem como para “viajar na maionese”, como diriam os representantes de gerações passadas.

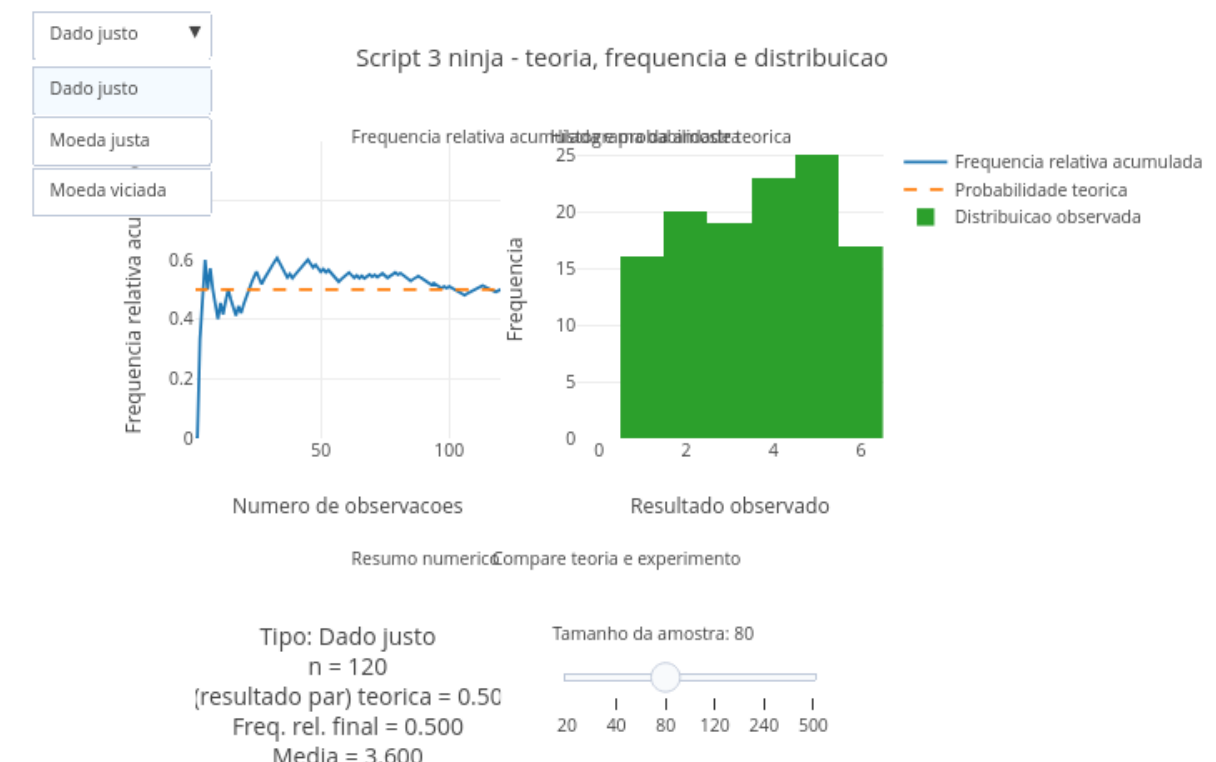
- E se aumentarmos muito o número de jogadas ?
- E se o dado fosse viciado ?

- E se comparássemos dois dados ?
- E se calculássemos também a frequência de cada face ?
- E se a média não se aproximasse de 3,5 ?
- E se a amostra for muito pequena ?
- E se repetirmos muitas vezes o experimento ?
- E se o experimento não for equilibrado ?
- E se duas amostras tiverem a mesma média, terão também a mesma dispersão ?

Parte dessas perguntas são facilmente respondidas com os aplicativos das [Figura 4.2](#) e [Figura 4.3](#).

Que tal um último JSPlotly no tema de Estatística e Probabilidade para compreender melhor o que se denomina Lei dos Grandes Números ?

Figura 4.4: Quando a frequência observada encontra a probabilidade teórica. Clique neste [LINK](#) para abrir o objeto interativo.



Esse terceiro script reúne o comportamento acumulado de um evento, a distribuição global dos resultados e o confronto entre dado observado e modelo teórico. Isso permite que você acompanhe, tentativa após tentativa, a *frequência relativa* acumulada de um evento, e não a média acumulada como no *app* anterior, e a compare com a probabilidade teórica correspondente. De fato, enquanto o script da [Figura 4.3](#) é um comparador estatístico de amostras, esse novo script da [Figura 4.4](#) se apresenta como um comparador entre experimento e teoria probabilística.

Agite antes de usar

O aplicativo é bem parecido com o da [Figura 4.3](#). Escolha um cenário

("dado justo", "moeda justa", "moeda viciada") e acompanhe o gráfico principal. Uma linha sobe e oscila, enquanto a outra fica fixa.

A linha que oscila representa a *frequência relativa acumulada*, instável no começo. Já a linha tracejada representa a *probabilidade teórica*. Rode o apicativo várias vezes, e observe se a linha experimental se aproxima da teórica.

Pode aumentar o tamanho da amostra também, de 20 (comportamento errático) a 500 (quase sobreposta), por exemplo. E observe o que o número de observações está fazendo com o acaso. Troque para a moeda viciada, e veja que a linha nunca vai para o valor de 0,5.

Verifique o que ocorreu olhando para o histograma. Novamente, o dado isolado fica bem diferente do padrão global. Nesse sentido, você acha que o acaso é realmente imprevisível, ou depende do número de observações ?

Essa é a *Lei dos Grandes Números* acontecendo na tela.

No caso do dado justo, por exemplo, o evento escolhido é "*obter resultado par*". A probabilidade teórica vale $3/6$, isto é, 0,5. Ao rodar a simulação, perceba que a frequência relativa acumulada não coincide imediatamente com esse valor, mas oscila ao seu redor e tende a se aproximar dele conforme o número de observações aumenta. Esse movimento é matematicamente simples mas significativo, porque mostra que teoria e experimento não precisam coincidir em cada instante para serem coerentes entre si.

Nas versões com moeda justa e moeda viciada, o script torna ainda mais evidente a diferença entre flutuação aleatória e tendência do sistema. Em uma moeda justa, a proporção de caras tende a se estabilizar em torno de 0,5; em uma moeda viciada, em torno de 0,75. A comparação visual entre essas situações permite perceber que certas diferenças observadas não são mero ruído, mas efeito do próprio modelo que gera os dados.

O histograma, por sua vez, funciona como uma leitura complementar. Enquanto a curva acumulada mostra aproximação progressiva entre frequência e teoria, o histograma revela como os resultados se distribuem na amostra. Já o painel numérico sintetiza informações importantes, como média, desvio e frequência final do evento observado.

Se quiser sedimentar o aprendizado visto até aqui, tente responder:

- A frequência relativa acumulada coincide com a probabilidade teórica desde o início ?
- O que acontece com essa aproximação quando o número de observações cresce ?
- Como diferenciar oscilações naturais do acaso de uma tendência sistemática ?
- O histograma confirma a mesma interpretação sugerida pela curva acumulada ?
- Em que sentido o experimento "aproxima" a teoria sem reproduzi-la exatamente em cada etapa ?

MESMO PADRÃO, OUTROS MUNDOS



Aqui há um mar...ou melhor, um oceano de situações parecidas em áreas bastante diversas, embora todas enraizadas pela Estatística e Probabilidade. Por exemplo, em pesquisas eleitorais, quando uma amostra pequena pode gerar conclusões instáveis e, por que não dizer, mesmo um tanto forçadas. Em testes clínicos os resultados precisam ser analisados com base em distribuições estatísticas (*t-Student*, *normal*), e não apenas em médias aritméticas. Sequências de vitórias ou derrotas nos esportes podem ocorrer por acaso, sem indicar necessariamente mudança de desempenho. Em redes sociais, números grandes de visualizações não significam, por si só, relevância ou qualidade.

Na genética mendeliana a proporção de fenótipos é ditada pela probabilidade envolvida nos cruzamentos. Quando um patógeno se alastra pela população, a distribuição de casos em estudos epidemiológicos é de vital importância para prever a disseminação e conter a doença. Variações em processos industriais são comumente observados pelo setor de controle de qualidade. E nas ciências, não importando qual, dificilmente se conclui qualquer coisa que não esteja repousada sobre distribuições e testes comparativos. Em todos esses contextos, a combinação entre probabilidade e estatística permite transformar dados em interpretações significativas.

Assim, probabilidade e estatística ajudam a pensar diante da incerteza, observar regularidades em meio ao acaso, e interpretar dados com mais cuidado, criticidade e responsabilidade.

POR DENTRO DA FERRAMENTA



Os scripts desse capítulo usam simulação computacional simples para gerar resultados aleatórios e transformá-los em tabelas e gráficos. Neles estão presentes idéias diversas consoantes às questões levantadas no capítulo, como a geração de números aleatórios, a repetição por laços, a contagem de frequências, e o cálculo de média e dispersão de dados.

Para ilustrar essas ações, sugere-se a seguir um *pseudocódigo*, uma mistura de programação com linguagem natural, e centrado na *lógica de programação* para o script da [Figura 4.2](#).

INÍCIO

definir número de jogadas n

criar lista de resultados
criar lista de médias acumuladas
criar lista para o eixo x

soma $\leftarrow 0$

PARA i de 1 até n :

 sortear um número de 1 a 6

 guardar o resultado sorteado

 adicionar o resultado à soma

 calcular média acumulada:
 média $\leftarrow$ soma / i

 guardar essa média

 guardar o número da jogada

FIM PARA

criar gráfico 1:

 eixo $x \rightarrow$ número da jogada
 eixo $y \rightarrow$ resultado obtido no dado

criar gráfico 2:

 eixo $x \rightarrow$ número da jogada
 eixo $y \rightarrow$ média acumulada

mostrar os dois gráficos

FIM

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos



O MUNDO TE CHAMA

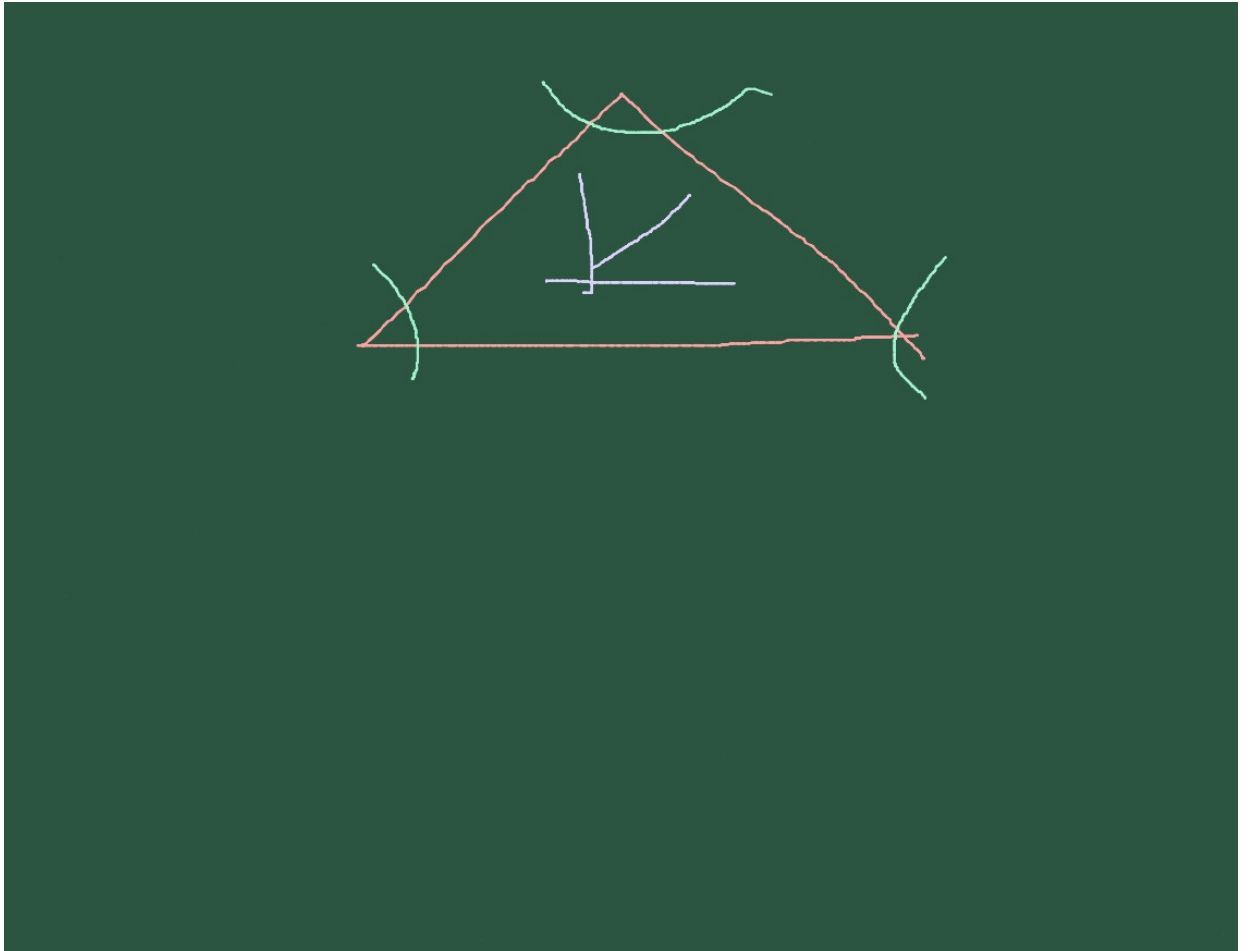
As ondas sonoras, o movimento das rodas, a corrente alternada, as vibrações mecânicas, o movimento das marés, as estações do ano, o batimento cardíaco, além de inúmeros outros sinais periódicos, aparecem em diferentes contextos da ciência e da tecnologia. A Trigonometria permite descrever esses fenômenos por meio de razões geométricas, funções periódicas e representações no plano, temas deste capítulo.

Depois de explorar, você consegue:

- reconhecer seno, cosseno e tangente como relações geométricas em triângulos retângulos;
- interpretar seno e cosseno como coordenadas no círculo trigonométrico;
- relacionar ângulo, rotação e repetição em representações gráficas;
- compreender a periodicidade em fenômenos matemáticos e físicos;
- estabelecer a conexão entre trigonometria e números complexos no plano;
- integrar diferentes representações de uma mesma ideia matemática por meio de modelos interativos.

Figura 5.1: A trigonometria como um caminho para interpretar formas, ângulos e relações.

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos



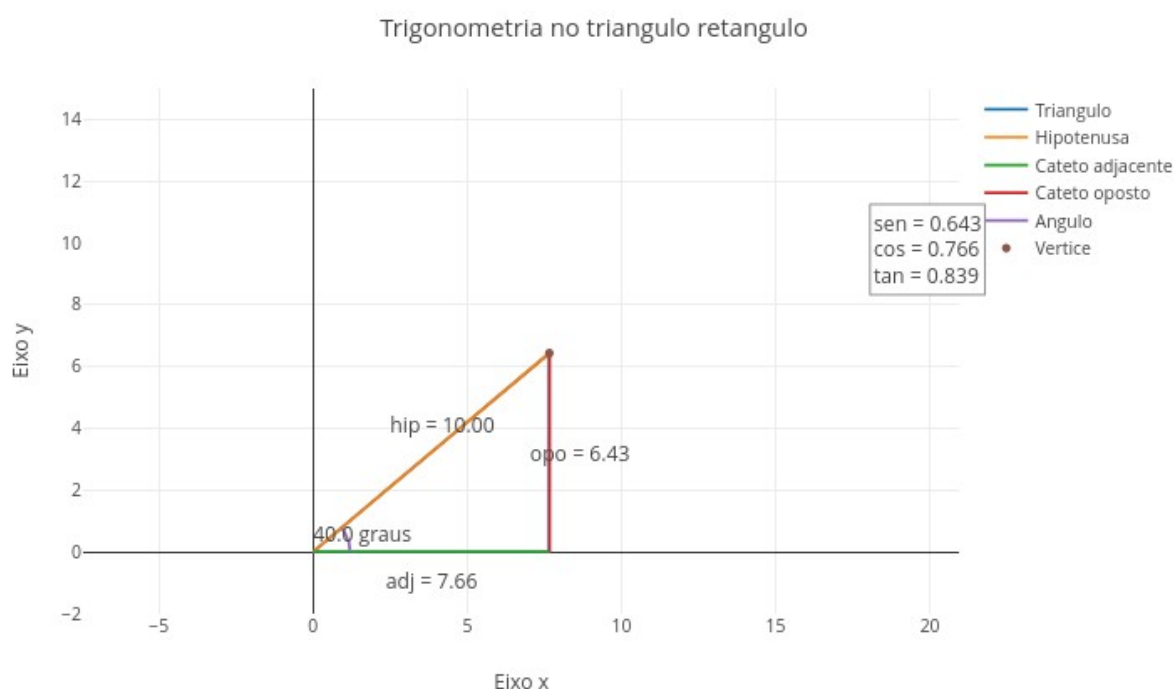
MEXA ANTES DE ENTENDER



5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

Você pode iniciar os temas desse capítulo com calma, apenas observando e editando o código que aparece no JSPlotly da [Figura 5.2](#), e que trata das relações em triângulos.

Figura 5.2: Seno, cosseno e tangente como relações geométricas entre lados de um triângulo retângulo. Clique neste [LINK](#) para abrir o aplicativo em nova aba.



O script mostra como 3 funções básicas, seno, cosseno e tangente, aparecem diretamente em um triângulo retângulo. A ideia é simples: ao escolher um ângulo e uma hipotenusa, essa o maior lado de um triângulo retângulo, o modelo calcula automaticamente os outros dois lados, denominados por cateto adjacente (próximo ao ângulo), e cateto oposto, bem como as razões trigonométricas correspondentes.

Agite antes de usar

Comece alterando apenas dois valores no início do código, e apenas observe o que acontece no *app*:

```
const ang_deg = 40;  
const hip = 10;
```

O valor `ang_deg` representa o ângulo, em graus. Para evitar triângulos muito achatados ou quase verticais, experimente valores entre 5 e 85. O valor `hip` define o tamanho da *hipotenusa*. Depois de modificar os valores, clique em `add` e observe como o triângulo muda.

Quando o ângulo aumenta, o cateto oposto cresce e o cateto adjacente diminui. Assim fica “fácil” perceber que o seno está associado ao cateto oposto, enquanto o cosseno ao cateto adjacente.

No código, isso aparece aqui:

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

```
const adj = hip * Math.cos(ang);  
const opo = hip * Math.sin(ang);  
const tanv = Math.tan(ang);
```

Ou seja, o triângulo é construído a partir das próprias relações trigonométricas. Você pode explorar alguns cenários para sedimentar esse aprendizado. E também pode optar por limpar a tela (clean) antes de cada experiência. Ou não limpá-la, e nesse caso permitindo a comparação entre valores no aplicativo.

1. *Ângulo de 30 graus.* Experimente editar o código como segue, e observe que o cateto oposto fica menor que o adjacente. E discuta por que o seno de 30° é menor que o cosseno de 30° .

```
const ang_deg = 30;  
const hip = 10;
```

2. *Ângulo de 45 graus.* Aqui os dois catetos ficam iguais, resultando no chamado triângulo isósceles, e com propriedades especiais entre seno e cosseno.

```
const ang_deg = 45;  
const hip = 10;
```

3. *Ângulo de 60 graus.* Dessa vez, o cateto oposto fica maior que o adjacente. Compare com o caso de 30° e observe a inversão entre seno e cosseno.

```
const ang_deg = 60;  
const hip = 10;
```

4. *Mesma forma, tamanho diferente.* Teste os valores abaixo.

```
const ang_deg = 40;  
const hip = 5;
```

Depois teste:

```
const ang_deg = 40;  
const hip = 15;
```

Perceba então que o triângulo aumenta ou diminui, mas os valores de seno, cosseno e tangente permanecem os mesmos. Isso ajuda a mostrar que essas razões dependem do ângulo, e não do tamanho absoluto do triângulo.

5. *Previendo valores.* Agora que você já percebeu que os valores para seno, cosseno e tangente são obtidos em função dos tamanhos dos lados de um triângulo retângulo e de suas razões, experimente prever alguns resultados para o script da [Figura 5.2](#). Para isso, basta esconder os textos que o gráfico apresenta, veja:

```
const mostrar_textos = false;
```

ou:

```
const mostrar_arco = false;
```

“Truquento”, não ?!

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

6. *Ângulos pequenos*. Compare 10, 20 e 30 graus. O que cresce mais rápido: o seno, o cosseno ou a tangente ?
7. *Aproximação de 90 graus*. O que acontece com a tangente quando o ângulo se aproxima de 90 graus ? Como isso aparece geometricamente ?
8. *Triângulos semelhantes*. Mude a hipotenusa mantendo o ângulo fixo. O que muda e o que permanece ?
9. *Relação entre forma e razão*. Compare triângulos diferentes com o mesmo ângulo. Por que seno, cosseno e tangente continuam os mesmos ?

Após testar esses cenários no JSPlotly, você deve perceber algumas relações de triângulos retângulos. Por exemplo, que o cateto adjacente corresponde a projeção horizontal, enquanto que o cateto oposto corresponde a projeção vertical. Que o seno cresce conforme a componente vertical aumenta, enquanto o cosseno diminui conforme a componente horizontal diminui. E que a tangente expressa a razão entre a altura e a base do triângulo.

Ou seja, quando o ângulo muda, as proporções mudam. Quando apenas o tamanho do triângulo muda, as proporções permanecem. Esses mesmos conceitos também aparecem no gráfico de uma função periódica, no ponto que gira sobre uma circunferência, ou na representação de um número complexo.

FAZENDO APARECER



A trigonometria pode ser compreendida como uma relação entre forma, movimento e representação. Num *triângulo*, as razões entre os lados descrevem proporções. Num *círculo*, elas descrevem coordenadas, enquanto que nos *gráficos*, tornam-se funções periódicas. E num *plano complexo*, convertem-se em rotação e orientação.

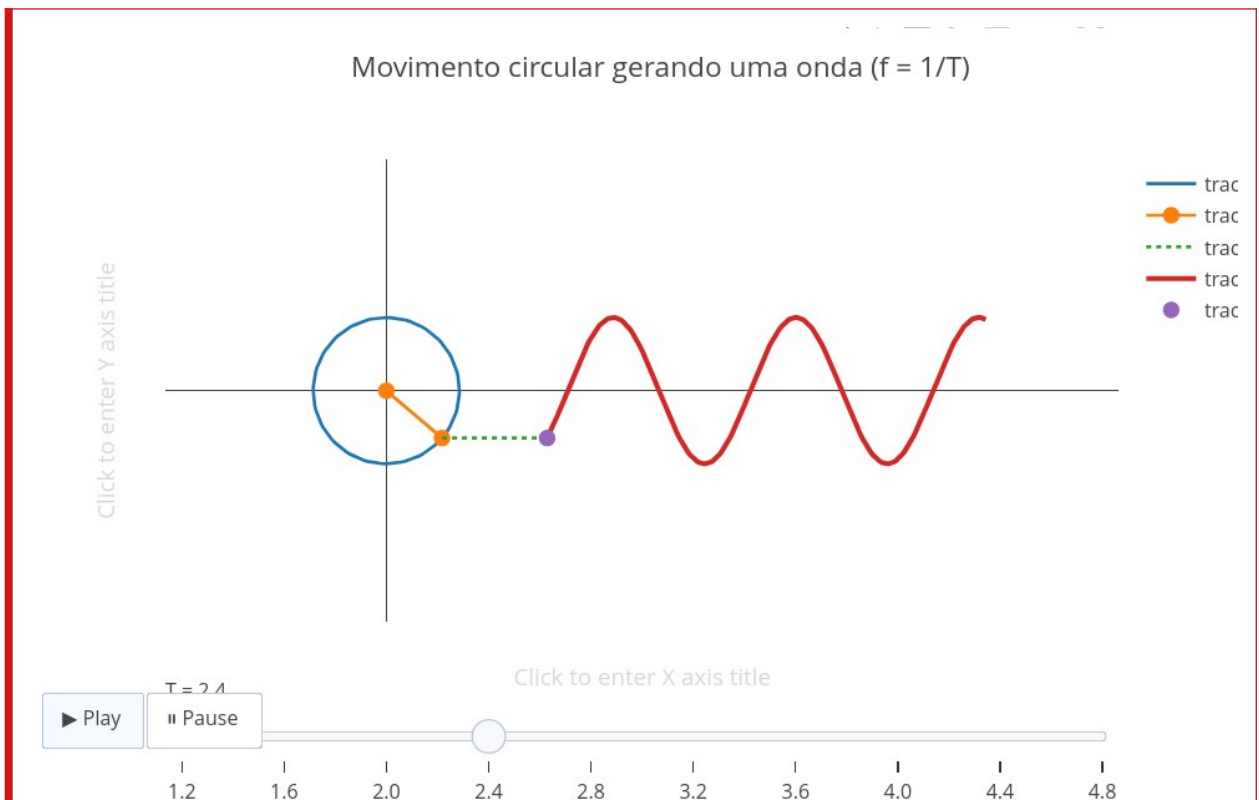
Para tornar essas ideias mais claras, comece pelo *app* de JSPlotly que segue, e que aborda, de forma realmente *animada*, o conceito trigonométrico de onda como uma projeção de movimento circular.

Agite antes de usar

Clique em add para executar o script, e em Play para iniciar a animação. Veja que o ponto gira em torno de um círculo no painel. À medida que ele se move, sua altura é projetada para a direita, formando uma curva senoidal, representando a altura de um ponto que gira.

Figura 5.3: Uma onda senoidal pode ser entendida como o rastro de um movimento circular. Clique neste [LINK](#) para abrir o app.

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos



O *slider* altera o valor de T , que representa o período do movimento. Valores menores de T geram ciclos mais rápidos (frequência maior), enquanto que valores maiores deixam o movimento mais lento e a onda mais alongada (comprimento de onda). No código, o período inicial está definido aqui:

```
let T0 = 2.5;
```

E os valores disponíveis no slider são criados neste trecho:

```
for (let T = 1.2; T <= 5.0; T += 0.4) {  
  steps.push({  
    label: T.toFixed(1),  
    method: "animate",  
    args: [gerarAnimacao(T)]  
  });  
}
```

Observe primeiro o ponto girando no círculo, e depois acompanhe a linha pontilhada horizontal que leva a altura desse ponto até o início da onda. A altura do ponto no círculo é a mesma altura do ponto marcado na onda. Isso mostra que a onda é construída a partir da projeção vertical do movimento circular. A relação trigonométrica aparece nesse trecho do código:

```
const px = Math.cos(ang);  
const py = Math.sin(ang);
```

A coordenada px indica a posição horizontal do ponto no círculo, e a coordenada py indica sua altura. É essa altura, dada pelo seno do ângulo,

que aparece na onda. Para “brincar” um pouco com o *app*, seguem alguns cenários.

1. *Período curto*. Use o slider em um valor baixo, como:

$T = 1.2;$

E veja que a onda fica mais comprimida, e que o movimento circular completa ciclos mais rapidamente. Isso aponta para uma maior frequência associada a um menor período.

1. *Período intermediário*. Mude T para 2.5. Observe que a animação fica mais equilibrada, permitindo que se observe com clareza a ligação entre o círculo e a onda.
2. *Período longo*. Agora use $T = 5.0$, e perceba que a onda fica mais alongada e o movimento parece mais lento. Ou seja, maior período significa menor frequência e maior comprimento de onda.
3. *Comparando forma e tempo*. Mantenha a amplitude fixa e altere apenas T . Observe que a altura máxima da onda não muda, e sim a distância entre repetições. Isso acontece porque o período e a amplitude são propriedades diferentes. O período controla a repetição, enquanto que a amplitude controla a altura.

Esse script mostra que, quando um ponto gira, sua altura sobe e desce de modo periódico. Essa oscilação vertical acompanhada no espaço forma uma onda. Isso é bem legal, porque agora você pode enxergar o círculo trigonométrico como uma máquina de gerar ciclos.

Em outras palavras, o círculo gera uma rotação que produz um seno, o qual gera uma onda que revela periodicidade.



Essas relações da trigonometria aparecem em inúmeros contextos da vida real. Só para citar alguns: ondas e vibrações, corrente alternada, processamento de sinais, navegação e localização, sonar, animações, computação gráfica, telecomunicações, eletrotécnica, e em quase todo fenômeno periódico, como o batimento cardíaco, ou as marolas que se sucedem à passagem de uma embarcação.

5.1 Números complexos

Uma extensão do conceito acima também ocorre em ramos da Física e Química, como a Eletrônica e estudos de interfaces, e que envolve os assim chamados números complexos.

O valor de um número num *eixo cartesiano*, aquele que tem um x e um y , pode ser dado de dois jeitos: ou pelas coordenadas x e y , denominadas coordenadas retangulares, ou pela trigonometria. Nesse caso, chamado por coordenadas polares, utiliza-se o ângulo que define o ponto junto a origem do gráfico. Essa forma trigonométrica (ou polar) para representar um ponto no espaço plano faz uso dos chamados números complexos.

Números complexos são expressões do tipo $z=a+bi$, onde a e b são números reais e i é a um termo imaginário, esse definido por $i^2=-1$. Eles estendem os números reais para permitir raízes quadradas de números negativos. O termo a é chamado de parte real, e b de parte imaginária.

A forma trigonométrica (ou polar) de um número complexo $z=a+bi$ é dada abaixo.

$$z=\rho(\cos\theta+i\sin\theta)\quad(5.1)$$

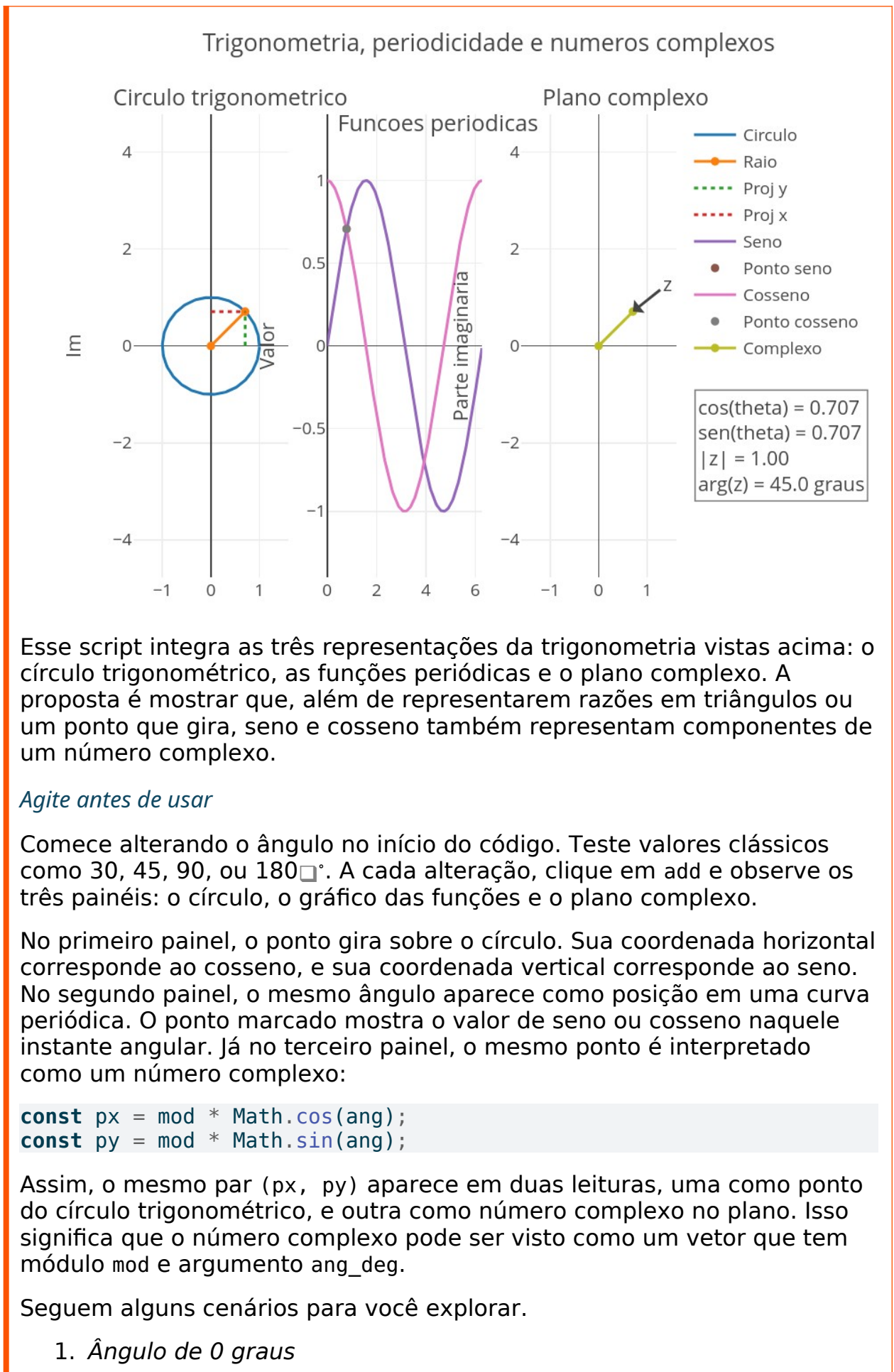
Onde ρ é o módulo $\rho=\sqrt{a^2+b^2}$ e θ é o argumento (ângulo).

Pode parecer estranho à primeira vista, mas essa representação facilita operações como multiplicação, divisão e potenciação. O eixo cartesiano em que essa representação geométrica tem seu lugar é também chamado de plano complexo ou plano de Argand. Por vezes, esse plano possui seus eixos denominados por real(x) e imaginário(y) !

Caramba !!! Quanta coisa esquisita. Que tal então um script de JSPlotly para se compreender um pouco melhor o *plano de Argand* e tornar a vida menos chata ?

Figura 5.4: Trigonometria, periodicidade e números complexos são temas que envolvem rotação. Clique neste [LINK](#) e abra o app.

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos



5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

```
const ang_deg = 0;  
const mod = 1;
```

O ponto fica sobre o eixo real positivo. O cosseno vale 1 e o seno vale 0. Esse é um bom ponto de partida para discutir por que o círculo trigonométrico começa no eixo horizontal.

2. *Ângulo de 90 graus.* O ponto vai para o eixo imaginário positivo. O cosseno se aproxima de 0 e o seno vale 1. Nesse cenário aparece a ligação entre rotação e coordenadas.
3. *Ângulo de 180 graus.* O ponto fica sobre o eixo real negativo. O cosseno vale -1 e o seno volta a 0. Esse cenário é típico do apresentado por alguns sinais e quadrantes.
4. *Aumento do módulo.*

```
const ang_deg = 45;  
const mod = 2;
```

O ponto mantém a mesma direção, mas se afasta da origem. Isso mostra que o módulo altera o tamanho do vetor, mas não o ângulo.

5. *Aumento da frequência*

```
const freq = 2;
```

A curva completa mais oscilações no mesmo intervalo. Esse cenário também auxilia na interpretação já vista para periodicidade, ondas, vibração e fenômenos repetitivos.

6. *Compare seno e cosseno.* Você pode comparar “seno”, “cosseno”, ou “ambos”, bastando alterar esse trecho do código:

```
const mostrar = "seno";
```

Observe o deslocamento entre as duas curvas para a opção “ambos”. O cosseno começa no valor máximo, enquanto o seno começa em zero. Essa diferença aparece porque são projeções diferentes do mesmo movimento circular.

Se quiser aprender um pouco mais do script, a *conversão de graus para radianos* acontece aqui:

```
const ang = ang_deg * Math.PI / 180;
```

Isso é necessário porque as funções trigonométricas em JavaScript usam radianos. Já as funções periódicas são construídas repetindo seno e cosseno ao longo de um intervalo angular:

```
y_sen.push(amp * Math.sin(freq * x));  
y_cos.push(amp * Math.cos(freq * x));
```

A variável `amp` controla a altura da onda, enquanto `freq` controla o número de repetições.

Dessa forma, pode-se dizer que seno e o cosseno podem ser entendidos ao mesmo tempo como razões de um triângulo, coordenadas de um ponto, e números complexos. Suas características permitem que sejam

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

adequadas para descrever fenômenos periódicos. E isso é pra lá de legal, pois você percebe agora que uma mesma grandeza angular pode organizar representações geométricas, gráficas e algébricas !

Em suma, percebe que um ponto que gira no círculo gera coordenadas, e essas coordenadas formam seno e cosseno. Quando acompanhadas ao longo do tempo, viram ondas (ciclos), e quando interpretadas como par real-imaginário, viram números complexos (rotação).



E SE...

Aqui a gente “deita e rola” na abstração verificável (ou não) com os scripts do capítulo. Em relação ao script da [Figura 5.2](#):

- E se o ângulo ficar muito pequeno ?
- E se o ângulo se aproximar de 90° ?
- E se a hipotenusa dobrar, mas o ângulo continuar igual ?
- E se dobrarmos a frequência ?
- E se a amplitude for reduzida pela metade ?
- E se o ponto girar no sentido horário em vez de anti-horário ?
- E se compararmos ângulos equivalentes, como 30 graus e 390 graus ?

Já para o script da [Figura 5.3](#):

- E se o período for muito pequeno ?
- E se o período for muito grande ?
- E se a velocidade v aumentar ?
- E se a amplitude A também fosse usada para aumentar o raio do círculo ?
- E se, em vez da projeção vertical, usássemos a projeção horizontal ?

E para o script da [Figura 5.4](#):

- E se o ângulo passar de 90° para 180° ?
- E se o módulo aumentar, mas o ângulo continuar igual ?
- E se a frequência dobrar ?
- E se mostrarmos apenas o seno ?
- E se mostrarmos apenas o cosseno ?
- E se compararmos o mesmo ângulo no círculo e no plano complexo ?
- E se o módulo do número complexo deixar de ser 1 ?



5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

Vimos no capítulo algumas situações em que o emprego da trigonometria é refletido no estudo de relações em triângulos, ondas e plano complexo, e tudo isso com “cadeira vitalícia” nas ciências e engenharia. Mas sempre tem lugar para mais uma cadeirinha.

Todos esses conceitos também estão espelhados no funcionamento do relógio analógico e seu movimento angular, nos ciclos naturais das marés, nas ondas sonoras, em rotação de vetores, em fasores da física, em animações, computação gráfica, modelagem de ciclos naturais, localização espacial, corrente alternada, e em todos os sinais elétricos senoidais.

Em relação a esses últimos, alguns sinais são transformados em ondas quadradas repetitivas, em função de certas restrições instrumentais. Isso é visto aos montes na chamada *PWM (Pulse Width Modulation)*, sinal de saída de microcontroladores, como placas Arduino, ESP32, e Raspberry pi.

POR DENTRO DA FERRAMENTA



Nos scripts desse capítulo, a ideia principal foi converter um ângulo em diferentes representações numéricas e geométricas. Aqui vamos destrinchar um pouquinho mais do script da [Figura 5.3](#), aquele de movimento circular gerando uma onda. O código calcula a frequência a partir do período:

```
const f = 1 / T;  
const omega = 2 * Math.PI * f;  
const lambda = v * T;
```

Isso significa que:

- T é o tempo de um ciclo completo;
- f é quantos ciclos ocorrem por unidade de tempo;
- ω é a velocidade angular;
- λ é o comprimento de onda.

Quando T diminui, a frequência aumenta. Quando T aumenta, a frequência diminui.

A animação é construída por uma sequência de *frames*. Em cada frame, o ponto gira um pouco mais e a onda é redesenhada:

```
const ang = omega * tempo;
```

Esse ângulo muda com o tempo e controla tanto o ponto no círculo quanto a fase da onda. Uma sugestão para pseudocódigo do script é dada abaixo.

```
INICIAR parâmetros principais  
  definir amplitude  $A$   
  definir velocidade  $v$   
  definir número de pontos da onda
```

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

```
definir quantidade de frames da animação

FUNÇÃO gerarFrame(T, tempo)

    calcular frequência:
         $f = 1 / T$ 

    calcular velocidade angular:
         $\omega = 2\pi f$ 

    calcular comprimento de onda:
         $\lambda = vT$ 

    calcular ângulo atual:
         $\text{ang} = \omega \times \text{tempo}$ 

    calcular posição do ponto no círculo:
         $px = \cos(\text{ang})$ 
         $py = \sin(\text{ang})$ 

    gerar pontos do círculo:
        PARA vários ângulos th
            calcular  $x = \cos(th)$ 
            calcular  $y = \sin(th)$ 
        FIM

    gerar pontos da onda:
        PARA vários pontos x
            calcular fase:
                 $\text{fase} = 2\pi(x/\lambda) + \text{ang}$ 

            calcular altura da onda:
                 $y = \sin(\text{fase})$ 
        FIM

    retornar:
        círculo
        ponto girando
        onda
        parâmetros associados

FIM FUNÇÃO

FUNÇÃO gerarAnimação(T)

    criar lista de frames

    PARA cada instante de tempo
        gerar frame usando gerarFrame(T, tempo)
        armazenar frame
    FIM
```

5. Quando o movimento se repete: trigonometria, periodicidade e números complexos

retornar animação completa

FIM FUNÇÃO

gerar animação inicial

criar gráficos:

círculo

raio girando

linha de projeção

onda

ponto da onda

criar slider:

alterar T

recalcular animação

criar botões:

Play

Pause

mostrar resultado final

6. Crescimento, Tempo e Decisão: Matemática Financeira

O MUNDO TE CHAMA



Você já percebeu que o dinheiro parece ter vida própria ?

Ele cresce no banco (e quase sempre cresce ainda mais para o banco), aumenta em financiamentos, muda de valor com o tempo e, às vezes, parece desaparecer silenciosamente do seu bolso. Isso acontece porque o tempo transforma valores. Mas o tempo, sozinho, não é o verdadeiro responsável. O que realmente altera o dinheiro são processos variados, como juros, inflação, investimentos, dívidas, consumo, aportes sucessivos, e crescimento acumulado.

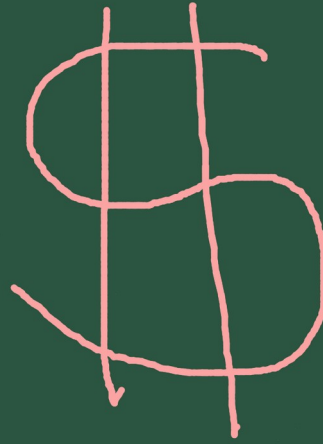
Todos eles dependem da forma como pequenas variações se acumulam ao longo do tempo. Em alguns casos, o crescimento acontece de forma previsível e linear, e em outros, acontece de forma acelerada, e quase explosiva. É justamente por isso que duas pessoas podem começar com o mesmo dinheiro e terminar em situações completamente diferentes alguns anos depois.

Neste capítulo, você tem a chance de explorar alguns mundos diferentes da Matemática Financeira, toda ela baseada em juros simples e juros compostos. Nos juros simples, o crescimento ocorre em linha reta e os juros são sempre calculados sobre o valor inicial. Já nos juros compostos, o valor cresce de modo acumulativo, com crescimento acelerado e exponencial, consistindo num dos fenômenos matemáticos mais importantes do mundo moderno.

Depois de explorar, você consegue:

- diferenciar crescimento linear e exponencial;
- interpretar juros em situações concretas;
- compreender o papel do tempo, da taxa e dos aportes;
- perceber a diferença entre valor nominal e valor real;
- modelar situações financeiras simples com código;
- tomar decisões mais conscientes em contextos cotidianos;
- explorar a matemática financeira com auxílio de aplicações computacionais.

Figura 6.1: Matemática financeira é uma forma de interpretar decisões humanas sobre dinheiro ao longo do tempo.

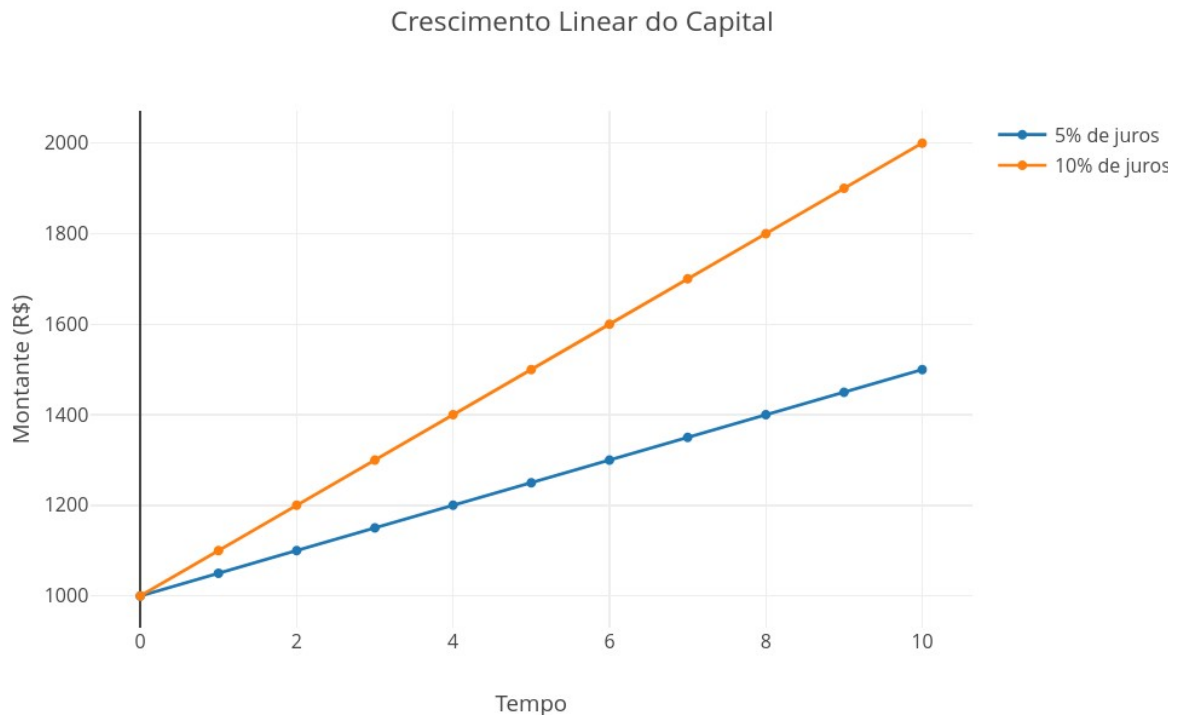


MEXA ANTES DE ENTENDER



A proposta inicial neste capítulo é você observar como um capital evolui quando submetido a uma taxa fixa ao longo do tempo, bem como perceber os padrões associados a dívidas, financiamentos ou rendimentos sob essa regra. Antes de qualquer definição formal, experimente o JSPlotly que segue, para juros simples.

Figura 6.2: Juros simples: quando o dinheiro cresce ou desaparece em linha reta. Clique com o botão direito neste [LINK](#) para abrir o app em nova aba.



Agite antes de usar

Você pode alterar os valores iniciais do código aqui, executando em seguida o script (add):

```
const capitalInicial = 1000;
const taxa = 0.05;
const tempoMax = 20;
```

Teste primeiro pequenas mudanças. Aumente o capital inicial, diminua a taxa, aumente o tempo máximo, e compare gráficos com taxas diferentes. Observe que, em juros simples, o crescimento do dinheiro não acelera. Isso acontece porque os juros são sempre calculados sobre o mesmo valor inicial.

Se o capital inicial é C , a taxa é i e o tempo é t , então o juro acumulado é:

$$J = C \cdot i \cdot t \quad (6.1)$$

E o montante final, o dinheiro acumulado (investimento ou dívida) é:

$$M = C + J \quad (6.2)$$

Substituindo J , temos:

$$M = C \cdot (1 + i \cdot t) \quad (6.3)$$

Perceba que, não importando as equações acima, há sempre uma relação linear entre montante e tempo.

Dessa forma, o script constrói uma lista de tempos e calcula, para cada período, o montante correspondente nesse trecho:

```
for (let t = 0; t <= tempoMax; t++) {
  let J = capitalInicial * taxa * t;
  let M = capitalInicial + J;

  tempo.push(t);
  juros.push(J);
  montante.push(M);
}
```

Nesse modelo, t representa o tempo, J os juros acumulados, e M o montante. O script então guarda os valores em listas para formar o gráfico. O resultado é uma reta crescente, pois o acréscimo a cada período é sempre o mesmo.

Uma sugestão para mexer no *app* é dada abaixo.

1. Execute o script com os valores originais;
2. Observe o formato da curva;
3. Altere taxa para 0.02, 0.05, 0.10 e 0.20;
4. Compare a inclinação das retas;
5. Altere `capitalInicial` para valores maiores ou menores;
6. Observe que o gráfico muda de escala, mas continua linear;
7. Aumente `tempoMax` e perceba que a reta se prolonga sem mudar de forma.

Traduzindo...nos juros simples, o tempo aumenta o montante de forma proporcional e linear, quer seja um investimento de capital, quer seja uma dívida.

Agora sim, alguns cenários para você explorar.

1. *Pequena poupança.* Edite o código e responda quanto o dinheiro cresce em um ano com uma taxa pequena.

```
const capitalInicial = 500;
const taxa = 0.02;
const tempoMax = 12;
```

2. *Empréstimo com taxa alta.* Quanto se paga ao final de 12 períodos ?

```
const capitalInicial = 1000;
const taxa = 0.10;
const tempoMax = 12;
```

3. *Tempo longo.* Mesmo com juros simples, o tempo pode tornar uma dívida pesada ?

```
const capitalInicial = 1000;
```

```
const taxa = 0.05;  
const tempoMax = 60;
```

4. *Comparação de capitais.* Compare o `capitalInicial` acima com um de 5000. O capital inicial muda a forma do gráfico ou apenas sua escala ?

O código usa uma estrutura muito comum em simulações matemáticas. Ele define valores iniciais, cria listas vazias, calcula resultados em um laço, e envia os dados para o gráfico. Essa mesma estrutura pode ser reutilizada em muitos outros modelos, como crescimento populacional, movimento uniforme, depreciação, consumo de energia e comparação de investimentos, por exemplo.

FAZENDO APARECER

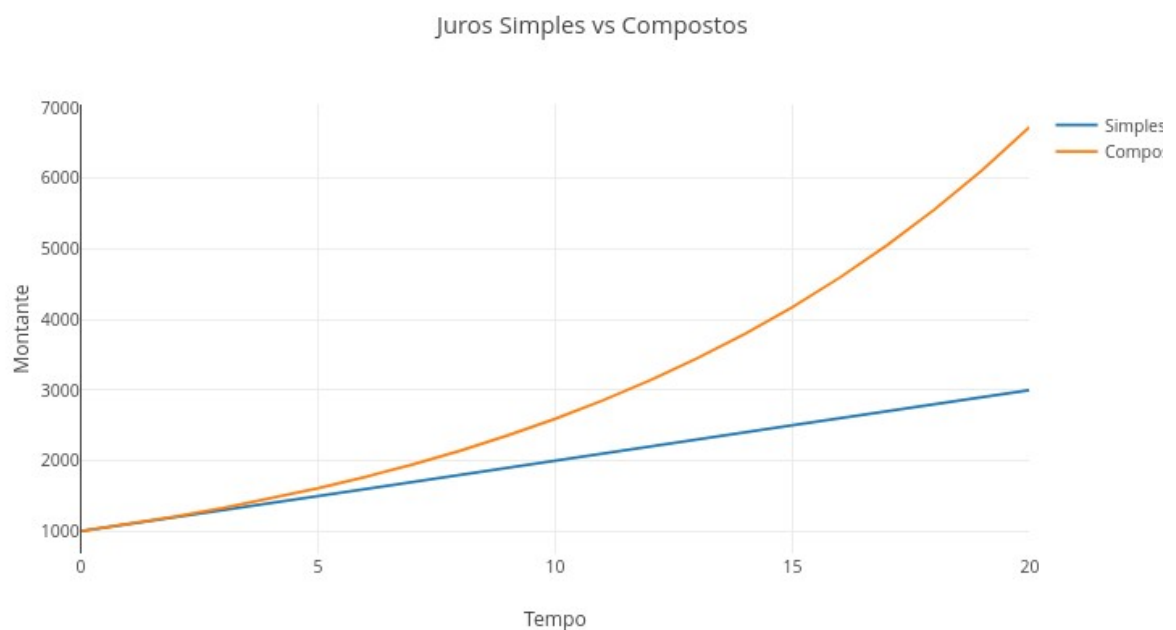


Na vida real, muitos contratos financeiros não usam juros simples por longos períodos. Isso acontece porque bancos, financiamentos, cartões e aplicações geralmente usam juros compostos, nos quais os *juros passam a gerar novos juros*. Por isso, o primeiro script funciona para apresentar o caso mais simples, linear e previsível. Depois dele, fica mais fácil perceber por que os juros compostos podem crescer muito mais rapidamente.

E também porque pequenas quantidades em dívidas ou investimentos crescem enormemente ao longo do tempo. Enquanto o *juro simples se aproxima de uma ideia linear*, o *juro composto se aproxima de uma ideia exponencial*. Essa é uma forma de ver a matemática financeira como uma aplicação direta de padrões de crescimento estudados em álgebra e funções.

O aplicativo de JSPlotly a seguir trata exatamente disso, os juros compostos. No juro composto, cada novo período incorpora os aumentos anteriores, produzindo o conhecido “*juros sobre juros*”.

Figura 6.3: Enquanto os juros simples caminham sobre o dinheiro, os juros compostos aprendem a voar. Clique neste [LINK](#) para abrir em nova aba.



Agite antes de usar

Execute o script original e observe as duas curvas. Depois altere os valores:

```
const C = 1000;
const i = 0.10;
const nMax = 20;
```

Teste diferentes taxas e tempos. Experimente taxas pequenas e altas,

bem como tempos curtos e longos. E observe em que momento a curva composta começa a “disparar”. Nos juros simples, o crescimento acontece sempre pela mesma quantidade, como está na [Equação 6.3](#). Já nos juros compostos, os juros são incorporados ao capital a cada período, e a fórmula passa a ser:

$$M = C(1+i)^t \quad (6.4)$$

Agora o crescimento deixa de ser linear e passa a ser exponencial. A diferença parece pequena no início, mas aumenta dramaticamente com o tempo. E isso pode ser demonstrado pelo trecho do script que calcula simultaneamente os dois modelos financeiros. Veja abaixo que os modelos mostram que o crescimento acumulativo muda completamente o comportamento do sistema.

```
let Ms = C * (1 + i * n);
let Mc = C * Math.pow(1 + i, n);
```

No trecho de código acima, Ms representa o montante em juros simples e Mc o montante em juros compostos. A função de *JavaScript* `Math.pow()` calcula potências. Os resultados são armazenados em listas e enviados ao gráfico, permitindo comparar visualmente a reta linear com a curva exponencial. Assim, a lógica geral é calcular juros simples e compostos para cada instante de tempo, guardar os resultados, e desenhar ambas as curvas.

Alguns cenários para você explorar o código.

1. *Taxa pequena*. Crescimentos exponenciais ainda aparecem com taxas pequenas ?

```
const i = 0.02;
```

2. *Tempo curto*. Por que juros simples e compostos parecem quase iguais no início ?

```
const nMax = 5;
```

3. *Longo prazo*. Por que investimentos de longo prazo favorecem juros compostos ?

```
const nMax = 50;
```

4. *Dívida explosiva*. O que acontece quando taxas altas se acumulam por muito tempo ?

```
const i = 0.25;
```

5. *Comparação de capitais*. Rode o trecho abaixo e depois altere a constante para 10000. O capital inicial altera apenas a escala ou também altera o comportamento matemático ?

```
const C = 500;
```

Na mundo real, quase todos os sistemas financeiros modernos utilizam juros compostos, como financiamentos, cartões de crédito, investimentos, empréstimos e aplicações bancárias. Por isso, compreender crescimento exponencial é uma forma de interpretar decisões financeiras reais. Isso porque o tempo pode ser um aliado poderoso em investimentos, tanto quanto um inimigo perigoso em dívidas.

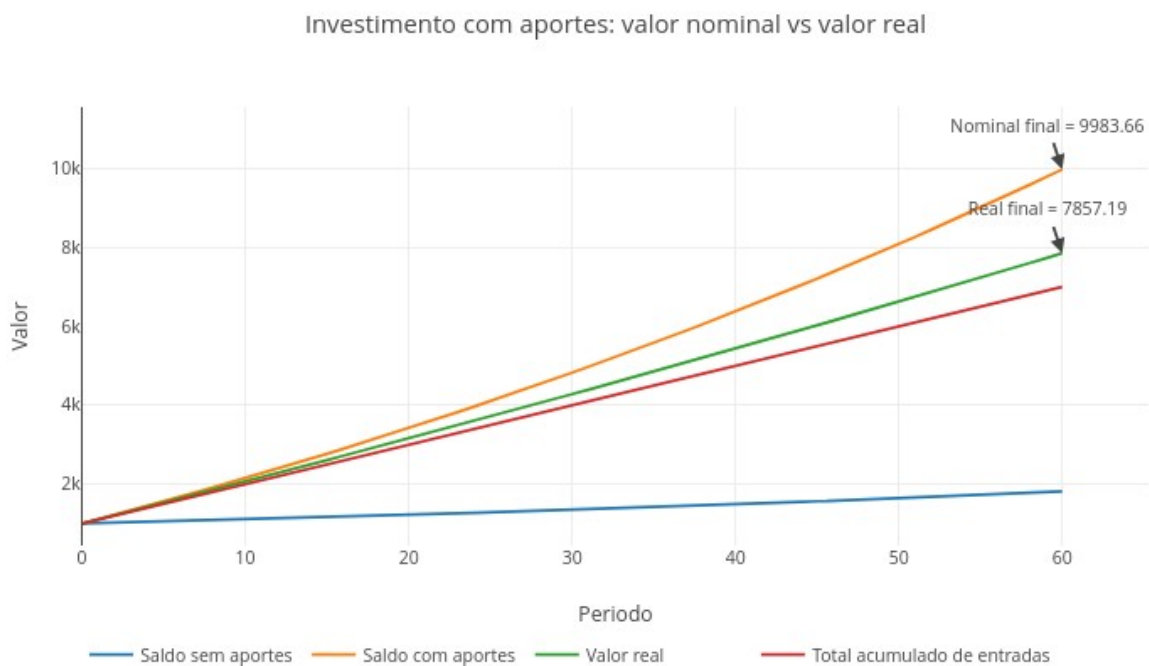
VOLTA PRO MUNDO



Agora vem o impacto real ! Juros compostos fazem parte do nosso cotidiano toda a vez que existem parcelas e/ou tempo envolvidos. É assim com o cartão de crédito, onde os juros compostos jogam contra você, bem como com investimentos, quando os juros compostos conspiram a seu favor. O tempo também pode virar um aliado ou vilão, como nos financiamentos e na inflação.

No mundo real, quase ninguém vive apenas com um capital parado. Em geral, há entradas sucessivas, parcelas, depósitos periódicos, reajustes, perdas inflacionárias e decisões tomadas sob incerteza. E é aí que a modelagem fica mais interessante. Que tal experimentar o script de JSPlotly na sequência, e que simula aportes, inflação e valor real ?

Figura 6.4: Juros, aportes e inflação: quando o valor nominal não conta a história toda. Clique neste [LINK](#) para abrir em nova aba.



Agite antes de usar

Nem todo crescimento de valor significa ganho real. Se o dinheiro aumenta, mas os preços também aumentam, parte desse ganho pode desaparecer. É aí que entra a inflação.

O *app* acima busca aproximar a matemática financeira do mundo real, apresentando o efeito dos juros e dos aportes ou acréscimos, bem como a diferença entre valor nominal e valor real. Assim, é possível verificar

quanto do crescimento observado representa um aumento verdadeiro de poder de compra.

Execute o script com os valores originais abaixo.

```
const modo = "investimento";
const capitalInicial = 1000;
const aportePeriodo = 100;
const taxaJuros = 0.01;
const taxaInflacao = 0.004;
const periodos = 60;
```

Observe as quatro curvas resultantes para saldo (sem e com aportes), valor real, e total acumulado de entradas. Depois, altere a taxa de juros, a inflação e o valor dos aportes. Veja principalmente quando o valor nominal cresce bastante, mas o valor real cresce pouco.

Em matemática financeira, é comum olhar apenas para o valor nominal. Mas o valor nominal mostra apenas a quantidade de dinheiro registrada. Já o valor real tenta responder o quanto esse dinheiro realmente vale em termos de poder de compra. No script, o valor real é calculado por um modelo simplista:

$$\text{valor real} = \frac{\text{valor nominal}}{(1 + \text{inflação})^t}$$

Essa divisão desconta o efeito acumulado da inflação, permitindo perceber que ganhar dinheiro e ganhar poder de compra não são exatamente a mesma coisa.

O código acompanha a evolução do saldo ao longo do tempo. A cada período, o saldo cresce pelos juros:

```
saldoComAporte = saldoComAporte * (1 + taxaJuros) + aportePeriodo;
```

Depois, o script calcula o valor real:

```
let real = saldoComAporte / Math.pow(1 + taxaInflacao, n);
```

E a função `Math.pow()` calcula o efeito acumulado da inflação ao longo dos períodos. Com isso, o gráfico mostra que o valor nominal pode subir, enquanto o valor real cresce menos, ou até perde força. E você fica sabendo se o gráfico mostra crescimento verdadeiro ou apenas crescimento aparente. Um breve tutorial para uso do *app* é dado abaixo.

1. Rode o script no modo investimento;
2. Observe a diferença entre saldo com aportes e saldo sem aportes;
3. Compare o saldo com aportes com o total acumulado de entradas;
4. Veja se os juros realmente geraram ganho acima do que foi depositado;
5. Observe a curva de valor real;
6. Aumente a inflação;
7. Veja como o valor real se distancia do valor nominal;
8. Aumente os aportes;

9. Observe como pequenos depósitos recorrentes podem alterar muito o resultado final;
10. Troque o modo para dívida e interprete o aportePeríodo como novos acréscimos à dívida.

A lógica do script é definir capital inicial, juros, inflação e aportes para cada período, atualizar saldo sem e com aporte, somar o total aportado, corrigir o saldo pela inflação, e desenhar as curvas no gráfico.

Seguem alguns cenários econômicos para você “brincar” sem ferir o bolso (ou a Bolsa!).

1. *Investimento conservador.* O rendimento supera a inflação com folga ou apenas mantém parte do poder de compra ?

```
const taxaJuros = 0.006;
const taxaInflacao = 0.004;
```

2. *Inflação alta.* É possível o dinheiro nominal crescer e, mesmo assim, perder valor real ?

```
const taxaJuros = 0.01;
const taxaInflacao = 0.012;
```

3. *Pequenos aportes constantes.* O tempo torna pequenos aportes relevantes?

```
const aportePeriodo = 50;
const periodos = 120;
```

4. *Aportes maiores.* O resultado final depende mais da taxa, do tempo ou da disciplina de aportar ?

```
const aportePeriodo = 300;
```

5. *Dívida com novos acréscimos.* O que acontece quando uma dívida recebe juros e ainda acumula novos gastos ?

```
const modo = "divida";
const capitalInicial = 1000;
const aportePeriodo = 100;
const taxaJuros = 0.03;
const taxaInflacao = 0.004;
```

Fica claro, portanto, ou deveria ficar, que o valor financeiro é mais uma interpretação que uma quantidade alta ou baixa de dinheiro. Um saldo ou dívida de R\$ 10.000 hoje não significa a mesma coisa que R\$ 10.000 daqui a muitos anos. Na vida real, investimentos, salários, dívidas, financiamentos e aposentadorias são afetados pela inflação. E isso decide junto ao saldo bancário quanto dinheiro você terá no futuro, e quanto desse dinheiro conseguirá comprar.

Por isso, decisões financeiras precisam considerar tempo, taxa de juros, inflação, frequência dos aportes, objetivo da decisão, e a diferença entre ganho nominal e ganho real. Nesse sentido, a matemática financeira serve para ajudar você a tomar melhores decisões.



E SE...

Acho que pra essa seção a gente precisaria de um consultor financeiro. Mas, na ausência de um, tente responder às questões apresentadas separadamente nos scripts do capítulo

Script 1.

- E se a taxa fosse zero ?
- E se a taxa fosse negativa, representando perda de valor ?
- E se duas pessoas investissem o mesmo valor, mas por tempos diferentes ?
- E se o dinheiro crescesse não em linha reta, mas em curva ?

Script 2.

- E se houvesse depósitos mensais além do rendimento ?
- E se a inflação reduzisse o valor real do dinheiro ?
- E se o rendimento variasse ao longo do tempo ?
- E se duas aplicações tivessem a mesma taxa, mas aportes diferentes ?
- E se o crescimento financeiro fosse comparado ao crescimento populacional ou biológico ?

Script 3.

- E se os aportes aumentassem com o tempo ?
- E se a inflação variasse a cada período ?
- E se a taxa de juros fosse menor que a inflação ?
- E se você comparasse investimento, poupança, financiamento e dívida no mesmo gráfico ?
- E se uma decisão aparentemente pequena, repetida por muitos períodos, mudasse completamente o futuro ?

MESMO PADRÃO, OUTROS MUNDOS



O modelo de juros simples ajuda a entender qualquer situação em que uma grandeza cresce sempre pela mesma quantidade. Por isso, essa lógica matemática vai além do dinheiro. Ela também representa cenários como a distância percorrida com velocidade constante, a produção diária de um bem, uma economia mensal fixa, o desgaste de um equipamento ou o aumento regular de uma dívida. A ideia por detrás permanece: todo o crescimento constante produz uma reta.

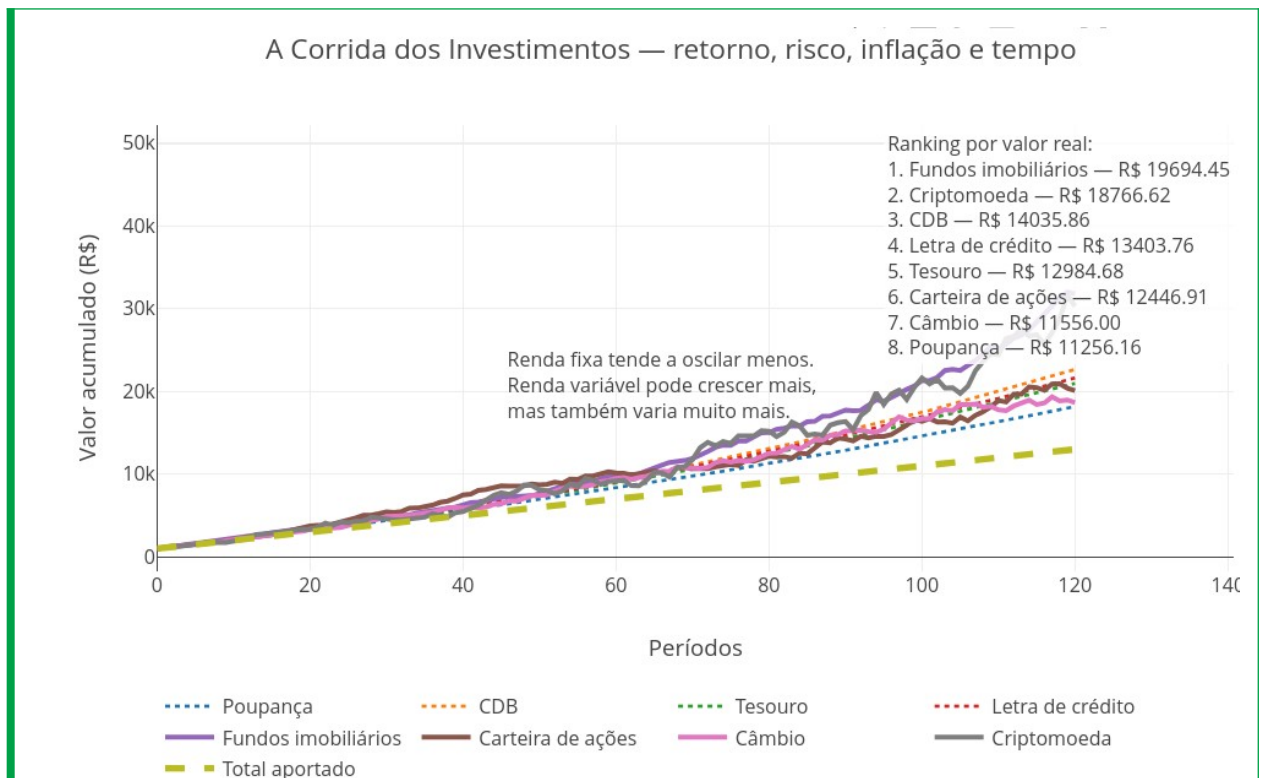
Para juros compostos no mundo real, basta associar sua formulação a contextos em que ocorre crescimento exponencial. Alguns exemplos são o crescimento populacional, epidemias, proliferação bacteriana, reações químicas, propagação de informação por redes sociais (e *fake news*), e evolução tecnológica, entre muitos.

Também é importante desdobrar os frutos desse capítulo dentro do mesmo mundo da Economia e de seu ramo de Finanças, para saber decidir da melhor forma sobre o futuro do seu dinheiro. Isso é válido em diversos panoramas, como a comparação entre compra à vista e parcelamento, a simulação de poupança para um objetivo futuro, a análise de empréstimos e financiamentos, e a discussão entre investir cedo ou investir mais tarde. Também é vital para discutir o efeito da inflação sobre salários e preços, para comparação entre guardar dinheiro em casa, aplicar com rendimento baixo ou aplicar com rendimento maior, bem como para ter uma leitura crítica da propaganda de bancos, cartões e financiamentos.

Essas decisões fazem parte do cotidiano de todos nós. E uma parte dessas decisões está intimamente ligada ao destino que damos para nosso dinheiro quando temos o mínimo para investi-lo. Para fechar esse capítulo, que tal um último JSPlotly, para simular exatamente isso, ou seja, diferentes formas de investimento monetário ?

A proposta é que você experimente diferentes formas de investimento em renda fixa (poupança, CDB, títulos públicos, letras de crédito) e em renda variável (fundos imobiliários, carteira de ações, câmbio, e criptomoeda), e verifique como risco, tempo, aportes, inflação e volatilidade podem mudar o resultado.

Figura 6.5: Renda fixa, renda variável e risco: mesmo dinheiro, futuros diferentes. Clique neste [LINK](#) para abrir em nova aba.



Nesse último script, os valores são hipotéticos e servem apenas para estudo. O objetivo não é recomendar investimentos, mas auxiliar você a compreender modelos de crescimento, risco e decisão.

Agite antes de usar

Imagine duas pessoas investindo, mas cada qual escolhendo um caminho diferente: poupança, CDB (*Certificado de Depósito Bancário*, um título de renda fixa emitido por bancos), títulos públicos, letras de crédito (títulos emitidos por bancos para financiar setores específicos), fundos imobiliários (investimento coletivo negociado na *Bolsa de Valores*), ações (pequenas frações do capital social de uma empresa), câmbio (operação de troca de uma moeda por outra) ou criptomoedas (moedas digitais descentralizadas para transações com segurança e criptografia).

Depois de muitos períodos, os resultados podem ser muito diferentes. E aí permanece a dúvida em saber qual/se o maior resultado final significa a melhor decisão ! Decisões financeiras envolvem retorno, risco, tempo, inflação e comportamento.

Execute o script com os valores originais, e depois altere os perfis para conservador ou agressivo. Observe como o aumento do risco muda a trajetória das curvas.

```
const capitalInicial = 1000;
const aportePeriodo = 100;
const periodos = 120;
const inflacao = 0.004;
const perfil = "moderado";
```

Perceba que a questão não é saber qual investimento é o melhor, mas sim

como diferentes combinações de risco e retorno produzem futuros diferentes.

No script, a renda fixa foi representada com menor oscilação, enquanto que a renda variável foi representada com maior oscilação. Isso permite visualizar diversas coisas. O modelo mostra, por exemplo, que o retorno esperado não é garantia e que o risco significa incerteza. Também permite enxergar que a volatilidade produz subidas e quedas, que a inflação reduz o poder de compra e que o tempo aumenta o efeito das decisões repetidas.

No código, cada investimento possui dois parâmetros principais: o retorno, que representa a tendência média de crescimento, e o vol, que representa a oscilação ao redor dessa tendência (volatilidade). A cada período, o script calcula:

```
saldo = saldo * (1 + taxaPeriodo) + aportePeriodo;
```

Assim, o dinheiro cresce pelos juros ou rendimento daquele período e recebe um novo aporte. No fim, o script também calcula o valor real, descontando o efeito acumulado da inflação.

```
valorReal = valorFinal / Math.pow(1 + inflacao, periodos);
```

Você pode usar o aplicativo como abaixo:

1. Rode o script no perfil "moderado";
2. Observe quais curvas crescem de forma mais regular;
3. Observe quais curvas oscilam mais;
4. Compare renda fixa e renda variável;
5. Aumente o tempo para 240;
6. Veja como o longo prazo amplia diferenças;
7. Aumente a inflação;
8. Observe como o valor real muda a interpretação;
9. Troque o perfil para "agressivo";
10. Veja que maior risco pode gerar maiores ganhos, mas também trajetórias mais instáveis.

Ao final, perceba que você está olhando também o caminho percorrido, e não somente o resultado final. Seguem alguns cenários “econômicos” para você testar.

1. *Perfil conservador*. Uma menor oscilação torna a trajetória mais previsível ?

```
const perfil = "conservador";
```

2. *Perfil agressivo*. Um maior risco sempre melhora o resultado final ?

```
const perfil = "agressivo";
```

3. *Longo prazo*. O tempo favorece todos os investimentos da mesma forma ?

```
const periodos = 240;
```

4. *Inflação mais alta.* Quais investimentos parecem bons nominalmente, mas perdem força em valor real ?

```
const inflacao = 0.01;
```

5. *Sem aportes.* Quanto da evolução vem do rendimento, e quanto vem da disciplina de aportar ?

```
const aportePeriodo = 0;
```

O script usa uma simulação com ruído controlado. Isso significa que algumas curvas não crescem de forma perfeitamente lisa. Elas oscilam, como acontece em muitos fenômenos reais. A lógica geral é definir capital, aportes, inflação e perfil de risco para cada investimento. Daí o código define o retorno médio e a volatilidade, simula período por período, aplica o rendimento, adiciona o aporte, corrige o valor final pela inflação, ordena os resultados por valor real, e finalmente desenha as curvas.

Na mundo real, ninguém conhece exatamente o futuro. Por isso, a matemática financeira não elimina a incerteza mas ajuda a fazer perguntas melhores. Ao invés de querer saber quanto se vai ganhar, pode-se abrir o leque de perguntas para o quanto se pode perder no caminho ou o quanto a inflação pode corroer. E também o quanto depende dos aportes, quanto depende do tempo, e quanto depende do risco assumido. Esse é um jeito de pensar a matemática financeira como cidadania financeira.

POR DENTRO DA FERRAMENTA



Ao longo desta obra, essa seção tem sido dedicada a espelhar o pensamento computacional e a lógica de programação junto aos diversos scripts de JSPlotly, finalizando com um pseudocódigo voltado a um desses scripts em cada capítulo.

Nesse capítulo sobre Matemática Financeira, por exemplo, você observou os conceitos de juros simples no Script no. 1 e de juros compostos no Script no. 2. Aprendeu como a inflação e aportes financeiros podem ajustar valores no mundo real com o Script no. 3, e mesmo como comparar as diversas opções do mercado de investimentos no Script no. 4.

Mas você já deve ter percebido também que, ao longos dos capítulos do livro, aquelas competências digitais passaram a integrar paulatinamente o próprio capítulo, diminuindo a necessidade de sua exclusividade para o tópico que se apresenta. A partir de agora, então, essa seção irá dedicar-se à **verdadeira ferramenta** por detrás de todos os scripts que podem ser feitos com o JSPlotly: a linguagem de programação *JavaScript*.

O entendimento de como opera essa ferramenta permitirá que você compreenda e modifique qualquer código deste livro, ajustando-o às suas preferências ou necessidades, bem como crie outros tantos que desejar. Além disso, poderá entender também o que há por detrás das páginas de internet e criar as suas próprias, como blogs e sites. Isso porque o JavaScript opera silenciosamente quando você clica num botão da tela de seu celular, preenche um formulário ou acompanha uma animação gráfica na internet.

Por óbvio, entretanto, esta “nanométrica” seção não lhe dará condições de sair por aí programando em JavaScript. Mas lhe dará munição básica para que você avance ao próximo nível nessa linguagem, bem como em qualquer outra linguagem de programação que desejar, como Python, Go ou C++. Isso porque o fluxo lógico e os componentes estruturais de qualquer linguagem de programação seguem o mesmo padrão universal.

JavaScript (JS), é uma linguagem de programação moderna, utilizada por grandes empresas e *big techs*, e focada na interatividade entre a página web e o usuário. Diferente de outras linguagens, *JavaScript* não precisa ser compilada separadamente por um programa, pois é interpretada a partir de máquinas homônimas presentes em navegadores comuns (*browser*), como Firefox, Chrome, Safari, Opera, Edge, etc.

Assim, *Javascript (JS)* é uma linguagem para web. Ela completa a tríade essencial, *HTML-CSS-JS*. Enquanto *HTML* preocupa-se com o conteúdo e *CSS* com o estilo, *JavaScript* ocupa-se do comportamento, ou seja, da interatividade do usuário frente à uma página web. Vamos então chamá-la agora carinhosamente por *JS*, somente.

Como ocorre com outras linguagens de programação, a linguagem *JS* possui bibliotecas, que são módulos independentes de códigos para cumprir certas funções. Nesse sentido, *JS* cumpre metade do nome do *JSPlotly*. A outra metade vem da principal biblioteca que é utilizada por

esse para a construção de gráficos, mapas, e alguns outros objetos interativos, [Plotly.js](#).

6.1 Estrutura de JS

Uma linguagem de programação normalmente possui características estruturais comuns. Isso quer dizer que há comandos para declarar uma variável e seu tipo, a entrada e saída de dados (resultados), a criação e uso de funções, e cálculos aritméticos, por exemplo. E tal como outras linguagens de programação, JS opera com uma lógica similar de declarações (palavras-chave, operadores, valores, expressões). Em resumo, a gente dizer as declarações de JS compõem:

1. Palavras-chave: sintaxe da linguagem JS;
2. Operadores: caracteres que realizam operações;
3. Valores: texto, números, verdadeiro/falso (variável "booleana"), "não definido", "nulo";
4. Expressões: trechos de código que produzem um único valor.

Do ponto de vista estrutural, por conseguinte, JS possui características que também são comuns a outras linguagens de programação:

1. Saída de dados;
2. Declaração de variáveis;
3. Tipos de dados;
4. Constantes;
5. Aritmética;
6. Vetores;
7. Funções;
8. Geração de dados aleatórios;
9. Objetos;
10. Operadores;
11. Estruturas de controle;

Por enquanto é só. Se quiser aprender um pouquinho mais de JavaScript, basta virar a página !

7. Matrizes e modelagem - Transformando relações em sistemas manipuláveis



O MUNDO TE CHAMA

Você já percebeu como o mundo moderno se apresenta hoje ? O dinheiro circula digitalmente entre setores variados da economia, dados imensos são conectados em redes sociais, imagens digitais em 3D são criadas por grades numéricas para jogos digitais, a inteligência artificial processa milhões de relações matemáticas ao mesmo tempo por pedaços de palavras (*tokens*), carros autônomos se locomovem sem motorista, e mesmo uma simples consulta de um produto no celular já faz com que você receba uma enxurrada de ofertas parecidas depois.

Essas situações variadas envolvem entradas, transformações numéricas, e saídas. Em todos esses casos, as transformações estão relacionadas a objetos que podem ser ampliados, comprimidos, girados, inclinados ou refletidos. Na base desses cálculos frequentemente existe uma matriz, uma “máquina” de relações matemáticas capaz de transformar posições no espaço.

Depois de explorar, você consegue:

- Representar transformações geométricas usando matrizes
- Relacionar rotações, escalas e deformações com operações matemáticas
- Interpretar matrizes como regras de transformação
- Simular sistemas que evoluem ao longo do tempo
- Observar como pequenas interações alteram trajetórias futuras
- Comparar diferentes cenários usando o mesmo modelo matemático
- Relacionar matrizes com situações do cotidiano, como redes, populações, economia e propagação
- Entender que matrizes podem descrever tanto mudanças no espaço quanto mudanças no tempo
- explorar, modificar e discutir modelos computacionais simples para matrizes.

Figura 7.1: Matrizes permitem transformar situações reais em modelos manipuláveis.

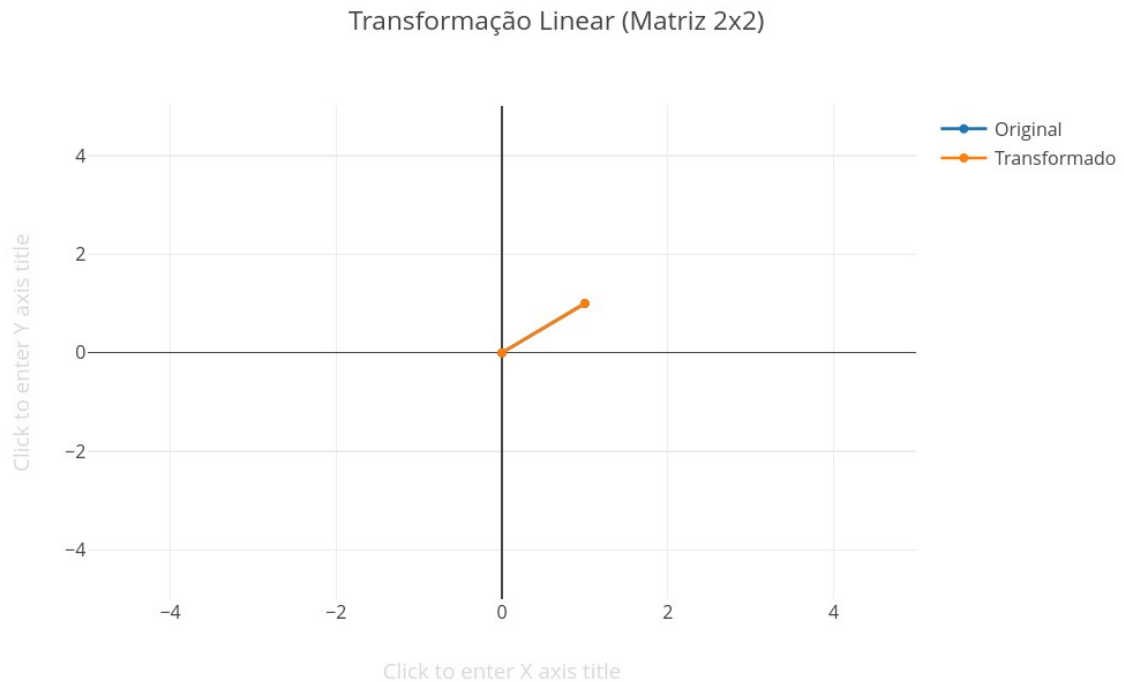
$$\begin{bmatrix} a_1 & b_1 \\ a_2 & b_2 \end{bmatrix}$$

MEXA ANTES DE ENTENDER



Antes de decorar regras, observe uma transformação acontecendo, com auxílio do script de JSPlotly da [Figura 7.2](#). No script você escolhe um ponto, aplica uma regra e o ponto muda de lugar.

Figura 7.2: Transformação linear: a matriz pode funcionar como uma regra que altera a posição de um ponto. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.



O JSPlotly acima permite ver como uma matriz 2×2 modifica um vetor no plano cartesiano, alterando sua direção, tamanho ou orientação. O ponto original e o ponto transformado aparecem no plano. Experimente mudar o valor de x e y, alterar os números da matriz, comparar o vetor original com o vetor transformado, e observar quando o ponto gira, estica, encolhe ou muda de direção.

Agite antes de usar

O *app* funciona como uma visualização geométrica da multiplicação matricial.

No *app*, altere os valores do ponto inicial:

```
input_x
input_y
```

Depois experimente modificar os elementos da matriz:

```
input_a
input_b
input_c
input_d
```

Observe como o vetor azul original é transformado em um novo vetor. Tente descobrir quais valores ampliam o vetor, quais refletem o vetor,

quais trocam as coordenadas, quais deformam o plano, e quais praticamente não alteram nada. Em suma, tente perceber o comportamento geométrico da matriz. Você pode o *app* da seguinte forma:

- a. Escolha um ponto inicial alterando:

```
input_x
input_y
```

- b. Modifique os elementos da matriz:

```
input_a
input_b
input_c
input_d
```

Observe a direção do vetor, seu tamanho e orientação, bem como reflexões e deformações.

- c. Compare o vetor original com o transformado.
d. Tente prever o resultado antes de rodar novamente.

Seguem também alguns cenários para você experimentar.

1. *Ampliação horizontal*. O eixo x é ampliado.

```
input_a = 2
input_b = 0
input_c = 0
input_d = 1
```

2. *Ampliação vertical*. O eixo y cresce.

```
input_a = 1
input_b = 0
input_c = 0
input_d = 2
```

3. *Reflexão vertical*. O vetor é espelhado no eixo y .

```
input_a = -1
input_b = 0
input_c = 0
input_d = 1
```

4. *Reflexão horizontal*. O vetor é espelhado no eixo x .

```
input_a = 1
input_b = 0
input_c = 0
input_d = -1
```

5. *Troca entre coordenadas*. As posições x e y trocam de lugar.

```
input_a = 0
input_b = 1
input_c = 1
input_d = 0
```

6. *Inclinação do plano*. O plano sofre uma deformação inclinada.

```
input_a = 1
input_b = 1
input_c = 0
input_d = 1
```

Agora mais alguns cenários, dessa vez muito usados em computação e engenharias, e que remetem a conceitos utilizados na organização de dados e solução de sistemas complexos. Eles falam sobre alguns “tipos de matrizes” que produzem comportamentos geométricos diferentes no plano. Um exemplo é a modelagem que ajusta pontos experimentais a uma linha (regressão linear) ou a uma curva (regressão não linear). Seguem esses cenários.

Considere inicialmente a matriz:

$$\begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} (7.1)$$

1. *Matriz identidade*. Nada muda no vetor transformado. A matriz atua como “não fazer nada”. Apesar disso, esse tipo de matriz é essencial para resolver sistemas de equações lineares e transformações geométricas envolvidas com matriz inversa.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} (7.2)$$

No script fica assim:

```
input_a = 1
input_b = 0
input_c = 0
input_d = 1
```

8. *Matriz simétrica*. Os elementos fora da diagonal principal são iguais. Esse tipo de matriz aparece muito em Física, computação gráfica e análise de dados, especialmente em sistemas onde as relações entre variáveis são recíprocas.

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} (7.3)$$

```
input_a = 1
input_b = 2
input_c = 2
input_d = 1
```

10. *Matriz nula*. Todos os elementos são zero. Nesse caso, qualquer vetor é transformado no vetor nulo, fazendo com que o ponto desapareça na origem.

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} (7.4)$$

```
input_a = 0
input_b = 0
```

```
input_c = 0
input_d = 0
```

11. *Matriz transposta.* A transposta é obtida trocando linhas por colunas. Compare essa matriz com sua versão original, e observe como pequenas mudanças reorganizam completamente a transformação geométrica.

$$\begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix} (7.5)$$

```
input_a = 1
input_b = -1
input_c = 2
input_d = 1
```

12. *Matriz diagonal.* Apenas os elementos da diagonal principal são diferentes de zero. Essas matrizes normalmente ampliam ou comprimem os eixos independentemente.

$$\begin{bmatrix} 3 & 0 \\ 0 & 0.5 \end{bmatrix} (7.6)$$

```
input_a = 3
input_b = 0
input_c = 0
input_d = 0.5
```

13. *Matriz inversa.* Algumas transformações podem ser desfeitas por outra matriz. A matriz inversa atua como um caminho de volta, restaurando a configuração original do vetor. Se a matriz ampliou o eixo x por 2, a matriz inversa reduz o eixo x pela metade, retornando o vetor à escala original. Essa matriz é usada em modelagem linear (quando se tem pontos que parecem desenhar uma reta) e não linear (quando se tem pontos que parecem desenhar uma curva). Considerando a matriz original abaixo:

$$\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} (7.7)$$

Ela amplia o eixo x por um fator 2. Então sua matriz inversa faz o caminho contrário:

$$\begin{bmatrix} 0.5 & 0 \\ 0 & 1 \end{bmatrix} (7.8)$$

No script:

```
input_a = 0.5
input_b = 0
input_c = 0
input_d = 1
```

Esse cenários todos reproduzem de forma simplificada algumas transformações matriciais que são comumente observadas em vários campos, tais como computação gráfica, modelagem 3D, jogos digitais,

inteligência artificial, robótica, Engenharia, Física, processamento de imagens e animações. Toda vez que um objeto gira, muda de escala ou se move em ambientes digitais, existe uma grande chance de matrizes estarem trabalhando silenciosamente por trás.

Essas transformações acima são denominadas por transformações lineares, e são comumente utilizadas para representar uma ação sobre um objeto. O efeito final permite que o objeto experimente *rotação*, *ampliação*, *redução*, *reflexão*, *cisalhamento* (corte ou deformação), *mistura*, *troca*, *fluxo*, *propagação* e *dependência* entre variáveis.

Por dentro do script

Uma matriz pode ser entendida como uma regra matemática que recebe um vetor e devolve outro. Pode-se considerar o vetor original como:

$$\begin{bmatrix} x \\ y \end{bmatrix} \quad (7.9)$$

e a matriz como:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad (7.10)$$

Por exemplo, se no script está escrito:

```
input_a = 2
input_b = 1
input_c = 0
input_d = 3
```

a matriz utilizada será:

$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

Essa matriz é a que será aplicada ao vetor original do plano cartesiano durante a transformação. Então o novo vetor será:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax+by \\ cx+dy \end{bmatrix} \quad (7.11)$$

Portanto, a nova coordenada x depende da combinação entre x e y . Já a nova coordenada y passa a depender dessa mistura, e com ambas as transformações controladas pela matriz. Quer saber como funciona essa transformação no código da [Figura 7.2](#) ?

O script apresenta dois vetores, um original e outro transformado pela matriz. A transformação ocorre pelas equações:

$$x' = ax + by \quad (7.12)$$

$$y' = cx + dy \quad (7.13)$$

Cada elemento da matriz possui uma função geométrica. Alguns valores

ampliam o vetor, outros comprimem, alguns refletem, e outros misturam os eixos x e y . O interessante é que o plano inteiro pode ser deformado usando apenas quatro números.

No código, o trecho abaixo faz exatamente a multiplicação da matriz pelo vetor.

```
let x2 = a * x + b * y;
let y2 = c * x + d * y;
```

A linha que segue desenha o vetor original saindo da origem.

```
x: [0, x],
y: [0, y]
```

Já a próxima, desenha o vetor transformado.

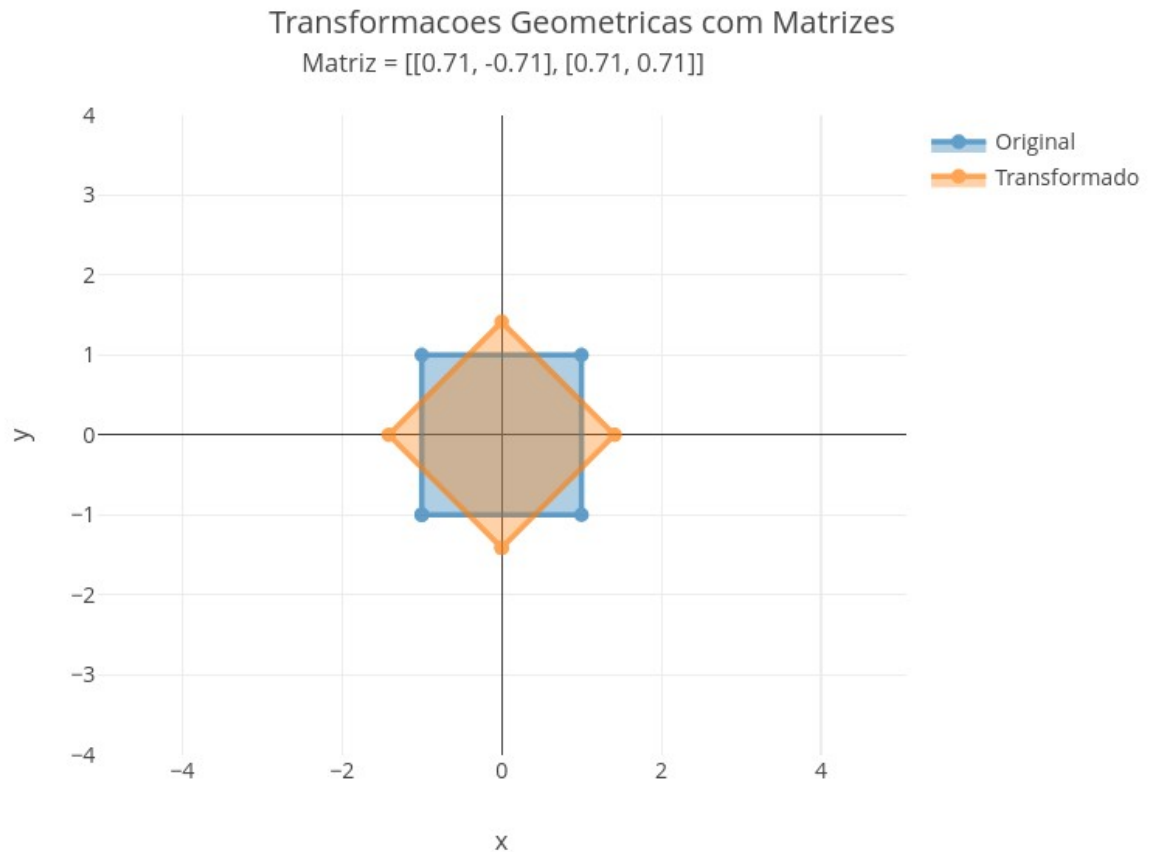
```
x: [0, x2],
y: [0, y2]
```



FAZENDO APARECER

O *app* da seção anterior possibilitou perceber que uma matriz 2×2 pode transformar um ponto ou vetor no plano. Mas imagine se, em vez de transformar apenas um vetor, você pudesse transformar uma figura inteira ! A ideia é a mesma, mas aplicada a todos os pontos que formam o quadrado. Fica então mais fácil perceber como rotações, ampliações e cisalhamentos podem ser descritos por matrizes. E esse é o objetivo do próximo script.

Figura 7.3: Quando uma matriz transforma todos os pontos de uma figura, ela transforma a aparência da figura. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.



Agite antes de usar

No *app*, altere primeiro o tipo de transformação. Você pode usar “rotacao”, “escala” ou “cisalhamento”:

```
input_tipo = "rotacao"
```

Depois modifique os valores de:

```
input_angulo  
input_escala
```

Observe como o quadrado original é transformado. Tente perceber quais transformações preservam a forma, quais alteram o tamanho e quais deformam a figura.

a. Escolha o tipo de transformação:

```
input_tipo = "rotacao"
```

b. Se escolher rotação, altere:

```
input_angulo = 45
```

c. Se escolher escala ou cisalhamento, altere:

```
input_escala = 1.5
```

d. Compare o quadrado original com o transformado.

e. Observe se a figura apenas gira, aumenta, diminui ou se deforma.

f. Tente prever o resultado antes de executar novamente.

Para ajudar diretamente nessas transformações, seguem alguns cenários interessantes.

1. *Rotação suave.* O quadrado gira ao redor da origem, mas mantém seu tamanho e sua forma.

```
input_tipo = "rotacao"  
input_angulo = 30  
input_escalas = 1.5
```

2. *Rotação de 90 graus.* A figura gira um quarto de volta no plano cartesiano.

```
input_tipo = "rotacao"  
input_angulo = 90  
input_escalas = 1.5
```

3. *Rotação negativa.* O quadrado gira no sentido horário.

```
input_tipo = "rotacao"  
input_angulo = -45  
input_escalas = 1.5
```

4. *Rotação completa.* A figura retorna praticamente à posição original após uma volta inteira.

```
input_tipo = "rotacao"  
input_angulo = 360  
input_escalas = 1.5
```

5. *Ampliação uniforme.* O quadrado aumenta de tamanho sem deformar sua geometria.

```
input_tipo = "escala"  
input_angulo = 45  
input_escalas = 2
```

6. *Redução uniforme.* A figura diminui, mas preserva a forma original.

```
input_tipo = "escala"  
input_angulo = 45  
input_escalas = 0.5
```

7. *Escala negativa.* A figura sofre inversão geométrica juntamente com mudança de tamanho.

```
input_tipo = "escala"  
input_angulo = 45  
input_escalas = -1
```

8. *Cisalhamento moderado.* O quadrado começa a inclinar lateralmente.

```
input_tipo = "cisalhamento"  
input_angulo = 45  
input_escalas = 1
```

9. *Cisalhamento intenso.* A deformação lateral se torna bastante evidente.

```
input_tipo = "cisalhamento"
```

```
input_angulo = 45
input_escalas = 2
```

10. *Cisalhamento negativo.* A inclinação ocorre para o lado oposto.

```
input_tipo = "cisalhamento"
input_angulo = 45
input_escalas = -1
```

11. *Quase uma reta.* Um cisalhamento extremo começa a deformar fortemente a figura.

```
input_tipo = "cisalhamento"
input_angulo = 45
input_escalas = 5
```

12. *Rotação pequena.* Pequenos ângulos produzem transformações discretas, úteis em animações suaves.

```
input_tipo = "rotacao"
input_angulo = 5
input_escalas = 1.5
```

13. *Ampliação extrema.* A figura cresce muito rapidamente.

```
input_tipo = "escala"
input_angulo = 45
input_escalas = 3
```

14. *Figura espelhada e ampliada.* A escala negativa combinada com módulo maior que 1 cria inversão e ampliação simultaneamente.

```
input_tipo = "escala"
input_angulo = 45
input_escalas = -2
```

15. *Sem transformação.* A figura permanece praticamente inalterada.

```
input_tipo = "escala"
input_angulo = 45
input_escalas = 1
```

Por dentro do script

No JSPlotly anterior, a matriz era preenchida diretamente com os valores a, b, c e d. Agora, esses valores são calculados automaticamente a partir do tipo de transformação escolhido.

Para uma rotação, por exemplo, a matriz é:

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \quad (7.14)$$

Para uma escala uniforme, a matriz é:

$$\begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix} \quad (7.15)$$

Para um cisalhamento horizontal, a matriz é:

$$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix} \quad (7.16)$$

Em todos os casos, a regra geral continua sendo a mesma:

\$\$

=

\$\$\{\text{\#eq-transformacaoGeral2}\}

A diferença é que agora essa transformação é aplicada a vários pontos da figura, um por vez.

O quadrado inicial é formado por listas de coordenadas:

```
let x = [-1, 1, 1, -1, -1];
let y = [-1, -1, 1, 1, -1];
```

Cada par (x[i], y[i]) representa um vértice do quadrado, e o último ponto repete o primeiro para fechar a figura. O trecho abaixo cria listas vazias para guardar os novos pontos transformados:

```
let x2 = [];
let y2 = [];
```

Em seguida, um laço (*loop*) percorre todos os pontos do quadrado:

```
for (let i = 0; i < x.length; i++) {
  x2.push(a*x[i] + b*y[i]);
  y2.push(c*x[i] + d*y[i]);
}
```

Esse último trecho é o coração do script. Ele aplica a matriz de transformação em cada vértice da figura. Assim, a mesma multiplicação matricial que antes transformava um ponto, agora transforma uma figura completa.

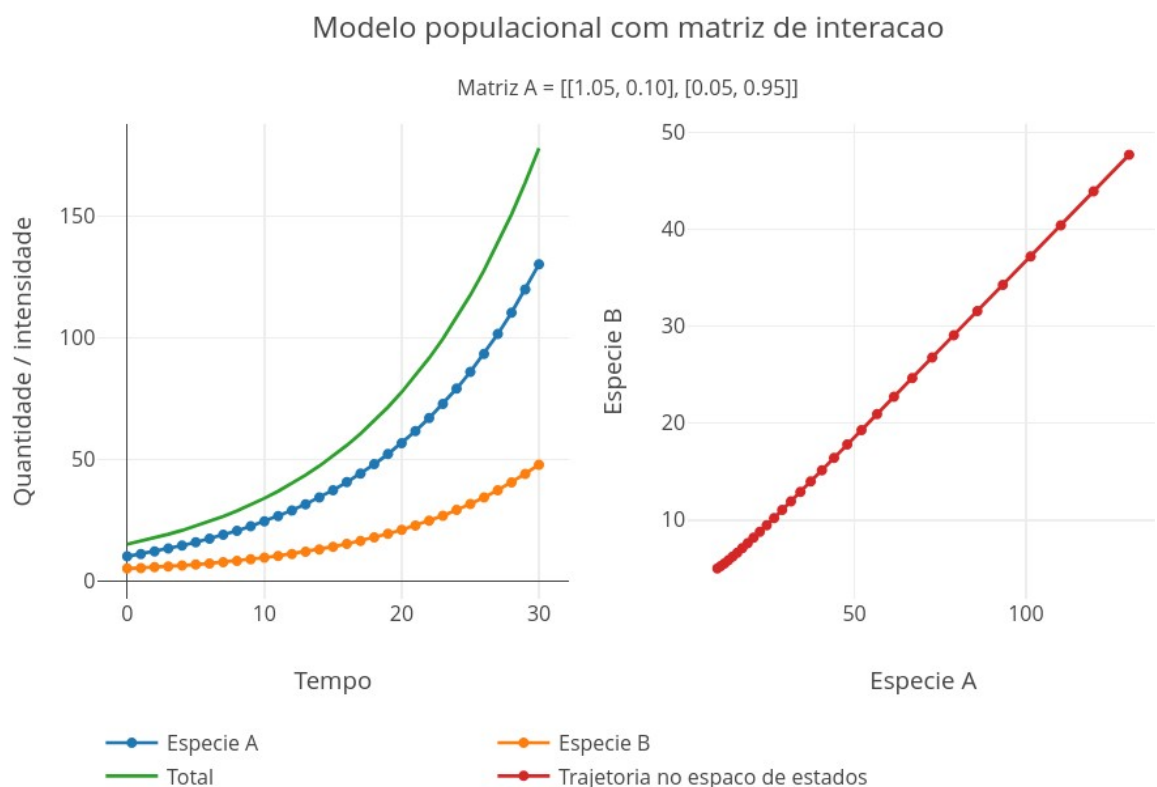
VOLTA PRO MUNDO



Em muitos problemas do mundo real, matrizes vão muito além de mudar posições no espaço. Elas descrevem como sistemas inteiros podem evoluir ao longo do tempo. Crescimento populacional, propagação de epidemias, predição por redes neurais, transporte e sinalização de trânsito (semáforos), circulação de dinheiro entre setores econômicos, redes de comunicação, e mesmo algoritmos de inteligência artificial são modelados por matrizes. Nesses casos, cada multiplicação matricial representa uma nova etapa do sistema, permitindo acompanhar tendências, equilíbrios, explosões de crescimento ou colapsos dinâmicos.

O próximo script propõe isso, usar matriz para transformar o estado de um sistema ao longo do tempo.

Figura 7.4: Matrizes e modelagem dinâmica: pequenas regras de interação podem produzir grandes mudanças ao longo do tempo. Clique neste [LINK](#) para abrir em nova aba.



O script mostra como uma matriz pode representar a evolução de um sistema ao longo do tempo. A ideia é simples:

$$estado(t+1) = A \cdot estado(t)$$

Ou seja, o estado futuro depende do estado atual e de uma matriz de transformação. Essa matriz define como duas partes do sistema influenciam uma à outra. Dependendo do cenário escolhido, essas partes podem representar duas espécies, dois setores econômicos, suscetíveis e infectados, dois nós de uma rede, e tantos outros pares.

O gráfico da esquerda mostra a evolução temporal. Ele responde perguntas como se A ou B crescem ou diminuem, se o total se conserva, ou se o sistema explode ou estabiliza. Já o gráfico da direita mostra a trajetória no espaço de estados, e em que cada ponto representa uma combinação entre A e B.

Agite antes de usar

Execute o script com algum cenário como “população”, “economia”, “epidemia”, ou “rede”.

```
const cenário = "populacao";
const passos = 30;
const intensidade = 1.0;
const inicialA = 10;
const inicialB = 5;
```

Depois altere o cenário e observe como a mesma estrutura matemática produz comportamentos diferentes. Em seguida, altere a intensidade (0.3; 1.5, por exemplo). E veja como a força das interações muda a trajetória do sistema.

A matriz A determina o que acontece com cada componente do sistema.

Por exemplo:

```
novoA = A[0][0] * estado[0] + A[0][1] * estado[1];
novoB = A[1][0] * estado[0] + A[1][1] * estado[1];
```

Isso significa que o novo valor de A depende de quanto A preserva de si mesmo, e de quanto B influencia A. Já o novo valor de B depende de quanto A influencia B, e de quanto B preserva de si mesmo.

Você pode usar o *app* seguindo o rápido tutorial abaixo.

1. Rode o script com cenário = "populacao";
2. Observe as curvas de A, B e Total no gráfico da esquerda;
3. Observe a trajetória no espaço de estados no gráfico da direita;
4. Troque para cenário = "economia";
5. Veja se o total cresce, diminui ou se estabiliza;
6. Troque para cenário = "epidemia";
7. Observe como suscetíveis e infectados se relacionam;
8. Troque para cenário = "rede";
9. Compare o fluxo entre os dois nós;
10. Aumente intensidade;
11. Veja como as interações ficam mais fortes;
12. Altere inicialA e inicialB;
13. Observe como o ponto de partida muda toda a trajetória.

Perceba nessa “*brincadeira*” que, em adição a seus valores iniciais, o sistema muda por causa da matriz que governa as relações. Seguem alguns cenários interessantes, também.

1. *População com interação fraca.* Quando as interações são fracas, cada componente mantém mais seu próprio comportamento ?

```
const cenário = "populacao";
const intensidade = 0.3;
```

2. *População com interação forte.* Uma pequena influência entre espécies pode se amplificar ao longo do tempo ?

```
const cenário = "populacao";
const intensidade = 1.8;
```

3. *Economia entre setores.* Um setor mais forte no início altera o fluxo econômico futuro ?

```
const cenário = "economia";
const inicialA = 20;
const inicialB = 5;
```

4. *Propagação simplificada.* Como a interação entre suscetíveis e infectados altera a evolução do sistema ?

```
const cenário = "epidemia";
const intensidade = 1.2;
```

5. *Rede de conexões.* Em uma rede, o ponto inicial muda o caminho da informação ou do fluxo ?

```
const cenário = "rede";
const inicialA = 5;
const inicialB = 20;
```

7. *Muitos passos.* O comportamento de longo prazo revela crescimento, queda, equilíbrio ou explosão ?

```
const passos = 80;
```

Por dentro do script

O script começa escolhendo um cenário:

```
const cenário = "populacao";
```

Cada cenário define uma matriz diferente. No caso populacional, por exemplo:

```
A = [
  [1.05, 0.10 * intensidade],
  [0.05 * intensidade, 0.95]
];
```

Essa matriz indica que:

- a espécie A cresce um pouco por si mesma;
- a espécie B influencia A;
- a espécie A influencia B;
- a espécie B diminui levemente por si mesma.

Depois o script define o estado inicial:

```
let estado = [inicialA, inicialB];
```

Em seguida, aplica repetidamente a matriz ao estado atual:

```
let novoA = A[0][0] * estado[0] + A[0][1] * estado[1];
```

```
let novoB = A[1][0] * estado[0] + A[1][1] * estado[1];  
estado = [novoA, novoB];
```

Cada repetição representa um novo passo no tempo e, por isso, uma matriz simples consegue gerar uma trajetória inteira.

E SE... 

Agora é hora de ir além....além até mesmo do bom senso ! Mas é pra isso que serve essa seção, certo ? Para refletir com pé no chão, mas permitindo-se colocar a cabeça nas nuvens.

- E se aumentar o ângulo de rotação fizesse a figura “girar” exatamente como um objeto em um jogo ou aplicativo ?
- E se ampliar a escala fosse parecido com usar *zoom* em uma imagem, mas controlado por uma matriz ?
- E se um pequeno cisalhamento fosse suficiente para deformar completamente uma figura após várias transformações ?
- E se a mesma matriz pudesse transformar tanto desenhos geométricos quanto populações, epidemias e redes ?
- E se uma pequena mudança na intensidade das interações alterasse completamente o futuro de um sistema ?
- E se duas espécies comesçassem quase iguais, mas terminassem muito diferentes após vários passos da simulação ?
- E se um pequeno número inicial de infectados acabasse dominando o sistema inteiro ?
- E se redes sociais, trânsito ou fluxo de informações funcionassem de modo parecido com os modelos do script ?
- E se o “estado atual” de um sistema influenciasse continuamente o próximo estado, como acontece em hábitos, economia ou estudos ?
- E se matrizes fossem uma forma de descrever mudanças que acontecem no tempo e no espaço ao mesmo tempo ?
- E se o trajeto mostrado no espaço de estados fosse parecido com acompanhar a evolução de escolhas ao longo da vida ?
- E se transformar figuras no plano e prever comportamentos futuros fossem apenas duas faces da mesma matemática ?

MESMO PADRÃO, OUTROS MUNDOS



Você já deve ter percebido, pelos parágrafos anteriores, que não há propriamente um conjunto ou outro de aplicações para matrizes. Quem sabe a limitação para seu uso seja aquela ditada pela criatividade humana, somente. Imagine, por exemplo, um desses aplicativos de celular ou computador que modificam um rosto ou uma paisagem. Uma imagem digital pode ser pensada como uma matriz de pixels, onde cada número representa uma intensidade de cor ou luminosidade. Modificar essa matriz pode clarear a imagem, escurecer a imagem, inverter cores, detectar bordas, borrar regiões, ou realçar detalhes. Isso é a “alma” dos manipuladores de imagens !

Uma rede social também pode ser representada por uma matriz de conexões. Se duas pessoas estão conectadas, a matriz registra essa relação, permitindo estudar influência, grupos, centralidade, propagação de informação e formação de comunidades. Em jogos e animações digitais as matrizes ajudam a mover personagens, girar objetos, alterar perspectiva, simular câmeras e transformar mundos 2D e 3D.

Em inteligência artificial, redes neurais usam muitas operações matriciais. Dados entram como vetores ou matrizes, pesos são organizados em matrizes, e o resultado é uma sequência de transformações, fazendo da IA uma grande cadeia de transformações matemáticas. Também em Ecologia matrizes podem ser empregadas para representar reprodução, mortalidade, competição, migração, e transição entre fases da vida. Isso permite projetar cenários para o crescimento da população, como sua estabilização, colapso ou explosão.

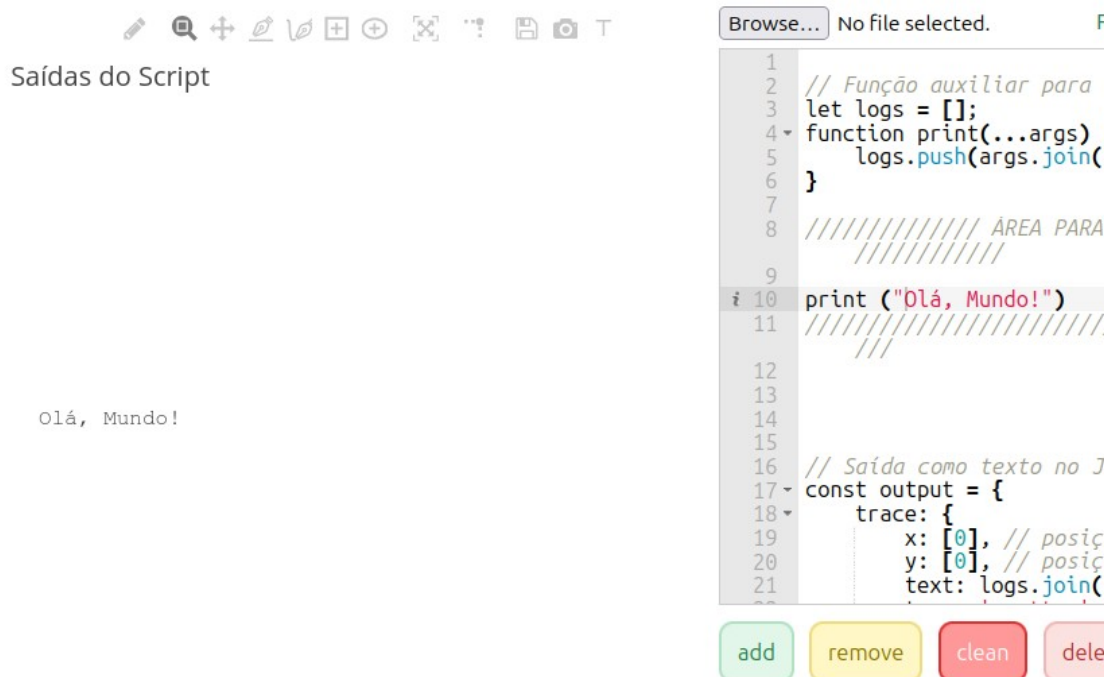
POR DENTRO DA FERRAMENTA



Nessa seção, damos sequência a um aprendizado panorâmico em *JavaScript*, voltado ou não à edição de objetos com JSPlotly, como realizado no capítulo anterior. Nesse sentido, segue um pouco sobre a estrutura de *JS* para aritmética e *arrays* (vetores).

E agora um truque bem bacana !!! **Você poderá treinar a linguagem JavaScript utilizando um JSPlotly específico para isso !**. Esse JSPlotly personalizado está disponível na [Figura 7.5](#) abaixo. Abra-o num navegador e deixe-o quietinho lá. Quando quiser rodar algum trecho desses treinamentos, basta copiar, colar o trecho no campo “ÁREA PARA DIGITAR COMANDOS”, e rodá-lo. Muito legal, não ?!

Figura 7.5: JSPlotly personalizado para treinamento da linguagem JavaScript
[JSPlotly_treinoJS.html](#). Basta clicar com o botão direito do mouse para abri-lo em outra janela do navegador. Deixe-o quietinho lá. Quando quiser executar algum trecho de código ao longo desse treinamento, basta copiar, colar no campo “ÁREA PARA DIGITAR COMANDOS”, e rodar no JSPlotly personalizado.



Aritmética

Diversas operações matemáticas são possíveis, quer pelo *JS* puro, quer pela instalação de bibliotecas específicas, tais como *Plotly.js* utilizada no JSPlotly, *mathjs* ou *numjs*. Segue abaixo uma lista de operações aritméticas realizadas com a linguagem:

1. + : adição de números ou concatenação de texto (*strings*);
2. - : subtração;
3. * : multiplicação;
4. / : divisão;
5. % : módulo;
6. ++ : incremento de valor;
7. -- : decremento;

8. ****** : exponenciação.

Obs: o operador de módulo % divide o primeiro operando pelo segundo operando e retorna o resto.

Os exemplos a seguir ilustram o uso desses operadores aritméticos, bem como de algumas constantes matemáticas. Para testá-los, cole no *JSPotly* customizado, como informado acima. Você pode experimentá-los separadamente ou em bloco.

```
let resultado = 0;

resultado = 5 + 3;           print("5 + 3 = ", resultado);
// 8
resultado = 10 - 4;         print("10 - 4 = ", resultado);
// 6
resultado = 6 * 7;          print("6 * 7 = ", resultado);
// 42
resultado = 20 / 5;         print("20 / 5 = ", resultado);
// 4
resultado = 17 % 3;         print("17 % 3 = ", resultado);
// 2
resultado = 2 ** 3;         print("2 ** 3 = ", resultado);
// 8
let contador = 5; contador++; print("contador++ = ", contador);
// 6
let decrementar = 8; decrementar--; print("decrementar-- = ",
decrementar); // 7
let soma = 10; soma += 5;    print("soma += 5 = ", soma);
// 15
let diferenca = 20; diferenca -= 7; print("diferença -= 7 = ",
diferença); // 13
let produto = 3; produto *= 4;   print("produto *= 4 = ",
produto); // 12
let quociente = 24; quociente /= 3; print("quociente /= 3 = ",
quociente); // 8
resultado = Math.sqrt(25);       print("sqrt(25) = ", resultado);
// 5
resultado = Math.abs(-15);       print("abs(-15) = ", resultado);
// 15
resultado = Math.floor(7.8);     print("floor(7.8) = ",
resultado); // 7
resultado = Math.ceil(7.2);     print("ceil(7.2) = ", resultado);
// 8
resultado = Math.round(9.4);     print("round(9.4) = ",
resultado); // 9
resultado = Math.max(42, 27);    print("max(42,27) = ",
resultado); // 42
resultado = Math.min(42, 27);    print("min(42,27) = ",
resultado); // 27
resultado = Math.sin(Math.PI/2); print("sin(π/2) = ", resultado);
```

Arrays

Um *array* em *JavaScript* representa uma estrutura de dados para

armazenamento de elementos ordenados. Esses elementos podem envolver qualquer tipo de dado, como números, strings, objetos, etc. Na prática, um *array* pode ser criado, acessado, filtrado, ou mapeado. Seguem exemplos:

Criação

```
const numeros = [1, 2, 3, 4, 5];
const cores = ['vermelho', 'verde', 'azul'];
```

Acesso

```
print(numeros[0]);
print(cores[2]);
```

Ou seja, para você acessar o *array* `numeros`, por exemplo, basta colar e rodar o seguinte trecho no JSPotly de treinamento:

```
const numeros = [1, 2, 3, 4, 5];
const cores = ['vermelho', 'verde', 'azul'];

print(numeros[0]);
```

Mapeamento (*map*)

```
const numeros = [1, 2, 3, 4, 5];
const quadrados = numeros.map(numero => numero * numero);
print(quadrados);
```

Uma palavrinha sobre métodos

Observe que, diferente do que já foi explicitado, agora aparece um comando múltiplo, como “*numeros.map*”. Existe uma infinidade de comandos com esta sintaxe em *JS*. Ele representa a função que será aplicada a cada elemento do *array* *numeros* pelo *método* *map*. Além disso, surge a sintaxe “=>” ou sintaxe de *arrow function* (função de seta), uma forma concisa de atribuir um mapeamento. Assim, o comando inteiro:

Em *JavaScript* *objetos* são tratados como no mundo real, ou seja, possuem *atributos* e *comportamentos*. Atributos descrevem as características que um objeto possui, e os comportamentos as ações que podem realizar. A notação em *JS* para referenciar as propriedades de um objeto podem ser, alternativamente:

```
objeto.propriedade
```

```
...ou...
```

```
objeto[propriedade]
```

Voltando ao exemplo acima e esmiuçando-o um pouquinho mais, o *método* *map()* (`numeros.map`) é chamado no *array* *numeros* (um *Array.prototype*). O *protótipo* permite transmitir propriedades e métodos para um objeto *JS*. No caso, o *método* *map()* itera sobre cada elemento do *array* original, aplica uma função a cada um deles e retorna um novo *array* com os resultados. Assim, `numero => numero * numero` é a *função* de

seta. E nesse caso, *numero* é o parâmetro de entrada da função, um atalho para o elemento atual do *array* sendo processado pelo *map*. Em adição, “=>” representa o separador entre o(s) parâmetro(s) e o corpo da função. Finalmente, “*numero * numero*” é a expressão que é executada, com o resultado sendo retornado pela função. Seguem outros exemplos para arrays e mapeamento:

```
const x = [0, 1, 2, 3, 4];
const y = x.map(valor => Math.pow(2, valor))
print(y);

const Vmax = 100;
const Km = 10;
const S_values = Array.from({length: 10}, (_, i) => i * 5);

const V_values = S_values.map(S => (Vmax * S) / (Km + S));

print("Valores de S:", S_values);
print("Valores de V:", V_values);
```

JavaScript, uma linguagem orientada a objetos

Ainda que o esforço para traduzir os comandos acima possa auxiliar como funciona a linguagem, é por óbvio esperar uma certa confusão de termos. Sendo assim, vamos subindo de nível!

Como diz o título acima, *JS* é uma linguagem orientada a objetos. Isso significa que o código é organizado em torno de *objetos*, que combinam dados (*propriedades*) e comportamentos (*métodos*) para modelar entidades do mundo real. Isso permite um código mais modular, reutilizável e fácil de gerenciar, ao organizar a lógica em “*partes*” menores e mais compreensíveis (*pensamento computacional*).

Objetos, propriedades, e métodos

Um *objeto* é uma estrutura que agrupa dados e comportamentos. Os dados ficam guardados em *propriedades*, e os comportamentos são implementados por *métodos* (funções dentro do objeto). Assim, propriedade são pares nome-valor armazenados dentro de um objeto, podendo variar como números, textos, arrays, e outros objetos. Já os métodos são funções associadas a um objeto, e que descrevem ações que ele pode executar.

Objetos em *JS* “[podem ser comparados a objetos da vida real](#)”, e guardam informações relacionadas entre si. Por exemplo, o objeto “*ponto*” abaixo representa um ponto no plano:

```
const ponto = { x: 2, y: 5 }; // objeto com duas propriedades
```

Assim, as propriedades são as características do objeto:

```
print(ponto.x); // mostra 2
print(ponto.y); // mostra 5
```

“Propriedades de objetos são basicamente as mesmas que variáveis normais em JavaScript, exceto pelo fato de estarem ligadas a objetos”. Propriedades podem ser acessadas por colchetes, [...].

Assim, um objeto pode ser encarado como um “pacote” contendo nome, conteúdo e ferramentas próprias. Virtualmente, tudo em *JavaScript* — arrays, funções, strings, números — é, de alguma forma, um objeto com suas propriedades e métodos. Segue um exemplo:

```
const carro = {
  marca: "Toyota",      // propriedade (dado)
  ano: 2022,            // propriedade
  ligar: function() {   // método (ação)
    console.log("O carro está ligado!");
  }
};

carro.ligar();          // chama o método
console.log(carro.marca); // acessa a propriedade
```

Reforçando (não custa nada...), um objeto *JavaScript* envolve uma coleção de propriedades, onde cada propriedade é definida por um par chave-valor, permitindo uma representação ordenada de elementos. Objetos podem conter diferentes tipos de dados como valores, como números, strings, booleanos, arrays, outras funções e até mesmo outros objetos. Objetos criados são facilmente acessados por seus atributos. Seguem exemplos para testes.

Criação

```
const pessoa = {
  nome: 'João',
  idade: 30,
  cidade: 'São Paulo'
};
```

Acesso

```
print(pessoa.nome);
print(pessoa['idade']);
```

Atributos

```
pessoa.profissao = 'Engenheiro';
print(pessoa);
```

Para um exemplo fechado de objetos:

```
let pessoa = {
  nome: "João",
  idade: 30,
  profissao: "Desenvolvedor",
```

```
hobbies: ["leitura", "música", "jogos"],
endereco: {
  rua: "Rua Principal, 123",
  cidade: "São Paulo",
},
saudacao: function () {
  print("Olá, meu nome é " + this.nome);
},
};

print(pessoa.nome); // Saída: "João"
print(pessoa.hobbies[0]); // Saída: "leitura"
pessoa.saudacao(); // Saída: "Olá, meu nome é João"
```

E por aí vai !! No próximo capítulo essa seção irá explorar um pouco sobre variáveis e funções.

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

O MUNDO TE CHAMA

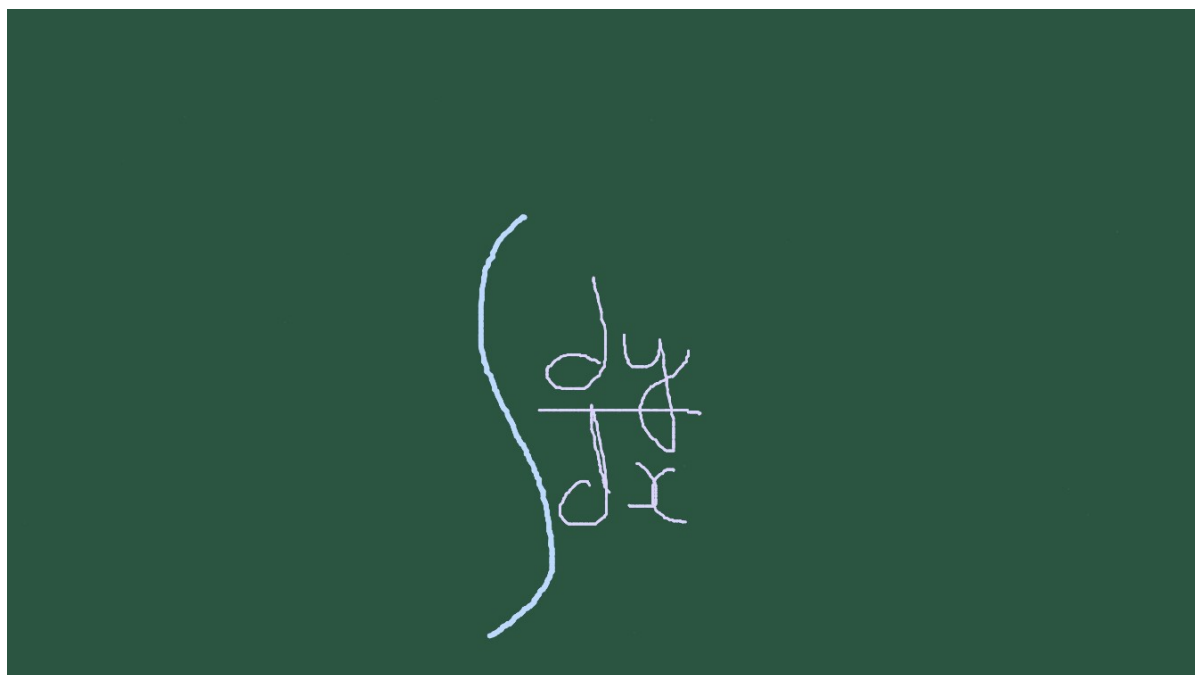


Você já notou que quase tudo que é importante muda com o tempo ? A velocidade aumenta ou diminui, a temperatura sobe e desce, a população cresce, o dinheiro rende (ou some), a energia se transforma. Sem contar aquele tempo que passa rápido quando você está jogando, mas se arrasta quando a semana é de provas.

Independentemente da situação, é desejável medir essas mudanças em tempo real, “capturar” um instantâneo dessas mudanças, ou ainda somar infinitas partes desses instantâneos para obter o todo.

Neste capítulo você verá que essas coisas estão ligadas a taxas de variação (derivadas) que se convertem em quantidades acumuladas (integrais), ou vice-versa. Possum em comum uma base conceitual clara (limites), e estão juntas tanto a funções como a dados coletados no mundo. Assim nasce o Cálculo !

Figura 8.1: A derivada mede o ritmo de mudança em um único instante, enquanto a integral acumula todos esses instantes.



Depois de explorar, você consegue:

- interpretar derivadas como taxas de variação instantânea em fenômenos matemáticos, físicos, químicos, biológicos e econômicos;
- compreender integrais como processos de acumulação, soma contínua e área sob uma curva;
- relacionar função, derivada e integral como três formas complementares de descrever o mesmo fenômeno;
- usar visualizações interativas para explorar limites, aproximações, inclinações e áreas;
- reconhecer cuidados importantes na modelagem, como domínio da função, escala do gráfico e interpretação das unidades;
- conectar cálculo diferencial e integral a situações reais envolvendo movimento, energia, crescimento, consumo e dados experimentais.
- simular situações do Cálculo por meio de aplicações interativas computacionais.

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

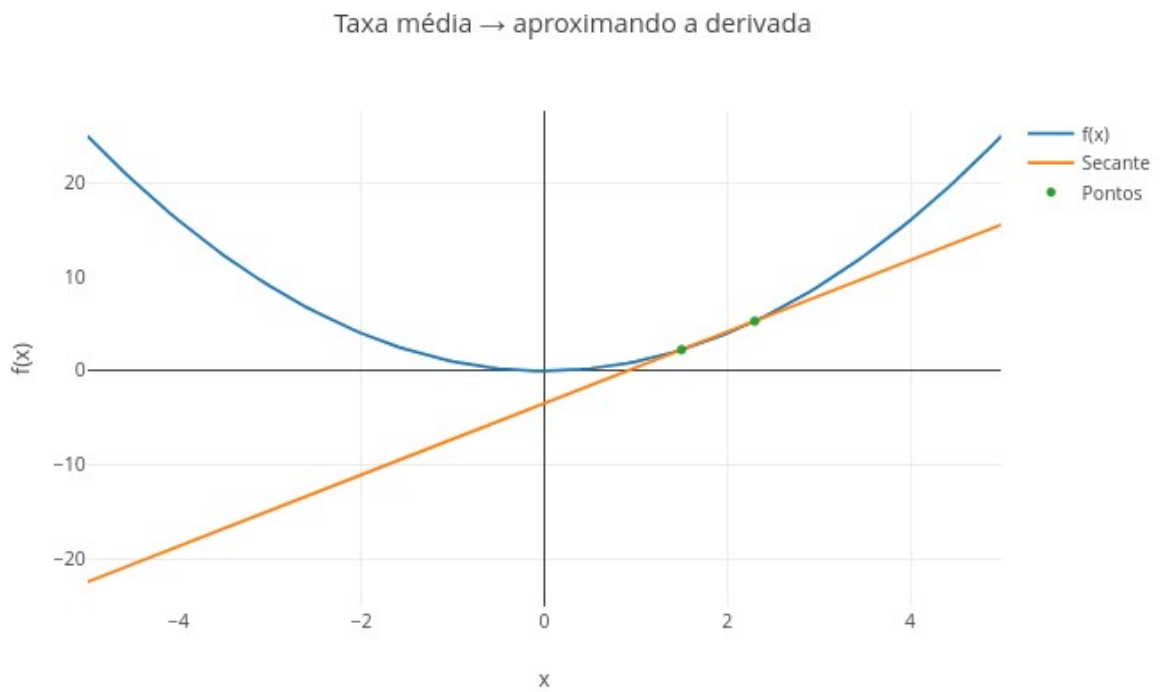
MEXA ANTES DE ENTENDER



8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

Antes de qualquer definição formal, experimente o JSPlotly abaixo sobre taxas de variação.

Figura 8.2: A derivada é o limite da taxa média. Clique com o botão direito neste [LINK](#) e abra em nova aba.



Agite antes de usar

O JSPlotly que inicia esse capítulo mostra como medir mudança em um ponto. O gráfico apresenta uma função $f(x)$, com dois pontos próximos e uma reta que corta a função (secante) ligando esses pontos (taxa média). Observando o gráfico, você pode notar que quando os pontos se aproximam ($h \rightarrow 0$), a taxa média vira taxa instantânea e a secante transforma-se na derivada, a inclinação da curva nesse ponto.

Inicialmente, execute o script com:

```
const funcao = (x) => x*x;  
const x0 = 1.5;  
const h = 0.8;
```

Agora reduza o valor de h e observe o que acontece com a reta secante.

```
const h = 0.4;  
const h = 0.1;  
const h = 0.01;
```

Depois, troque a função e compare os comportamentos. Você pode utilizar algumas, como as que foram apresentadas na seção sobre *JavaScript* do capítulo anterior, como “ $x**3$ ” (o mesmo que x^3), “ $\text{Math.sqrt}(x)$ ” (o mesmo que $\sqrt{x}$), ou a função que segue, por exemplo.

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

```
const funcao = (x) => Math.sin(x);
```

Perceba que medir uma velocidade é achar a mudança exatamente em um ponto. A taxa média mede a variação entre dois pontos, ou seja, “quanto mudou dividido pelo intervalo”. Mas a taxa de mudança em um único ponto é medida quando esses pontos estão tão próximos que você não percebe a distância entre eles, parecendo que são um só. É esse processo que leva à *derivada*. Siga o modo de usar abaixo para o *app*.

1. Rode o script;
2. Observe a curva $f(x)$;
3. Veja os dois pontos marcados;
4. Observe a reta secante;
5. Diminua h ;
6. Veja os pontos se aproximando;
7. Observe a reta mudando de inclinação;
8. Continue diminuindo h ;
9. Perceba que a reta “encosta” na curva;
10. Esse é o comportamento da tangente.

Veja que a tangente ou derivada nada mais são do que a inclinação de uma reta quando os dois pontos se tornam praticamente o mesmo. A ideia do *app* é mostrar de forma intuitiva e geométrica uma primeira noção do Cálculo, a de que uma taxa de variação ocorre quando uma secante (uma taxa média) vira uma tangente (uma taxa instantânea).

Que tal explorar alguns cenários agora ?

1. *Distância grande*. A reta representa bem a inclinação local ?

```
const h = 1.5;
```

Pergunta:

2. *Aproximação*. A reta começa a representar melhor a curva ?

```
const h = 0.3;
```

3. *Quase no ponto*. A reta parece tocar a curva em apenas um ponto ?

```
const h = 0.01;
```

4. *Função diferente*. a ideia funciona para qualquer função ?

```
const funcao = (x) => Math.tan(x);
```

5. *Mudando o ponto*. A inclinação depende do ponto escolhido ?

```
const x0 = -2;
```

Portanto, observe que quando $h \rightarrow 0$, a secante vira tangente. Sua representação clássica como derivada é dada abaixo:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}; \text{ derivada (8.1)}$$

Por dentro do script

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

O script começa definindo a função:

```
const funcao = (x) => x*x;
```

Depois escolhe um ponto:

```
const x0 = 1.5;
```

E uma pequena variação:

```
const h = 0.8;
```

Os dois pontos são:

```
xA = x0  
xB = x0 + h
```

A inclinação da secante é calculada como uma taxa média:

```
m = (yB - yA) / (xB - xA)
```

Depois o script desenha a reta secante. Quando h diminui, essa reta começa a “encostar” na curva, fazendo surgir a derivada nesse limite.

VOLTA PRO MUNDO



8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

Os conceitos do JSPlotly da [Figura 8.2](#) acima aparecem em muitos contextos. Sempre que algo muda, é possível se determinar quão rápido está mudando num tempo específico. O outro “lado da moeda” no Cálculo é a integral, a soma das pequenas partes. Enquanto a derivada mede a mudança, a integral acumula essa mudança. O *app* que segue busca apresentar um pouco essa ideia, também de forma geométrica.

Figura 8.3: Uma integral pode ser entendida como o acúmulo de pequenas contribuições. Clique com o botão direito neste [LINK](#) e abra o app em nova aba.



Agora o aplicativo mostra como calcular a área sob uma curva usando retângulos, um procedimento chamado Soma de Riemann. O gráfico mostra a função $f(x)$ e vários retângulos dispostos ao longo do intervalo de x . A soma total das áreas desses retângulos aproxima o valor da integral da função. Quanto menor o intervalo, maior o número de retângulos e, quanto maior esse número, mais próxima estará a soma das áreas do valor da integral de $f(x)$.

Execute o script com:

```
const funcao = (x) => x*x;  
const a = 0;  
const b = 3;  
const n = 10;
```

Agora aumente o número de retângulos para 20, 50, e 100, e observe como a aproximação melhora.

Depois teste outras funções, como feito para a [Figura 8.2](#), e compare os resultados. Segue um exemplo:

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

```
const funcao = (x) => Math.sin(x);
```

Para retângulos simples, sabemos calcular a área (vide capítulo sobre geometria). Mas, e para curvas ? A solução é dividir a região em partes pequenas, cada pedaço sendo aproximado por um retângulo. Somando todas as áreas temos a integral. Observe que a integral se aproxima da área de uma região com uma borda curva quanto menor for a distância entre os intervalos dessa curva. Ou seja, quanto menor o intervalo, mais precisa fica a aproximação. Segue um modo de usar para o *app*.

1. Rode o script;
2. Observe a curva $f(x)$;
3. Veja os retângulos;
4. Observe o valor da área;
5. Aumente n ;
6. Veja os retângulos ficarem mais finos;
7. Compare a área calculada;
8. Continue aumentando n ;
9. Observe a aproximação melhorar;
10. Teste outra função.

O aplicativo mostra então o que acontece com a soma quando os retângulos ficam cada vez menores. Alguns contextos para você explorar um pouco mais:

1. *Poucos retângulos*. A aproximação é boa ou grosseira ?

```
const n = 5;
```

2. *Mais subdivisões*. Mude agora o valor n para 30 retângulos. A área parece mais próxima da curva ?
3. *Muitas subdivisões*. Agora vá para 100 retângulos. A soma parece estabilizar em um valor ?
4. *Intervalo diferente*. A área depende do intervalo escolhido ?

```
const a = 1;  
const b = 4;
```

5. *Outra função*. O método funciona para qualquer função ?

```
const funcao = (x) => Math.sin(x);
```

A curva representa a função e os retângulos as aproximações locais da área. A soma dessas áreas é a integral. Quanto menor o dx , melhor a aproximação. Traduzindo para o bom português, a integral representa a soma de infinitas partes pequenas. Ou, na forma de uma equação clássica:

$$\int_a^b f(x) dx; \text{ integral(8.2)}$$

Nessa [Equação 8.2](#), o símbolo $\int$ representa a integral entre os limites a e b .

Por dentro do script

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

O script começa definindo a função:

```
const funcao = (x) => x*x;
```

E o intervalo:

```
const a = 0;  
const b = 3;
```

Depois define o número de subdivisões:

```
const n = 10;
```

A largura de cada retângulo é:

```
dx = (b - a) / n;
```

Para cada retângulo:

```
let xi = a + i*dx;  
let yi = funcao(xi);
```

A área é aproximada por:

```
soma += yi * dx;
```

Assim, o script constrói a integral como uma soma.

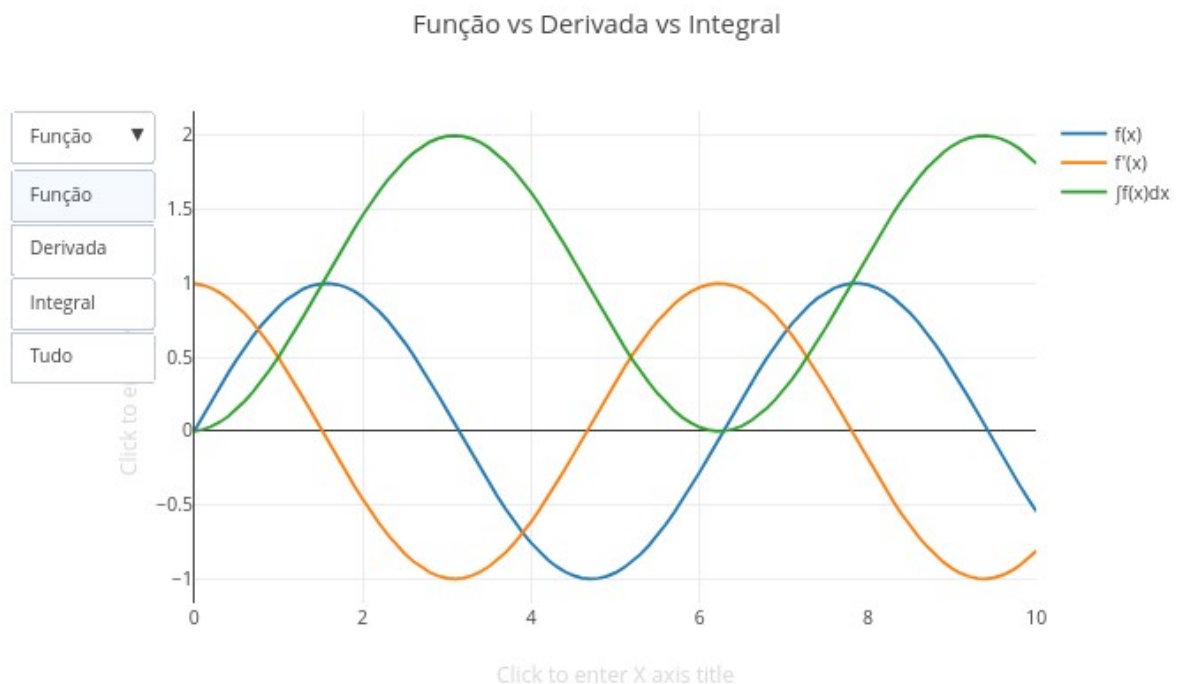
FAZENDO APARECER



Depois de experimentar os JSPlotly das [Figura 8.2](#) (derivação) e [Figura 8.3](#) (integração), você deve ter percebido que se tratam de operações inversas, uma levando à outra, e com a função $f(x)$ no meio. Essa é a base do chamado Teorema Fundamental do Cálculo. Dessa forma, função, derivada e integral “*são farinhas do mesmo saco*”, três formas de se observar uma mesma mudança. Que tal avançar então um bocadinho nessa integração ?

Figura 8.4: Derivar é observar a mudança, enquanto integrar é observar seu acúmulo. Clique com o botão direito neste [LINK](#) e abra o app em nova aba.

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança



Depois de explorar a derivada como taxa de variação e a integral como soma acumulada, o *app* da [Figura 8.4](#) coloca as duas ideias lado a lado. Agora, a mesma função pode ser observada de três jeitos diferentes: seu valor, sua mudança e seu acúmulo.

Agite antes de usar

O script compara três curvas: a função original $f(x)$, sua derivada aproximada $f'(x)$, e sua integral acumulada $\int f(x)dx$. A ideia é que uma função pode ser estudada pelo que ela vale (a função, propriamente dita), por como ela muda (derivada) e pelo que ela acumula (integral). Execute o script com:

```
const f = (x) => Math.sin(x);
```

Use os botões Função, Derivada, Integral, ou Tudo. Depois, teste outras funções:

```
const f = (x) => x*x;  
const f = (x) => Math.cos(x);  
const f = (x) => Math.exp(0.2*x);
```

Observe como a derivada e a integral mudam junto com a função original, mostrando que há uma relação entre uma função, sua taxa de mudança e seu acúmulo. A derivada mostra como a função está mudando em cada ponto, enquanto que a integral mostra quanto a função acumulou desde o início do intervalo. Resumindo, enquanto a *função* mostra o estado atual, a *derivada* mostra a velocidade da mudança e a *integral* o histórico acumulado. Segue um modo de usar o *app*:

1. Rode o script;

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

2. Clique em Função;
3. Observe apenas a curva original;
4. Clique em Derivada;
5. Compare onde a função sobe e onde a derivada é positiva;
6. Clique em Integral;
7. Observe como o acúmulo cresce ou diminui;
8. Clique em Tudo;
9. Compare as três curvas juntas;
10. Troque a função original;
11. Repita a comparação.

Aqui tem uma desfecho bem bacana, que é quando você percebe que a derivada e a integral revelam o que a função sozinha não mostra. Vamos para alguns cenários ?

1. *Função seno*. Por que a derivada parece deslocada em relação à função original ?

```
const f = (x) => Math.sin(x);
```

2. *Função quadrática*. Por que a derivada cresce de forma mais simples que a função ?

```
const f = (x) => x*x;
```

3. *Função cosseno*. Como a derivada muda quando a curva começa no valor máximo ?

```
const f = (x) => Math.cos(x);
```

4. *Crescimento exponencial*. Por que crescimento exponencial também produz derivada crescente ?

```
const f = (x) => Math.exp(0.2*x);
```

5. *Tudo ao mesmo tempo*. Use o botão Tudo. As três curvas contam a mesma história ou histórias diferentes sobre a mudança ?

A função mostra o comportamento original, a derivada mostra a inclinação local, e a integral mostra o acúmulo ao longo do caminho. Quando a função está positiva, a integral tende a crescer. Quando a função está negativa, a integral pode diminuir. Quando a função sobe rapidamente, a derivada fica maior. Quando a função atinge máximos ou mínimos, a derivada se aproxima de zero. Por conseguinte, o cálculo permite olhar para o mesmo fenômeno de formas diferentes: estado, mudança e acúmulo.

Por dentro do script

O script começa definindo a função:

```
const f = (x) => Math.sin(x);
```

Depois percorre o intervalo de 0 a 10:

```
for (let i = 0; i <= 10; i += dx)
```

Em cada ponto, calcula o valor da função:

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

```
let yi = f(i);
```

A derivada é aproximada por uma diferença pequena:

```
let d = (f(i+dx) - f(i)) / dx;
```

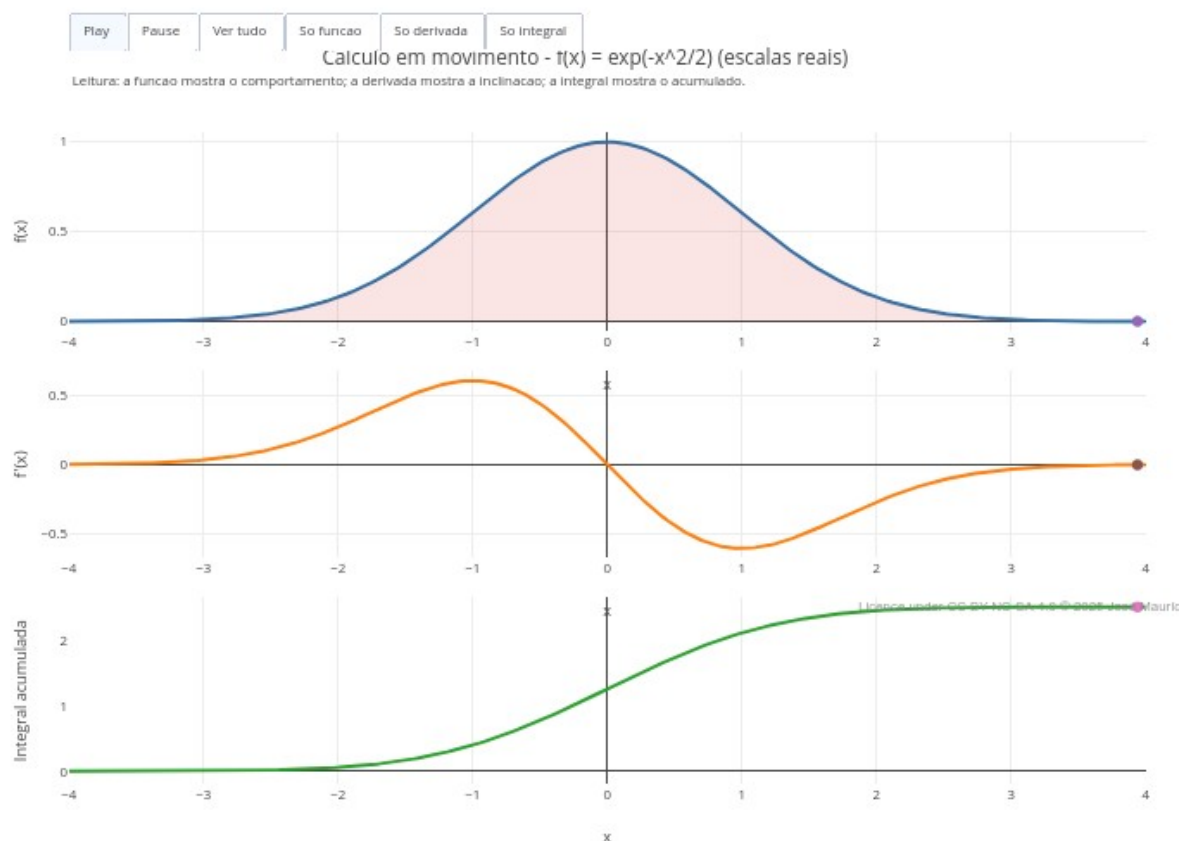
Essa expressão mede quanto a função mudou em um intervalo pequeno. A integral acumulada é construída somando pequenas áreas:

```
soma += yi * dx;
```

Assim, o script combina os dois processos centrais do cálculo, aproximando as inclinações e acumulando as áreas.

Função, derivada e integral, ainda que sejam interligados, tem lá seu grau de abstração para a compreensão dos conceitos envolvidos. Nessa linha, o JSPlotly que segue busca diminuir esse desconforto, apresentando uma *animação* que junta os três objetos de estudo num movimento simultâneo, e permitindo que o cálculo ganhe vida.

Figura 8.5: Função, derivada e integral são três modos de se observar a mesma trajetória. Clique com o botão direito neste [LINK](#) e abra em nova aba.



O script mostra o cálculo como um processo dinâmico, onde é possível se observar simultaneamente a função $f(x)$, sua derivada $f'(x)$, sua integral acumulada, uma partícula se movendo ao longo do eixo x , e a área sendo construída em tempo real.

Agite antes de usar

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

No gráfico, o topo representa a função (estado atual), o meio representa a derivada (inclinação) e a base representa a integral (acúmulo). E a área preenchida mostra a integral sendo construída.

Execute o script e clique em Play. Observe o ponto se movendo na função, a inclinação mudando na derivada e a área crescendo na integral. Agora altere para uma das funções matemáticas abaixo:

```
const tipoFuncao = "sin";  
const tipoFuncao = "poly";  
const tipoFuncao = "exp";  
const tipoFuncao = "gauss";
```

E teste:

```
const normalizar = true;
```

Compare os comportamentos e observe como a função, sua taxa de mudança e seu acúmulo evoluem juntos. À medida que o ponto avança, a função mostra “onde estamos”, a derivada mostra “como estamos mudando” e a integral mostra “quanto já acumulamos”. Isso sugere que derivar e integrar são duas formas complementares de descrever o mesmo processo. Segue um tutorial de uso para o script:

1. Rode o script;
2. Clique em Play;
3. Observe o ponto se movendo na função;
4. Veja a derivada mudando ao longo do caminho;
5. Observe a área sendo construída;
6. Pause a animação;
7. Analise um ponto específico;
8. Troque a função;
9. Compare comportamentos;
10. Ative/desative normalizar;
11. Use os botões “Só função”, “Só derivada”, “Só integral”, ou “Ver tudo”.

E agora alguns cenários “derivados” e “integrados” à cena.

1. *Oscilação (seno)*. Por que a integral sobe e desce ao longo do tempo ?

```
const tipoFuncao = "sin";
```

2. *Crescimento contínuo*. Por que derivada e função crescem juntas ?

```
const tipoFuncao = "exp";
```

3. *Pico central (gaussiana)*. Essa curva é característica da distribuição normal observada na distribuição estatística dos fenômenos naturais. Por que a derivada muda de sinal no centro ?

```
const tipoFuncao = "gauss";
```

4. *Polinômio*. Como a forma da curva afeta a derivada ?

```
const tipoFuncao = "poly";
```

5. *Normalização*. Por que comparar formas pode ser mais útil que comparar valores absolutos ?

```
const normalizar = true;
```

6. *Velocidade da animação*. O que você percebe melhor quando a animação desacelera ?

```
const passoFrame = 2;
```

Veja que o ponto que se move conecta tudo, a posição, a inclinação, e o acúmulo (*histórico*).

Por dentro do script

O script começa escolhendo o tipo de função:

```
const tipoFuncao = "gauss";
```

Depois constrói o domínio:

```
const dx = (xmax - xmin) / (N - 1);
```

A função é definida dinamicamente:

```
function f(x)
```

A derivada é aproximada numericamente:

```
(f(x2) - f(x1)) / (x2 - x1)
```

A integral acumulada é construída por soma trapezoidal:

```
soma += ((yAnterior + yi) / 2) * dx;
```

A animação acontece aqui:

```
frames.push({
```

Por fim, cada frame move o ponto e atualiza a posição na função, o valor da derivada e a área acumulada.

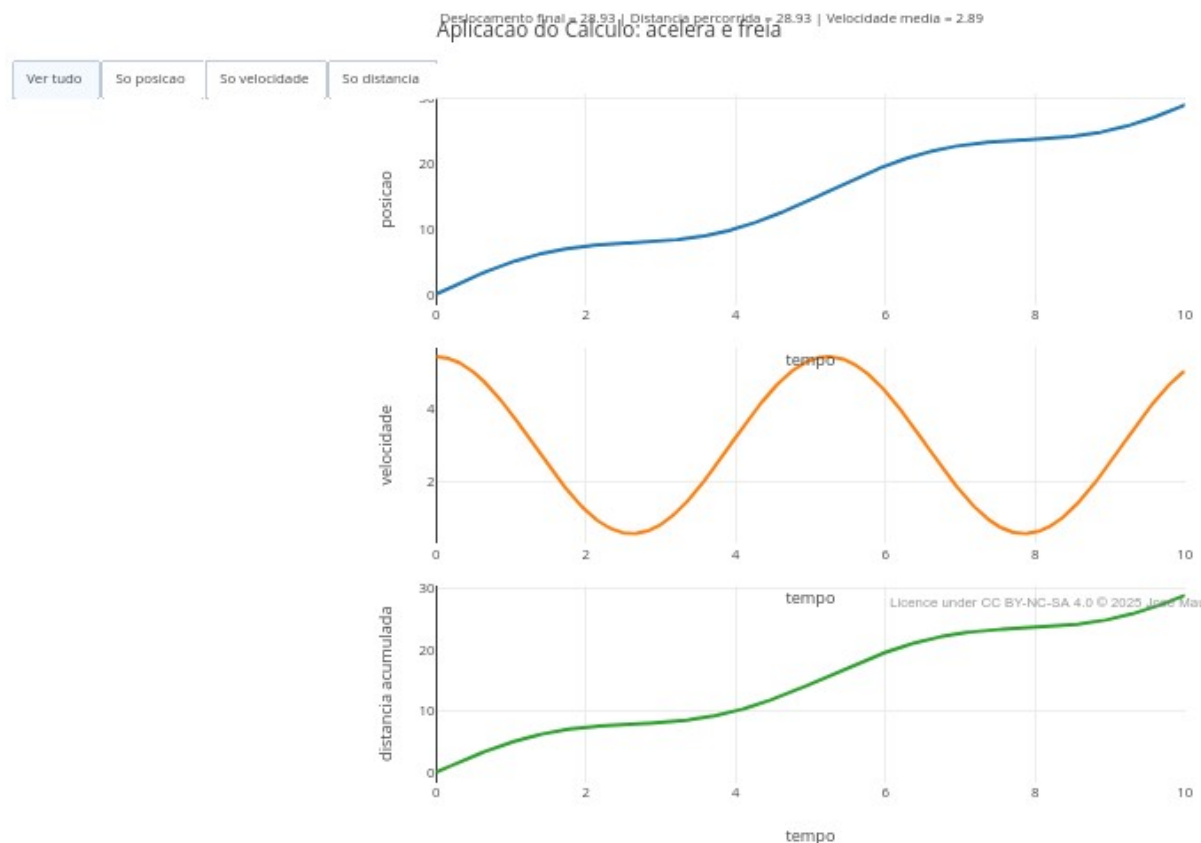
E SE...



8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

Agora é hora de ir além. Mas antes, que tal um último JSPlotly ? Só pra mostrar como a derivada e a integral agem como ferramentas para explicar o mundo real.

Figura 8.6: Velocidade e distância percorrida constituem um revezamento entre derivada e integral. Clique com o botão direito neste [LINK](#) e abra em nova aba.



Esse último JSPlotly da [Figura 8.6](#) mostra o cálculo aplicado para descrever movimento, velocidade e distância.

Agite antes de usar

O script mostra três grandezas fundamentais: posição $s(t)$, velocidade $v(t)$, e distância acumulada. A ideia é que a derivada mede como a posição muda, enquanto que a integral mede quanto se percorreu. Observe como os gráficos mudam juntos e verifique como posição, velocidade e distância estão conectadas.

Execute o script com:

```
const tipoMovimento = "acelera_freia";
const intensidade = 1.0;
const tempoFinal = 10;
```

Agora explore outros movimentos, como:

```
const tipoMovimento = "uniforme";
const tipoMovimento = "acelerado";
const tipoMovimento = "oscilatorio";
```

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

E altere:

```
const intensidade = 2.0;  
const tempoFinal = 15;
```

A posição mostra “onde está”, enquanto que a velocidade mostra “como está se movendo”, e a distância o “quanto já percorreu”.

Matematicamente falando, a velocidade é a derivada da posição e a distância é a integral da velocidade (em módulo). Segue um tutorial rápido para uso do *app*.

1. Rode o script;
2. Observe o gráfico de posição;
3. Veja como ele evolui no tempo;
4. Ative “Só velocidade”;
5. Compare com a inclinação da posição;
6. Ative “Só distância”;
7. Veja o acúmulo ao longo do tempo;
8. Clique em “Ver tudo”;
9. Observe os três juntos;
10. Troque o tipo de movimento;
11. Compare os comportamentos;
12. Aumente a intensidade.

Com algumas experiências acima, você deve verificar que a velocidade explica a forma da posição e da distância. Vão alguns cenários bem “movimentados” agora.

1. *Movimento uniforme*. Por que a velocidade é constante ?

```
const tipoMovimento = "uniforme";
```

2. *Movimento acelerado*. Por que a velocidade cresce ao longo do tempo ?

```
const tipoMovimento = "acelerado";
```

3. *Movimento oscilatório*. Por que a velocidade muda de sinal ?

```
const tipoMovimento = "oscilatorio";
```

4. *Acelera e freia*. Como a combinação de movimentos altera o comportamento ?

```
const tipoMovimento = "acelera_freia";
```

5. *Intensidade maior*. O movimento muda só de escala ou de comportamento ?

```
const intensidade = 2.5;
```

Matematicamente, a velocidade pode ser formulada como na [Equação 8.3](#), e a posição, como na [Equação 8.4](#) abaixo:

$$v(t) = \frac{ds}{dt} \quad (8.3)$$

e

$$d(t) = \int_0^t |v(\tau)| d\tau \quad (8.4)$$

Por dentro do script

O script começa definindo a posição:

```
function s(t)
```

Cada tipo de movimento cria um comportamento diferente. Depois calcula a velocidade numericamente:

```
v = (pos[i + 1] - pos[i - 1]) / (2 * dt);
```

Isso aproxima a derivada. A distância é calculada acumulando a velocidade:

```
distanciaAcumulada += ((|v1| + |v2|)/2) * dt;
```

Isso corresponde à integral da velocidade em módulo. No final, o script calcula o deslocamento total, a distância percorrida e a velocidade média, tudo aparecendo como anotação no gráfico.

Bom ! A gente só não pode se esquecer de que essa é aquela seção cujo título remonta a “como divagar sem culpa”. Então, segue uma “penca” dessas divagações !!

- E se a velocidade de um carro não fosse mostrada no painel, mas você tivesse que deduzi-la apenas observando como a posição muda com o tempo ?
- E se um motorista acelerasse suavemente — como isso apareceria na derivada do movimento ?
- E se dois trajetos diferentes levassem ao mesmo destino, mas com velocidades completamente distintas ao longo do caminho ?
- E se a distância percorrida fosse muito maior que o deslocamento final — o que isso diria sobre o percurso ?
- E se o gráfico da posição fosse conhecido, mas a velocidade fosse um mistério — você conseguiria reconstruí-la ?
- E se a área sob o gráfico de velocidade fosse a única informação disponível — você conseguiria descobrir o caminho percorrido ?
- E se o movimento fosse irregular, com acelerações e frenagens constantes — o cálculo ainda daria conta ?
- E se o tempo fosse observado em câmera lenta — você perceberia melhor a relação entre função, derivada e integral ?
- E se cada instante do movimento fosse uma fotografia — como reconstruir o filme completo ?
- E se o cálculo fosse, na verdade, uma linguagem para descrever mudanças contínuas ?
- E se, em outro planeta, a gravidade fosse diferente — como isso mudaria os gráficos de posição e velocidade ?
- E se na estrela Betelgeuse o tempo fluísse de forma diferente — a derivada ainda faria sentido ?

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

- E se o espaço fosse curvo — como interpretar posição, velocidade e área ?
- E se o movimento não fosse em linha reta, mas em múltiplas dimensões — como generalizar esses gráficos ?
- E se a própria noção de “taxa de variação” mudasse — o cálculo continuaria válido ?
- E se a função descrevesse não posição, mas energia — o que a derivada representaria ?
- E se a integral não fosse área, mas quantidade acumulada de matéria ou informação ?
- E se partículas subatômicas não tivessem trajetórias definidas — o que significaria derivar algo ?
- E se o universo fosse discreto, e não contínuo — o cálculo ainda funcionaria ?
- E se o tempo tivesse começo e fim — como ficariam as integrais ?
- E se o crescimento de uma população seguisse uma função — o que sua derivada revelaria ?
- E se a taxa de reação química variasse ao longo do tempo — como prever o total transformado ?
- E se o coração fosse um gráfico — qual seria sua derivada ?
- E se o cérebro funcionasse como um sistema de sinais — o cálculo ajudaria a interpretá-lo ?
- E se o mercado financeiro fosse analisado como uma função — o que a derivada indicaria ?
- E se o aprendizado fosse acumulativo — qual seria sua integral ?
- E se tudo que muda no mundo pudesse ser descrito por uma função ?
- E se tudo que muda pudesse ser medido por uma derivada ?
- E se tudo que se acumula pudesse ser descrito por uma integral ?
- E se o cálculo não fosse apenas matemática, mas uma forma de enxergar o próprio universo ?

MESMO PADRÃO, OUTROS MUNDOS



8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

Tanto a derivada como a integral aparecem em cenários do mundo real de formas tão distintas que não dá para abraça-las além de uma pequena fração. Derivada, por exemplo, está presente nas medidas de velocidade instantânea, como aquela que o radar mede nas estradas, no crescimento populacional, na variação de temperatura, nas taxas de crescimento da Economia, e nos movimentos e forças da Física.

A integral, por sua vez, se apresenta na distância percorrida a partir da velocidade, na energia acumulada, na área sob gráficos experimentais, na Economia (acúmulo de lucro) e na Biologia (crescimento ao longo do tempo). Sempre que você acumula algo ao longo de um intervalo, sem perceber, está fazendo uma integral.

Função, derivada e integral se conectam também em diversos aspectos do mundo, como na aceleração e frenagem de carros, em ondas e vibrações, no fluxo de energia e em reações químicas ao longo do tempo.

POR DENTRO DA FERRAMENTA



Separação de comandos, inserção de comentários, e boas práticas

Os comandos em *JS* são separados por *ponto e vírgula*, “;”. Já os *comentários* constituem trechos não interpretados, comumente utilizados em programação. Comentários são tremendamente úteis quando se deseja colocar um título no *script* de código, por exemplo, para explicar quem é quem nas variáveis ou determinada ação no código, e apontar uma futura melhoria, entre muitos (lista *ToDo*, ou “para fazer”). Em *JS*, os comentários são alternativamente inseridos por “//” antes de cada comando, ou por “/.../”, para comentários mais extensos.

Códigos em *JS* podem ser inseridos também junto à páginas web (HTML) com seus comandos intercalados pela notação que segue (etiquetas ou *tags*):

Uma característica interessante no *JS* é que tanto faz colocar ou não espaços entre os comandos, pois a linguagem não interpreta esses espaços. Contudo, dentro das *boas práticas em programação*, é aconselhável separar blocos de comandos por *identação* de 2 espaços, de modo a facilitar a leitura do código. Outro cuidado que deve ser apreciado diz respeito ao uso de fonte maiúscula ou minúscula. *JS* interpreta diferentemente termos em caixa alta ou baixa do teclado, então é bom prestar atenção pra não incorrer em erro na interpretação.

Declaração de variáveis

Entre os diversos tipos de variáveis, duas são frequentemente utilizadas: *const* e *let*. A variável *const* é utilizada para se atribuir um valor fixo a uma variável no código, enquanto a variável *let* é empregada quando se deseja reatribuir um valor a ela, por meio de algum cálculo ou laço iterativo. Exemplificando, quando se deseja variáveis de controle que mudam ao longo do tempo (*sliders*, passos, intervalos).

Obs: em versões antigas de *JS* é também empregado *var*, hoje em desuso.

“Botando a mão na massa”

Como esse treinamento envolve instruir o algoritmo com uma *entrada* para que sua interpretação resulte numa *saída*, aqui propomos essa última pelo comando *print*, como segue.

```
const a = -0.2;  
print(a)
```

Para esse treinamento, use o *JSPlotly* personalizado do capítulo anterior. Assim, teste a atribuição da variável *const* e do comando *print* por cópia do trecho acima e cola no campo de texto **ÁREA PARA DIGITAR COMANDOS** do script. Existem outros comandos de saída para *JS*, como *console.log()* e *document.body.appendChild()*, mas não utilizáveis pelo *JSPlotly*.

Pode-se atribuir também um texto para apresentar a constante, veja:

```
const a = -0.2;
print("constante:", a)

const a = 5; // variável constante (não se altera ao longo do código)
const b = 10; // variável que pode alterar seu valor no código
print(a*b)
```

Outro exemplo, com inserção de texto antes do resultado:

```
let a = 5;
const b = 10;

let resultado = a + b;
print("Soma de a + b:", resultado);
```

const versus *let*

Pode parecer que *const* e *let* são distintas apenas na aparência, já que operam de modo intercambiável. Contudo, para explicitar a diferença entre os dois comandos, apague o ecrã gráfico (*clean*) e substitua a linha de código da seção anterior pela que segue na **ÁREA PARA DIGITAR COMANDOS** do script:

```
let passo = 0.5; // usuário pode alterar
let x0 = 0; // domínios dinâmicos
print(passo, x0)

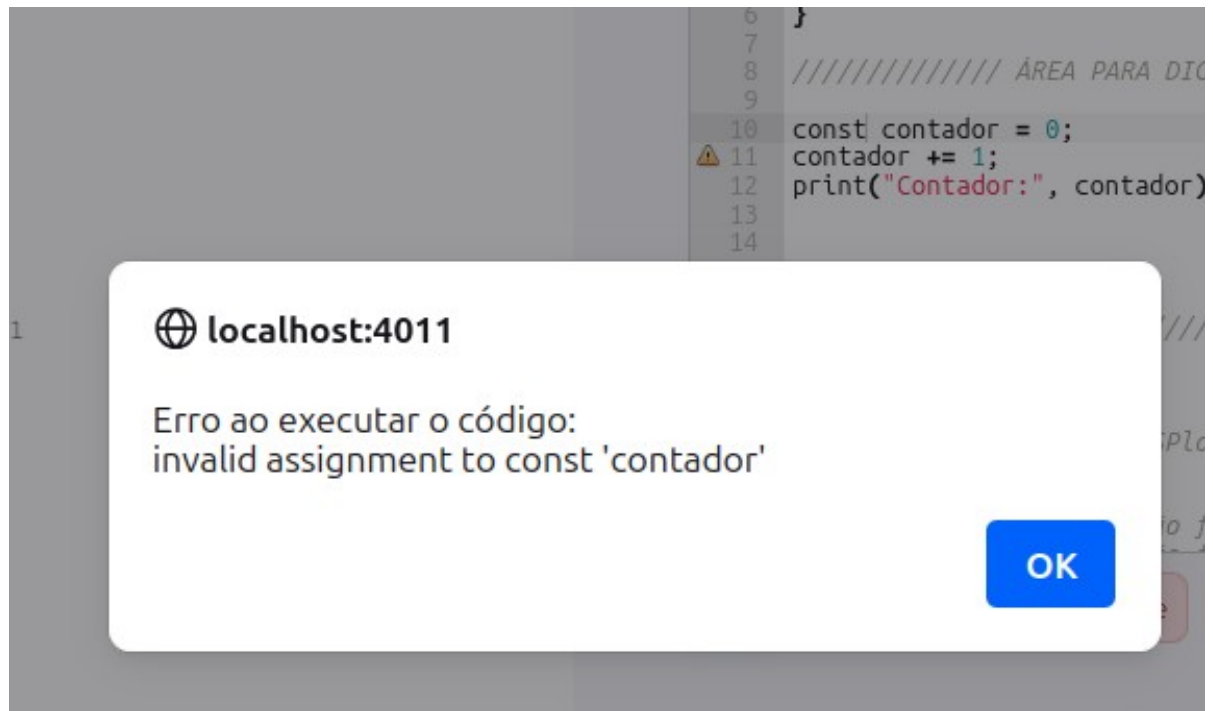
// ... mais tarde:
passo = 0.2; x0 = 30;
print(passo, x0)
```

Aparentemente, o resultado sugere que na prática nada muda, sendo a diferença mais centrada na clareza e segurança do código. Mas não é bem assim. Experimente agora trocar o campo pra executar o código abaixo:

Agora substitua a variável do “*contador*” para *const* ao invés de *let*. O resultado incorrerá no erro abaixo:

Figura 8.7: Erro gerado quando se substitui a variável let por const num trecho de reatribuição de valor.

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança



Resumindo: para constantes, use *const* e para variável reatribuível use *let*.

Tipos de dados

Como é comum também em outras linguagens, *JavaScript* possui alguns tipos comuns para definição de dados. Para uso de *Plotly.js*, são bem significativos:

Seguem alguns exemplos práticos. Experimente trocar cada um na saída *print*:

```
// Numbers:  
let comprimento = 116;  
let peso = 75;  
  
// Strings:  
let color = "Yellow";  
let lastName = "Johnson";  
  
// Booleans  
let x = true;  
let y = false;  
  
// Object:  
const membro = {firstName:"John", lastName:"Doe"};  
  
// Array object:  
const carros = ["Saab", "Volvo", "BMW"];  
  
// Date object:  
const datas = new Date("2022-03-25");
```

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

```
// Saídas:  
print(comprimento)
```

Constantes

Por vezes, é necessária a inserção de constantes matemáticas em equações para construção de gráficos ou para fazer cálculos. Seguem alguns exemplos. Para a saída no *JSPlotly* customizado abaixo, basta o bom e velho `print` que segue ao também bom e velho *clean*.

```
Math.PI - valor de (pi), aproximadamente 3.14159  
Math.E - valor da constante de Euler, aproximadamente 2.71828  
Math.LN2 - logaritmo natural de 2, aproximadamente 0.693  
Math.LN10 - logaritmo natural de 10, aproximadamente 2.302  
Math.LOG2E - logaritmo de e na base 2, aproximadamente 1.442  
Math.LOG10E - logaritmo de e na base 10, aproximadamente 0.434  
Number.MAX_VALUE - maior número positivo  
Number.MIN_VALUE - menor número positivo...
```

Para constantes naturais, contudo, é necessário defini-las antecipadamente, como em:

```
// Definindo constantes físico-químicas comuns  
const Avogadro = 6.02214076e23; // Número de Avogadro (mol-1)  
const Boltzmann = 1.380649e-23; // Constante de Boltzmann (J/K)  
const Planck = 6.62607015e-34; // Constante de Planck (J·s)  
const gas = 8.31446; // Constante dos gases ideais (J/(mol·K))  
const luz = 299792458; // Velocidade da luz no vácuo (m/s)
```

Funções

Funções constituem um bloco de códigos para efetuar alguma ação. Em *JavaScript* uma função possui a seguinte sintaxe:

```
function nome(parâmetro1, parâmetro2, parâmetro3) {  
  // código para execução  
}
```

Seguem alguns exemplos de funções para testes.

```
function soma(a) { // define o nome e os "parâmetros" da função  
  return a + a // define os argumentos da função (no caso, a*a),  
               // pra retorna dos valores  
}  
  
// Teste  
let x = soma(7); // aplica parâmetros  
print(x) // saída da função
```

Obs: O comando *return*, por vezes omitido, se apresenta útil quando se deseja que o “retorno” seja concluído somente ao final do fluxo do script, sem resultados intermediários.

```
function expoente(a, b) { // define o nome e os "parâmetros" da  
  função
```

8. Cálculo Diferencial e Integral - Quando o mundo começa a mudar, e você aprende a medir essa mudança

```
    return a ** b; // define os argumentos da função (no caso, a^b),
                  // pra retorna dos valores
}

// Teste da função
let x = expoente(4, 3); // aplica parâmetros
print(x);               // saída da função

function indexa(vetor){
    return vetor + 1
}

var vetor = [1, 2, 3, 4, 5];
print(indexa(vetor))

function Celsius(Fahrenheit) {
    return (5/9) * (Fahrenheit-32);
}

let conversao = Celsius(115);
print(conversao)
```

Funções de seta (*arrow functions*)

Como visto anteriormente, uma notação alternativa é a função *arrow*, que abrevia a sintaxe de funções. Segue um exemplo:

```
const multiplica = (a, b) => a * b;
print(multiplica(4, 6));
```

Se gostou até aqui...não perca as **estruturas de controle** do capítulo final desta obra.